GNN-Based Adaptive Coin Policy Learning for Discrete-Time Quantum Walks

Angel D. Colon
SUNY Polytechnic Institute
Utica, NY, USA

Advisor: Chen-Fu Chiang
SUNY Polytechnic Institute
Utica, NY, USA

***Abstract*—Discrete-time quantum walks (DTQWs) on graphs are a foundational model for coherent transport, spatial search, and mixing, yet practical performance depends critically on the choice of the coin operator that locally mixes coin states before each shift. Standard designs typically fix a global coin (e.g., Hadamard/Grover) or hand-engineer analytic schedules for narrow symmetry classes, leaving a gap between theory-driven constructions and the heterogeneous, irregular graphs encountered in realistic algorithmic and network settings. This work introduces *Adaptive Coin Policy Learning* (ACPL), a learning-based framework that treats coin selection as a graph-structured control problem: a permutation-equivariant policy π_ω maps a graph instance, node features (including optional positional information), and normalized time to vertex- and time-local coin parameters, which are lifted into unitary operators $C_v(t) \in U(d_v)$ by construction and composed into DTQW step operators $U_t = S\big(\oplus_{v\in V} C_v(t)\big)$. We develop a procedural episode generator (dataset-as-distribution) with manifest-locked determinism, pre-registered splits, and CI-style evaluation, enabling reproducible benchmarking across canonical regimes: state transfer, marked-node search, fast mixing (TV-to-uniform), and robust transport under disorder. Empirically, ACPL yields substantial improvements over fixed-coin baselines on fast mixing and on regular-graph transfer, while ablation studies isolate the roles of spatial adaptivity, temporal choreography, and positional information and validate permutation-consistency as a non-negotiable sanity check. We further articulate scalability limits in graph size and horizon, and outline extensions toward hardware-realistic constraints and noise models, multi-objective control, multi-particle walks, and continuous-time limits via Trotterized connections. Overall, ACPL converts DTQW coin design from a graph-specific handcraft into a systematic, generalizable, and auditable synthesis procedure for quantum-walk-based algorithms and transport. Code and reproducibility artifacts are available at https://github.com/Colonad/acpl-qwalk.git**

***Index Terms*—discrete-time quantum walk, coined quantum walk, graph neural networks, permutation equivariance, quantum control, robust optimization, mixing time, spatial search, state transfer, reproducible machine learning**

I. Introduction

DISCRETE-TIME coined quantum walks (DTQWs) provide a physically grounded and algorithmically expressive model of coherent transport on graphs, with canonical applications spanning marked-vertex search, state transfer, and mixing phenomena. In the coined DTQW formalism, the evolution at each step is driven by two structured ingredients: (i) a *local coin* acting on the outgoing "ports" at each vertex, and (ii) a *shift* that permutes amplitudes along graph arcs according to the underlying wiring. Concretely, if the walk is represented in the arc basis, one step takes the form

$$\boldsymbol{\psi}_{t+1} = \boldsymbol{S}\,\boldsymbol{C}(t)\,\boldsymbol{\psi}_t, \tag{1}$$

where $\boldsymbol{C}(t)$ is block-diagonal (a direct sum of local coins) and $\boldsymbol{S}$ is a permutation unitary determined solely by the reverse-arc pairing (flip–flop shift). This separation is not merely notational: the shift operator depends only on the graph connectivity and port map, while all task-specific controllability is mediated through the choice of coins. [1]–[3]

Why coins are the natural control knob. Hand-designed coins such as Hadamard (for degree 2) and Grover-style diffusion coins (for regular degrees) are historically successful but inherently *task-agnostic*. They typically encode a uniform "mixing" bias that may be suitable for a specific family (e.g., lines, grids, hypercubes) and a specific objective (e.g., a particular search oracle), yet they do not adapt to heterogeneous degree, local geometry, obstacles, marked roles, or disorder. This becomes especially problematic for defendable claims about (i) generalization across graph sizes or families, and (ii) robustness under realistic perturbations such as phase disorder and dephasing. [4], [5]

Adaptive Coin Policy Learning (ACPL). This work studies coin design through the lens of modern machine learning: rather than selecting a single fixed coin family a priori, we *learn* a policy that chooses local coins adaptively as a function of graph structure, node roles/geometry, and time. Formally, let $G = (V, E)$ be an undirected graph equipped with a port labeling (equivalently, a consistent arc index and reverse-arc map). Denote by d_v the degree of vertex v, and let the arc Hilbert space be

$$\mathcal{H} \;=\; \mathrm{span}\{|v, a\rangle : v \in V,\; a \in [d_v]\}, \tag{2}$$

so that $\dim(\mathcal{H}) = \sum_{v\in V} d_v = |E^{\rightarrow}|$ equals the number of directed arcs. A local coin at vertex v is a unitary $\boldsymbol{C}_v(t) \in U(d_v)$ acting on $\mathrm{span}\{|v, a\rangle\}_{a=1}^{d_v}$, and the global coin is the direct sum

$$\boldsymbol{C}(t) \;=\; \bigoplus_{v\in V} \boldsymbol{C}_v(t). \tag{3}$$

The shift $\boldsymbol{S}$ acts by reversing arcs (a permutation matrix), implying $\boldsymbol{S}^\dagger = \boldsymbol{S}^{-1} = \boldsymbol{S}$ and $\boldsymbol{S}^2 = \boldsymbol{I}$. [2]

ACPL learns a *graph- and time-conditioned* policy

$$\pi_{\boldsymbol{\omega}} :\; (G, \boldsymbol{X}, t, \mathsf{task}) \;\mapsto\; \{\boldsymbol{\theta}_{v,t}\}_{v\in V} \;\mapsto\; \{\boldsymbol{C}_v(t)\}_{v\in V}, \tag{4}$$

where $\boldsymbol{X} \in \mathbb{R}^{|V|\times F}$ are node features (e.g., degree, Laplacian positional encodings for irregular graphs, coordinates for grids, or bitstrings/Hamming weights for hypercubes), and $\boldsymbol{\theta}_{v,t}$ are coin parameters emitted per node and per time step. The policy is instantiated as a permutation-equivariant graph neural network (GNN) encoder followed by a temporal controller (e.g., GRU/Transformer) and a nodewise coin head. Crucially, we map parameters to *unitaries by construction*: for $d_v = 2$ we use an $SU(2)$ parameterization, while for $d_v > 2$ we use exponential/Cayley-type lifts (or equivalent retractions) to guarantee $\boldsymbol{C}_v(t) \in U(d_v)$ for all v, t. [3]

Tasks and evaluation regime. The learned policy is optimized end-to-end through the DTQW simulator using task-specific objectives and is evaluated across multiple algorithmic tasks: *(i)* fast mixing (approach the uniform distribution quickly), *(ii)* marked-node search (concentrate probability mass on a marked set Ω), *(iii)* state transfer (maximize terminal probability at a target vertex), and *(iv)* robust transport under disorder (maintain transfer/search performance under edge-phase and/or coin dephasing noise). [4], [5] To support rigorous, committee-proof claims, evaluation is structured around manifest-driven procedural episode generation, explicit IID/OOD splits, standardized baselines, targeted ablations, and robustness sweeps over disorder strength. In particular, we include ablations that isolate the causal role of positional information (NoPE), spatial adaptivity (GlobalCoin), temporal adaptivity (TimeFrozen), and permutation consistency (NodePermute). [**?**], [36]

A. Contributions

This work makes the following contributions:

1) **A unified ML formulation of coin design as policy learning.** I recast coin selection in coined DTQWs as a graph-conditioned, time-conditioned control problem in which the policy outputs per-node, per-time coin parameters $\{\boldsymbol{\theta}_{v,t}\}$ that determine local unitary coins $\{\boldsymbol{C}_v(t)\}$. The resulting framework makes "learned interference choreography" the central optimization object, rather than relying on a small set of hand-designed coins.
2) **Unitary-by-construction coin parameterizations for variable-degree graphs.** I provide a parameterization pipeline that guarantees valid unitary local coins across heterogeneous degrees: $SU(2)$ for $d = 2$ and exponential/Cayley-type lifts (or equivalent retractions) for $d > 2$. This ensures the learned policy respects quantum mechanical constraints at every training step, preventing non-unitary drift that would invalidate the dynamics.
3) **A differentiable DTQW simulator pipeline suitable for end-to-end learning.** I implement (and validate) an end-to-end simulator consistent with the arc-basis DTQW semantics: block-diagonal application of local coins and a flip–flop shift implemented as an arc permutation derived solely from the port map / reverse-arc pairing. This structure supports backpropagation through time (BPTT) for gradient-based learning and also admits RL-style alternatives when objectives are non-smooth.
4) **Manifest-driven procedural dataset / episode generator with explicit split semantics.** Rather than treating "the dataset" as a static file, I define it as a distribution of episodes generated from a manifest of graph families, sizes, horizons, tasks, and noise models. Episodes are deterministically regenerable from 64-bit seeds, enabling leak-free IID/OOD split definitions, exact reproducibility, and robust auditing of experimental claims.
5) **CI-style evaluation: baselines, ablations, and robustness sweeps.** I provide a committee-defendable evaluation protocol that combines: (i) fixed-coin algorithmic baselines, (ii) architecture and symmetry ablations (NoPE, GlobalCoin, TimeFrozen, NodePermute), and (iii) robustness sweeps over disorder parameters with standardized reporting artifacts (tables/figures and summary statistics). This makes it possible to attribute gains to specific architectural choices and to quantify failure modes under perturbations.
6) **Code provenance and originality auditability.** I separate what is standard prior art (DTQW formalism, PyTorch/PyG primitives) from what is newly implemented for ACPL (policy wiring, simulator integration, manifest-driven episode generation, evaluation scaffolding, and reproducibility tooling), and I document software and bibliography attributions to ensure the project is transparent and academically defensible.

B. Overview and paper organization

Fig. 2 summarizes the procedural episode generator used throughout this work, while Fig. 3 summarizes the end-to-end ACPL pipeline from episode specification to policy inference, DTQW simulation, and learning updates. The remainder of this paper is organized as follows: Section III introduces DTQW fundamentals (state space, coins, shift, measurement) and the task objectives; Section **??** presents the ACPL policy architecture (GNN encoder, temporal controller, and unitary coin maps); Section V formalizes the procedural dataset, manifest schema, and IID/OOD split protocol; Section **??** details experimental setup, baselines, and reporting templates; Section VIII presents results across transfer, search, mixing, and robustness; finally, Section XI discusses limitations and future directions.

II. Related Work

This section situates Adaptive Coin Policy Learning (ACPL) within four intersecting literatures: (i) discrete-time quantum walks (DTQWs) and coin design, (ii) quantum-walk algorithms for search, transport, and mixing, (iii) quantum control and learning-based synthesis of quantum dynamics, and (iv) graph representation learning and permutation-equivariant neural architectures. Our aim is not an exhaustive survey, but a precise map from prior assumptions to the methodological gap ACPL addresses: *learned, graph-conditional, unitary-by-construction, time-dependent coin synthesis that generalizes across graph instances under reproducible evaluation*.

A. Coined discrete-time quantum walks and coin operator design

a) Standard coined DTQW formalism.: The coined DTQW step is typically written as $U = SC$, where S is a shift/permutation on directed edges (arcs) and C is a coin unitary applied locally at vertices. Many canonical constructions use a *fixed* coin family: Hadamard coins for degree-2 structures and Grover diffusion-like coins for regular graphs, with variants that incorporate phase shifts or marked-vertex reflections for search. In these works, the coin is often chosen for symmetry reasons (translational invariance on lattices, regularity, or high symmetry groups), which yields analytic tractability but limits applicability to irregular graphs and heterogeneous degree distributions.

b) Gap addressed by ACPL.: Classical DTQW theory provides powerful tools for analyzing specific walk operators, but it does not directly solve the inverse problem: *Given a graph instance and a task objective, synthesize a good coin schedule automatically.* ACPL targets this inverse design problem by representing the coin selection as a policy π_ω conditioned on graph structure and time, while enforcing $C_v(t) \in U(d_v)$ by construction.

B. Quantum-walk algorithms: search, hitting, transport, and mixing

a) Spatial search and marked-vertex amplification.: Quantum walk search is a central algorithmic use case where the walk is modified by marking operations (oracle-style phase flips or coin modifications) to amplify probability mass on a marked set. Much of the literature focuses on asymptotic scaling and the existence of speedups on particular graph families (complete graphs, hypercubes, lattices) or under spectral assumptions. These results often rely on highly structured coins and walk operators whose eigenspaces can be characterized.

b) State transfer and quantum transport.: Transport objectives (maximizing terminal probability at a target node, or achieving high-fidelity transfer) are closely related to controllability and interference engineering. While certain graphs admit near-perfect transfer under carefully engineered dynamics, performance is sensitive to graph geometry, spectrum, and phase matching. In practice, the design of a transfer schedule typically requires either analytic insight for a specific graph family or numerical optimization on a per-instance basis.

c) Mixing and convergence to uniformity.: Mixing in quantum walks differs fundamentally from classical mixing due to unitarity: instantaneous distributions oscillate and need not converge. Consequently, mixing is often defined through time-averaged distributions or by measuring distance-to-uniform at selected horizons. Prior work proposes families of coins that empirically improve spreading on lattices or regular graphs, but these are rarely optimized end-to-end for a given instance distribution.

d) Gap addressed by ACPL.: Across these algorithmic regimes, the common pattern is that the coin/operator is typically hand-selected or analytically derived for a narrow class. ACPL provides a single, learnable framework for *instance-conditional* coin synthesis that can be trained on distributions of graphs and tasks, then applied to unseen graphs while keeping the evaluation contract fixed (manifest-locked, CI-style).

C. Quantum control and learning-based synthesis of quantum dynamics

a) Optimal control viewpoint.: From a quantum control perspective, choosing coins $\{C_v(t)\}$ is selecting time-indexed control unitaries that shape the terminal distribution. Traditional approaches treat the control variables as continuous parameters optimized by gradient-based methods (e.g., GRAPE-like methods, adjoint-based gradients) or by heuristic search, typically on a fixed physical system and a fixed objective. These methods can yield high-quality controls but are expensive to rerun for each new graph instance and may not generalize without explicit meta-learning.

b) Learning-to-control (policy learning).: Modern learning-based control methods replace per-instance optimization with policy learning: a neural network learns a mapping from system descriptions to control sequences. ACPL follows this paradigm, but in a structured way tailored to DTQWs: (i) the policy is permutation-equivariant with respect to node relabeling, (ii) the action is a family of local coins lifted to $U(d_v)$ by construction, and (iii) the objective is evaluated on the reduced position state, which is operationally meaningful for measurement.

c) Gap addressed by ACPL.: A key novelty is the combination of *graph-conditioned meta-control* with *unitary-by-construction local actions* and *task suites that probe distinct interference regimes* (transfer/search/mixing/robustness) under a single reproducible pipeline. This distinguishes ACPL from both classical quantum optimal control (which typically optimizes a fixed system) and generic RL control (which often lacks symmetry guarantees and unitarity-preserving action maps).

D. Graph neural networks and permutation-equivariant architectures

a) Message passing and inductive bias.: Graph neural networks (GNNs) are the de facto standard for learning on combinatorial structures due to permutation equivariance: relabelling vertices permutes embeddings but does not change the represented function's semantics. This symmetry aligns exactly with the physical relabeling symmetry of DTQWs: node indices should not matter, only graph structure and labeled roles (source/target/marked nodes).

b) Positional information.: Because many graphs are non-Euclidean and may have automorphisms, pure message passing can be insufficient to disambiguate roles that depend on global geometry. The literature proposes positional encodings (e.g., Laplacian eigenvectors, random-walk features) to inject global structural signals. ACPL inherits this idea: positional encodings are included when the task/graph family benefits, and their necessity is tested via the NoPE ablation to avoid over-reliance on fragile global coordinates.

c) Gap addressed by ACPL.: While GNNs are widely used for node/edge prediction and graph-level regression/classification, ACPL uses GNNs as *control synthesizers*: the output is not a discrete label but a physically constrained operator schedule. This repurposes equivariant representation learning as a tool for quantum-walk operator design.

E. How ACPL differs: a short synthesis

We summarize the distinguishing features of ACPL as the conjunction of the following properties:

- **(R1)** Policy learning over distributions of graph instances (meta-control), not per-instance tuning.
- **(R2)** Permutation-equivariant graph conditioning, aligned with DTQW relabeling symmetry.
- **(R3)** Unitary-by-construction local action map $\Phi : \theta_{v,t} \mapsto C_v(t) \in U(d_v)$.
- **(R4)** Unified benchmark suite probing transport, search, mixing, and robustness under a single pipeline.
- **(R5)** Reproducibility contract: manifest-locked episodes, deterministic regeneration, CI-style evaluation and ablations.

a) Predicate-logic statement of the intended novelty.: Let $\mathcal{D}$ be a distribution over graph-task episodes and let Π be a set of admissible control laws. ACPL targets the existence of a learned $\pi_\omega \in \Pi$ such that:

$$\exists\, \pi_\omega \in \Pi\ \forall\, \epsilon \sim \mathcal{D}: \quad \pi_\omega(\epsilon) \text{ produces a valid unitary schedule} \quad \wedge \quad \mathbb{E}\big[f(\rho_T^{(P)}(\pi_\omega, \epsilon))\big] \text{ is improved over fixed-coin baselines.}$$

with all claims reported under fixed, auditable evaluation manifests. In this sense, ACPL is best viewed as a reproducible, symmetry-respecting learning framework for *operator synthesis* in DTQW-based computation and transport.

III. Physical and Mathematical Background

A. Quantum-mechanical preliminaries (states, dynamics, and measurement)

The operational substrate of this work is standard (finite-dimensional) quantum mechanics. A closed quantum system is described by a complex Hilbert space $\mathcal{H}$; pure states are unit vectors $|\psi\rangle \in \mathcal{H}$ with $\| |\psi\rangle \|_2 = 1$, and mixed states are density operators $\rho \succeq 0$ with $\mathrm{Tr}(\rho) = 1$. For a pure state $|\psi\rangle$, the associated density matrix is $\rho = |\psi\rangle\langle\psi|$.

Unitary evolution.: Closed-system dynamics are generated by unitary operators. In continuous time, the Schrödinger equation is $\mathrm{i}\hbar \frac{d}{dt} |\psi(t)\rangle = H |\psi(t)\rangle$, whose solution is $|\psi(t)\rangle = e^{-\mathrm{i}Ht/\hbar} |\psi(0)\rangle$. In this thesis, we focus on *discrete-time* dynamics, where a sequence of unitaries $\{U_t\}_{t=0}^{T-1}$ produces a stroboscopic evolution

$$|\psi_{t+1}\rangle = U_t |\psi_t\rangle, \qquad t = 0, 1, \ldots, T-1, \tag{5}$$

so that $|\psi_T\rangle = U_{T-1} \cdots U_0 |\psi_0\rangle$. Unitarity preserves norm and therefore total probability.

Born rule and projective measurements.: A projective measurement is specified by orthogonal projectors $\{\Pi_y\}_y$ summing to identity. If the system is in state ρ, then the probability of outcome y is $\Pr(y) = \mathrm{Tr}(\Pi_y \rho)$ (Born rule). This rule is the only bridge from amplitudes (complex numbers) to observable probabilities, and it is the reason that our objectives are always defined in terms of terminal distributions (or reduced densities) rather than raw amplitudes.

Subsystems and partial trace.: When $\mathcal{H} = \mathcal{H}_A \otimes \mathcal{H}_B$ factorizes, the reduced state on A is $\rho^{(A)} = \mathrm{Tr}_B(\rho)$. The reduced state is *operationally complete* for any observable that acts only on A: for all Hermitian A on $\mathcal{H}_A$,

$$\mathrm{Tr}\big((A \otimes I)\rho\big) = \mathrm{Tr}\big(A\, \rho^{(A)}\big). \tag{6}$$

In our setting, the walker's *position* is the only measured quantity, while the *coin* is an internal degree of freedom that mediates interference and control (and is therefore traced out at evaluation). This is explicitly the operational role stated in our project specification.

B. Graphs, ports, and the arc basis (irregular graphs included)

Let $G = (V, E)$ be a finite graph with $|V| = n$ and $|E| = m$. For $v \in V$, let d_v denote its degree. Because this project targets *families* of graphs (lines, grids, hypercubes, irregular graphs, etc.), we require a formulation that remains well-defined when degrees vary across vertices.

A convenient way to represent coined quantum walks on irregular graphs is the *arc (port) basis*. Each undirected edge $\{u, v\} \in E$ induces two directed arcs (or half-edges), one outgoing from u and one outgoing from v. At each vertex v, we index its outgoing arcs by a *local port label* $a \in \{1, \ldots, d_v\}$. We write the corresponding basis vector as $|v, a\rangle$ (position v, port a). The local port indexing is encoded by a deterministic *port map* (an involutive matching of arcs across edges), which is how the simulator implements a flip-flop shift on irregular graphs.

Definition 1 (Port map and involution)**.** *A* port map *is a bijection*

$$\mathrm{port\_map} : \{(v, a) : v \in V,\ a \in [d_v]\} \to \{(v, a) : v \in V,\ a \in [d_v]\}.$$

such that for every arc (v, a) there exists a unique matched arc (u, a') across the same edge and

$$\mathrm{port\_map}(v, a) = (u, a'), \qquad \mathrm{port\_map}(u, a') = (v, a).$$

Equivalently, port_map *is an involution:* $\mathrm{port\_map} \circ \mathrm{port\_map} = \mathrm{id}$.

Remark 1 (Why ports matter for irregular degrees)**.** *The key reason this construction works for irregular graphs is that* port_map *never mixes different port cardinalities: it only swaps* matched arc indices *across an edge. This avoids undefined behavior that would arise if a shift attempted to apply a uniform coin dimension to vertices with different degrees.*

C. Coined discrete-time quantum walks (DTQWs)

Joint Hilbert space.: A coined DTQW evolves on a joint Hilbert space

$$\mathcal{H} \;=\; \mathcal{H}_P \otimes \mathcal{H}_C, \tag{7}$$

where $\mathcal{H}_P \cong \mathbb{C}^{|V|}$ is the position space spanned by $\{|v\rangle\}_{v\in V}$, and $\mathcal{H}_C$ is a direct sum of local coin spaces $\mathcal{H}_C(v) \cong \mathbb{C}^{d_v}$ associated with ports at each node. In the arc basis, a pure state at time t is

$$|\psi_t\rangle = \sum_{v\in V}\sum_{a=1}^{d_v} \psi_t(v,a)\;|v,a\rangle\,, \qquad \sum_{v,a}|\psi_t(v,a)|^2 = 1.$$

Local coins and the block-diagonal global coin.: At each vertex v and time t, the walk applies a *local coin* unitary

$$C_v(t) \in U(d_v),$$

acting only on $\mathcal{H}_C(v)$. The global coin is the block-diagonal direct sum over vertices,

$$C_t \;=\; \bigoplus_{v\in V} C_v(t), \tag{8}$$

which is explicitly the construction used in our ACPL formulation.

Flip-flop shift.: The shift operator S is a permutation unitary acting on the arc basis, defined by the port map:

$$S\,|v,a\rangle \;=\; |u,a'\rangle \quad \text{where} \quad (u,a') = \mathrm{port\_map}(v,a). \tag{9}$$

Matrix-wise, S is a permutation matrix with entries $S_{(u,a'),(v,a)} = 1$ iff $(u,a') = \mathrm{port\_map}(v,a)$, else 0, again matching the project definition.

One-step propagator.: A single DTQW step is

$$U_t \;=\; S\,C_t, \qquad |\psi_{t+1}\rangle = U_t\,|\psi_t\rangle\,. \tag{10}$$

This is the exact unitary dynamics statement used throughout ACPL-qwalk.

Lemma 1 (Unitarity of coin and shift)**.** *If $C_v(t) \in U(d_v)$ for all $v \in V$ and S is defined by a port-map permutation, then C_t and S are unitary operators on $\mathcal{H}$.*

Proof: Each $C_v(t)$ is unitary on $\mathcal{H}_C(v)$; therefore the direct sum $C_t = \bigoplus_v C_v(t)$ is unitary on $\mathcal{H}_C$ (blockwise). The shift S is a permutation matrix in the arc basis, hence $S^\dagger S = SS^\dagger = I$. ∎

Theorem 1 (Unitarity of the DTQW step)**.** *Under the assumptions of Lemma 1, the one-step propagator $U_t = SC_t$ is unitary and preserves normalization.*

Proof: A product of unitary operators is unitary: $U_t^\dagger U_t = C_t^\dagger S^\dagger S C_t = C_t^\dagger C_t = I$. Thus $\|\,|\psi_{t+1}\rangle\,\|_2 = \|U_t\,|\psi_t\rangle\,\|_2 = \|\,|\psi_t\rangle\,\|_2$. ∎

D. Position-only observables: reduced density and terminal distribution

Why we trace out the coin.: All tasks considered in ACPL (state transfer, marked-node search, fast mixing, and robust transport) depend only on *where the walker is* at the end of the episode, not on the internal coin state. Accordingly, evaluation uses the reduced position density $\rho_T^{(P)} = \mathrm{Tr}_C(\rho_T)$ and/or the position probability vector P_T.

Definition 2 (Reduced position density and position distribution)**.** *Let $\rho_T = |\psi_T\rangle\langle\psi_T|$ be the terminal density on $\mathcal{H} = \mathcal{H}_P \otimes \mathcal{H}_C$. The reduced position density is*

$$\rho_T^{(P)} = \mathrm{Tr}_C(\rho_T).$$

If we measure only position, the relevant projectors are $\Pi_v = |v\rangle\langle v| \otimes I_C$, and the Born rule yields

$$P_T(v) \;=\; \mathrm{Tr}(\Pi_v \rho_T) \;=\; \Big\langle v|\rho_T^{(P)}|v\Big|v|\rho_T^{(P)}|v\Big\rangle\,. \tag{11}$$

This is the exact operational statement used in our project documents.

Uniqueness and operational completeness.: The reduced state $\rho_T^{(P)}$ is the *unique* object reproducing all statistics of observables that act only on position (i.e., $A_P \otimes I_C$), which justifies defining objectives in terms of P_T or $\rho_T^{(P)}$.

Proposition 1 (Position-only statistics depend only on $\rho_T^{(P)}$)**.** *For any Hermitian A_P acting on $\mathcal{H}_P$,*

$$\mathrm{Tr}\big((A_P \otimes I_C)\rho_T\big) = \mathrm{Tr}\big(A_P\,\rho_T^{(P)}\big).$$

Proof: This is the defining property of the partial trace, specialized to the P/C bipartition. ∎

Concrete formula in the arc basis.: In the arc basis $\{|v,a\rangle\}$, $P_T(v)$ is simply the reduction over coin ports:

$$P_T(v) \;=\; \sum_{a=1}^{d_v} |\psi_T(v,a)|^2, \tag{12}$$

and more generally $\rho_T^{(P)}(v,v') = \sum_{a=1}^{d_v} \langle v,a|\,\rho_T\,|v',a\rangle$. This fact is essential both theoretically (operational meaning) and computationally (efficient evaluation), and it is explicitly the recommended implementation strategy (reshape and sum over the coin axis).

Gauge/basis invariance and "no backaction".: Because we compute P_T and $\rho_T^{(P)}$ without measuring the coin, evaluation does not induce measurement backaction. Moreover, local rephasings or relabelings within the coin space leave $\rho_T^{(P)}$ unchanged; this is precisely why the project adopts $\rho_T^{(P)}$ as the core evaluation object.

Theorem 2 (Coin-gauge invariance of position statistics)**.** *Let G_t be any unitary that acts as a block-diagonal change of basis on the coin subsystem (i.e., $G_t = I_P \otimes G_{C,t}$, where $G_{C,t}$ may itself be block-diagonal across vertices/ports). Define $\tilde{\rho}_T = (I \otimes G_{C,T})\,\rho_T\,(I \otimes G_{C,T})^\dagger$. Then $\mathrm{Tr}_C(\tilde{\rho}_T) = \mathrm{Tr}_C(\rho_T)$ and therefore $\tilde{P}_T(v) = P_T(v)$ for all v.*

Proof: By cyclicity of trace and the defining property of partial trace, $\mathrm{Tr}_C((I \otimes G)\rho(I \otimes G)^\dagger) = \mathrm{Tr}_C(\rho)$ for any coin-only unitary G. Then $\tilde{P}_T(v) = \langle v| \mathrm{Tr}_C(\tilde{\rho}_T) |v\rangle = \langle v| \mathrm{Tr}_C(\rho_T) |v\rangle = P_T(v)$. ∎

E. Classical random walks, mixing, and total-variation distance

Classical random walks on graphs provide both intuition and baselines. A (lazy) random walk on an undirected graph has transition matrix M with entries $M_{uv} = 1/d_u$ if $\{u, v\} \in E$ (possibly with self-loop mass for laziness). The distribution $p_t \in \Delta^{n-1}$ evolves linearly as $p_{t+1} = p_t M$, converging (under mild conditions) to a stationary distribution.

Total variation distance.: A standard notion of mixing uses total variation (TV) distance: for distributions $p, q \in \Delta^{n-1}$,

$$d_{\mathrm{TV}}(p, q) = \frac{1}{2}\|p - q\|_1.$$

In our quantum-walk setting, the fast-mixing objective is defined by comparing the terminal position distribution P_T to a target stationary distribution ρ^* (uniform in the mixing suite),

$$\mathrm{TV}(P_T, \rho^*) = \frac{1}{2}\sum_{v \in V} |P_T(v) - \rho^*(v)|, \qquad \rho^*(v) = \frac{1}{|V|}. \tag{13}$$

This is the exact mixing error used and documented in the project.

Quantum vs. classical mixing.: Quantum walks do not converge in the same way as ergodic Markov chains because unitary dynamics are reversible. Nevertheless, *finite-horizon* mixing objectives are meaningful and practically useful: for a fixed horizon T, we may ask that P_T (or a time-average $\bar{P}_T$) be close to a target distribution. This is exactly the regime in which learning a time- and position-dependent coin schedule is a well-posed control problem.

F. Objectives and robust evaluation: mean, disorder, and CVaR

Terminal objectives.: The project uses task returns $f(\psi_T)$ whose dependence on ψ_T is mediated through P_T or $\rho_T^{(P)}$. For example: (i) state transfer or search uses success probability on a target set Ω, $P_\Omega(T) = \sum_{m \in \Omega} P_T(m)$, and (ii) mixing uses the TV objective in (13).

Disorder and risk-sensitive metrics.: To model robustness, we introduce a disorder variable ξ (e.g., edge-phase noise or coin dephasing), making the terminal success probability itself random: $P_T(j^*; \xi)$. Two canonical summaries are:

- the mean performance $\mathbb{E}_\xi[P_T(j^*; \xi)]$ (risk-neutral), and
- the Conditional Value at Risk (CVaR), a risk-sensitive tail statistic.

In our project definition, CVaR at level $\alpha \in (0, 1]$ is

$$\mathrm{CVaR}_\alpha \;=\; \frac{1}{\alpha}\int_0^\alpha Q_u \, du, \tag{14}$$

where Q_u denotes the u-quantile of the random success probability (lower tail). CVaR is particularly appropriate when rare but severe degradations matter (e.g., a policy that occasionally fails catastrophically under disorder).

G. A formal-logic view: well-formed episodes and invariants

Because ACPL operates as a *procedural episode generator* and a *simulator-driven learning system*, it is useful to state the correctness conditions of an "episode" using predicates. This makes explicit which properties are assumed by the physics (unitarity, normalization) and which are assumed by the graph representation (ports, matchings).

Predicates.: We introduce the following predicates (intended as specification, not implementation):

$$\begin{aligned}
&\mathsf{Graph}(G) \iff G = (V, E) \text{ finite},\\
&\mathsf{DegOK}(G) \iff \forall v \in V,\ d_v \in \mathbb{N},\\
&p := \mathrm{port_map},\\
&\mathsf{PortMapOK}(G, p)\\
&\qquad \iff \forall v \in V,\ \forall a \in [d_v],\\
&\qquad\quad p^2(v, a) = (v, a),\\
&\mathsf{CoinUnitary}(C_t)\\
&\qquad \iff \forall v \in V,\ C_v(t) \in U(d_v),\\
&\mathsf{ShiftOK}(S)\\
&\qquad \iff S \text{ acts as in (9)}.
\end{aligned}$$

Then an admissible coined DTQW step at time t can be specified by the first-order sentence

$$\begin{aligned}
\forall t \in \{0, \dots, T-1\}:\ &\mathsf{Graph}(G)\\
&\wedge \mathsf{DegOK}(G)\\
&\wedge \mathsf{PortMapOK}(G, \mathrm{port_map})\\
&\wedge \mathsf{CoinUnitary}(C_t)\\
&\wedge \mathsf{ShiftOK}(S)\\
&\Rightarrow \mathsf{Unitary}(U_t).
\end{aligned}$$

where $\mathsf{Unitary}(U_t)$ abbreviates $U_t^\dagger U_t = I$. This implication is precisely the content of Theorem 1, grounded in the project's definition of S and C_t on the arc basis.

Propositional "invariants" used throughout the thesis.: At a higher level, the following propositional invariants guide the remainder of the report:

1) **(I1) Physical validity:** coins are unitary and the step is unitary (hence probabilities sum to 1).
2) **(I2) Observable correctness:** objectives depend only on P_T (or $\rho_T^{(P)}$), not on coin readout.
3) **(I3) Irregular-graph support:** shifts are defined by port matchings and never mix incompatible degrees.
4) **(I4) Metric well-posedness:** mixing is evaluated by TV-to-uniform, transfer/search by terminal success probability, and robustness by mean/CVaR under disorder.

H. (Bridge to ACPL: why a learned coin is a control policy

The DTQW formalism above becomes a control problem once we allow $C_v(t)$ to vary with node and time. In ACPL, a learning policy with weights ω outputs coin parameters $\theta_{v,t}$,

which are then mapped to a physical unitary via a lift $\Phi : \mathbb{R}^p \to U(d_v)$; the project fixes notation so that ω denotes network weights and $\theta_{v,t}$ denotes coin parameters. The mixing objective uses TV distance (13), and robustness can be risk-shaped by CVaR (14). These components are expanded in the next chapter (Problem Formulation / ACPL), but the mathematical background needed for that formulation is entirely contained in the objects defined here: $|\psi_t\rangle$, $U_t = SC_t$, and the position marginal P_T.

IV. Policy Architecture: GNN-Based Coin Synthesis

A. Design Requirements and Architectural Overview

Adaptive Coin Policy Learning (ACPL) treats the *coin operators* as the control degrees of freedom of a coined discrete-time quantum walk (DTQW). The central architectural requirement is to learn a policy that, for each time step and (optionally) for each vertex, emits *physically valid* (unitary) local coins while respecting the symmetries of graph-structured inputs and enabling generalization across graph sizes and topologies.

Quantum walk state space (arc/port model).: Let $G = (V, E)$ be a finite (typically undirected) graph. For each vertex $v \in V$ with degree d_v, define the local coin subspace

$$\mathcal{H}_C(v) \cong \mathbb{C}^{d_v}, \tag{15}$$

and the global "arc coin" space as the direct sum

$$\mathcal{H}_C \;=\; \bigoplus_{v \in V} \mathcal{H}_C(v). \tag{16}$$

Let $\mathcal{H}_P = \mathrm{span}\{\,|v\rangle : v \in V\,\}$ denote position space. The joint Hilbert space is

$$\mathcal{H} \;=\; \mathcal{H}_P \otimes \mathcal{H}_C, \tag{17}$$

with a convenient basis given by arc/port labels $\{|v, a\rangle : v \in V,\ a \in \{1, \ldots, d_v\}\}$.

One-step dynamics (coin then shift).: At each discrete time $t \in \{0, \ldots, T-1\}$, the walk applies a block-diagonal global coin

$$C_t \;=\; \bigoplus_{v \in V} C_v(t), \qquad C_v(t) \in U(d_v), \tag{18}$$

followed by a flip–flop shift S that permutes arc indices via the reverse-arc (port-map) pairing:

$$U_t \;=\; S\,C_t, \qquad |\psi_{t+1}\rangle = U_t|\psi_t\rangle. \tag{19}$$

Crucially, S depends only on graph wiring (the port map) and is independent of the numerical entries of $C_v(t)$; operationally, S is a fixed permutation unitary with $S^\dagger = S^{-1} = S$ and $S^2 = I$. This decomposition is especially convenient for irregular graphs because C_t is block-diagonal in the arc basis and S merely reindexes amplitudes across arc coordinates.

Policy objective (control through unitary coins).: We learn a policy with weights ω (all trainable neural parameters) that outputs coin parameters $\theta_{v,t}$, which are then mapped to unitaries $C_v(t)$:

$$\pi_\omega :\ (G, X, t) \mapsto \{\theta_{v,t}\}_{v \in V}, \qquad C_v(t) = \Phi_{d_v}(\theta_{v,t}) \in U(d_v). \tag{20}$$

The mapping Φ_{d_v} is a *unitary lift* (Sec. IV-E) that guarantees physical validity.

Architecture (GNN encoder + temporal controller + unitary head).: The policy is implemented as a composition

$$\pi_\omega \;=\; \mathrm{Head} \circ \mathrm{Controller} \circ \Phi_{\mathrm{GNN}}, \tag{21}$$

where:

- Φ_{GNN} is a permutation-equivariant message-passing encoder producing node embeddings z_v,
- Controller is a time-aware module (GRU or lightweight Transformer) that conditions on t (normalized) to produce per-node control codes,
- Head maps control codes to coin parameters $\theta_{v,t}$, then Φ_{d_v} maps parameters to unitaries $C_v(t)$.

B. Graph Representation and Node Features

The policy consumes a graph representation (e.g., sparse edge list / adjacency) and a node-feature matrix

$$X = [x_v]_{v \in V} \in \mathbb{R}^{|V| \times F}. \tag{22}$$

Features must *not* leak arbitrary node IDs; instead they are constructed from graph-intrinsic quantities and (when available) canonical geometric coordinates.

Canonical feature composition.: We use a block-structured feature vector

$$x_v \;=\; \Big[\ \tilde{d}_v\ ;\ \pi_{\mathrm{pos}}(v)\ ;\ \chi_{\mathrm{geom}}(v)\ ;\ \chi_{\mathrm{role}}(v)\ \Big], \tag{23}$$

where:

- $\tilde{d}_v := d_v / \max_{u \in V} d_u$ is normalized degree,
- $\pi_{\mathrm{pos}}(v) \in \mathbb{R}^K$ is a positional encoding (PE) such as Laplacian eigenvectors,
- $\chi_{\mathrm{geom}}(v)$ is a graph-family coordinate (e.g., $(x/L, y/L)$ on grids; bitstring in $\{0,1\}^n$ on hypercubes),
- $\chi_{\mathrm{role}}(v)$ are optional task-agnostic structural flags (e.g., boundary indicators) used sparingly.

Laplacian positional encodings and sign-fixing.: For irregular graphs, we use Laplacian PEs. Let L be a (combinatorial or normalized) Laplacian with eigenpairs $L\varphi_k = \lambda_k \varphi_k$. Define

$$\pi_{\mathrm{pos}}(v) = \big(\varphi_1(v), \ldots, \varphi_K(v)\big), \tag{24}$$

with a deterministic sign convention per eigenvector (e.g., flip φ_k so that the entry of maximum magnitude is positive). This ensures that PEs behave consistently under node relabeling and do not introduce spurious symmetry breaking.

Permutation-covariant feature requirement (predicate form).: Let P be a permutation matrix acting on node order. The feature construction must satisfy:

$$\forall P \in \mathsf{Perm}(|V|): \quad X' = PX, \quad A' = PAP^\top, \tag{25}$$

i.e., relabeling nodes permutes feature rows in the same way as adjacency.

C. Permutation-Equivariant GNN Encoder

The encoder Φ_{GNN} produces node embeddings $\{z_v\}_{v\in V}$ that summarize local structure while respecting graph symmetries.

Message passing (general MPNN form).: Let $h_v^{(0)} = x_v$. For layers $\ell = 0, \dots, L-1$:

$$m_{u\to v}^{(\ell)} = M^{(\ell)}\Big(h_u^{(\ell)},\, h_v^{(\ell)},\, e_{uv}\Big), \tag{26}$$

$$\bar{m}_v^{(\ell)} = \sum_{u\in\mathcal{N}(v)} m_{u\to v}^{(\ell)}, \tag{27}$$

$$h_v^{(\ell+1)} = U^{(\ell)}\Big(h_v^{(\ell)},\, \bar{m}_v^{(\ell)}\Big), \tag{28}$$

with shared parameters across all nodes/edges in each layer. We set $z_v := h_v^{(L)} \in \mathbb{R}^H$.

Typical encoder instantiations.: Depending on the experiment family, $M^{(\ell)}, U^{(\ell)}$ instantiate standard layers: GCN (renormalized adjacency), GIN (sum + MLP), GraphSAGE (mean + MLP), GAT (attention-weighted sum), or PNA (multi-aggregator + degree scalers). Depth is kept small ($L \in \{2, 3\}$) with residual connections and normalization to mitigate over-smoothing and stabilize training.

Theorem 3 (Permutation-Equivariance of Message Passing)**.** *Let Φ_{GNN} be defined by (26)–(28) with sum aggregation and shared parameters. For any permutation matrix P and permuted inputs $(A', X') = (PAP^\top, PX)$, the resulting embeddings satisfy*

$$Z' = PZ, \tag{29}$$

where $Z = [z_v]_{v\in V}$ and $Z' = [z'_v]_{v\in V}$ denote the stacked embeddings under (A, X) and (A', X'), respectively.

Proof: The claim follows by induction on layers. For $\ell = 0$, $H'^{(0)} = X' = PX = PH^{(0)}$. Assume $H'^{(\ell)} = PH^{(\ell)}$. Under permutation, neighborhoods map bijectively: $\mathcal{N}'(Pv) = \{Pu : u \in \mathcal{N}(v)\}$. Since $M^{(\ell)}$ and $U^{(\ell)}$ are shared, each message term is computed identically up to relabeling, and the sum in (27) is invariant to the order of neighbors. Therefore $\bar{m}'^{(\ell)} = P\bar{m}^{(\ell)}$ and then (28) yields $H'^{(\ell+1)} = PH^{(\ell+1)}$. Setting $Z = H^{(L)}$ completes the proof. ∎

Propositional and predicate logic statement (architecture invariant).: Define the predicate $\mathsf{Eqv}(\Phi)$ meaning "Φ is permutation-equivariant." Then

$$\Big(\mathsf{Eqv}(\Phi_{\mathrm{GNN}}) \;\wedge\; \mathsf{NodewiseShared}(\mathrm{Controller}) \;\wedge\; \mathsf{NodewiseShared}(\mathrm{Head})\Big) \;\Rightarrow\; \mathsf{Eqv}(\pi_\omega).$$

Equivalently, in explicit quantifiers:

$$\forall P \in \mathsf{Perm}(|V|),\ \forall t: \quad \pi_\omega(P\cdot G,\ PX,\ t) = P\,\pi_\omega(G, X, t). \tag{30}$$

D. Temporal Controller and Time Conditioning

Quantum interference patterns are inherently time-structured; consequently, the policy must coordinate *when* to spread amplitude and *when* to refocus it. We provide the controller with normalized time

$$\tau_t := \frac{t}{T} \in [0, 1], \tag{31}$$

and form the node–time input

$$s_{v,t} := \big[z_v \;;\; \tau_t \;;\; \pi_{\mathrm{pos}}(v)\big] \in \mathbb{R}^{H+1+K}. \tag{32}$$

We emphasize that τ_t is *not* a node attribute; it is injected at control time to enable transfer across nearby horizons T.

GRU controller (default).: A lightweight GRU produces per-node hidden states $h_{v,t}$:

$$h_{v,t} = \mathrm{GRU}(s_{v,t}, h_{v,t-1}), \qquad h_{v,-1} = 0. \tag{33}$$

This induces causal temporal dependence and tends to be stable under backpropagation through time.

Transformer alternative (optional).: A small causal Transformer may replace (33) when the task benefits from longer-range temporal interactions (e.g., pulse trains or repeated focusing). In either case, the controller is parameter-shared across nodes to preserve equivariance.

E. Coin Head and Unitary Lifting Map

The head maps controller states to coin parameters $\theta_{v,t}$, and a lift map Φ_{d_v} converts parameters to a unitary matrix $C_v(t) \in U(d_v)$.

Head (parameters).:

$$\theta_{v,t} = W_{\mathrm{head}}\, h_{v,t} + b_{\mathrm{head}}. \tag{34}$$

The parameter dimension depends on degree: $p(d_v) = 3$ for $d_v = 2$ (SU(2) Euler angles) and typically $p(d_v) = d_v^2$ (or $d_v^2 - 1$) for $d_v > 2$ via Lie-algebra coordinates.

1) Degree-2 coins: SU(2) via Euler ZYZ angles: For $d_v = 2$ (lines/cycles), we use the standard ZYZ parameterization:

$$\begin{aligned} \theta_{v,t} &= (\alpha_{v,t}, \beta_{v,t}, \gamma_{v,t}) \in \mathbb{R}^3, \\ C_v(t) &= R_z(\alpha_{v,t})\, R_y(\beta_{v,t})\, R_z(\gamma_{v,t}) \in \mathrm{SU}(2). \end{aligned} \tag{35}$$

This is sufficient because physically irrelevant global phases can be treated as gauge and need not be learned explicitly in the degree-2 setting.

2) Degree-$d > 2$ coins: exponential map on $u(d)$ (or Cayley retraction): For $d_v > 2$, we parameterize a skew-Hermitian generator $K_{v,t} \in u(d_v)$:

$$K_{v,t} \;=\; i\sum_{k=1}^{q} \theta_{v,t}^{(k)}\, G_k, \qquad K_{v,t}^\dagger = -K_{v,t}, \tag{36}$$

where $\{G_k\}_{k=1}^q$ is an orthonormal Hermitian basis (e.g., generalized Gell–Mann plus identity). Then

$$C_v(t) \;=\; \exp(K_{v,t}) \in U(d_v). \tag{37}$$

As a faster alternative with stable gradients, one may use the Cayley retraction

$$C_v(t) \;=\; (I - K_{v,t})(I + K_{v,t})^{-1}, \tag{38}$$

whenever $I + K_{v,t}$ is invertible (which holds generically and can be enforced by mild regularization).

Proposition 2 (Unitarity of the Lift)**.** *If $K^\dagger = -K$, then $\exp(K)$ is unitary. If $K^\dagger = -K$ and $I + K$ is invertible, then $(I - K)(I + K)^{-1}$ is unitary.*

Proof: For the exponential, $(\exp(K))^\dagger = \exp\big(K^\dagger\big) = \exp(-K) = (\exp(K))^{-1}$. For the Cayley map, define $C = (I-K)(I+K)^{-1}$. Then $C^\dagger = \big((I+K)^{-1}\big)^\dagger (I-K)^\dagger = (I+K^\dagger)^{-1}(I-K^\dagger)$. Using $K^\dagger = -K$ gives $C^\dagger = (I-K)^{-1}(I+K) = C^{-1}$, hence $C^\dagger C = I$. ∎

Assembling the global coin.: Given all local blocks, the global operator C_t in (18) is applied to the state by grouping amplitudes by vertex and multiplying by the corresponding $d_v \times d_v$ matrix; explicitly forming a dense global matrix is unnecessary.

F. End-to-End Differentiability and Gradient Flow Through the DTQW

Many ACPL objectives depend only on the *terminal position distribution* rather than the internal coin state. Let $\rho_t = |\psi_t\rangle\langle\psi_t|$ be the joint density. The reduced position density is

$$\rho_t^{(P)} := \mathrm{Tr}_C(\rho_t), \tag{39}$$

and the terminal position probabilities are

$$P_T(v) \;=\; \langle v|\rho_T^{(P)}|v\rangle \;=\; \sum_{a=1}^{d_v} |\psi_T(v,a)|^2. \tag{40}$$

This reduction is operationally correct: it reproduces the statistics of all position-only observables $A_P \otimes I_C$ and avoids coin measurement back-action.

Generic learning objective (expectation over episodes).: Let e denote an episode sampled from a distribution $\mathcal{D}$ (graph, initial state, task parameters, disorder). We maximize a task return $f_e(\psi_T)$ (or equivalently minimize a loss $\mathcal{L}_e = -f_e$):

$$J(\omega) := \mathbb{E}_{e\sim\mathcal{D}}\big[f_e(\psi_T(\omega))\big], \qquad \min_\omega \; \mathbb{E}_{e\sim\mathcal{D}}\big[\mathcal{L}_e(\omega)\big]. \tag{41}$$

This formulation cleanly supports robust objectives (e.g., expectation over disorder ξ) and empirical risk minimization via mini-batches.

Adjoint/Backprop-through-time (BPTT) structure.: Define the step unitaries $U_t(\omega) = S\,C_t(\omega)$ and evolution $|\psi_{t+1}\rangle = U_t|\psi_t\rangle$. For a differentiable scalar loss $\mathcal{L}$ depending on ψ_T, define adjoints $\lambda_T := \partial\mathcal{L}/\partial\psi_T$ and

$$\lambda_t \;=\; U_t^\dagger\,\lambda_{t+1}, \qquad t = T-1,\dots,0. \tag{42}$$

Then, for any coin-parameter component θ influencing C_t,

$$\frac{\partial\mathcal{L}}{\partial\theta} \;=\; 2\,\mathrm{Re}\left(\sum_{t=0}^{T-1}\left\langle \lambda_{t+1},\; S\,\frac{\partial C_t}{\partial\theta}\,\psi_t\right\rangle\right), \tag{43}$$

where $\langle\cdot,\cdot\rangle$ is the complex inner product. Autodiff frameworks implement (43) implicitly, including the Fréchet derivative for $\exp(K)$ in (37).

G. Stochastic Policies and RL-Compatible Variant (When Needed)

Some tasks introduce non-differentiabilities (e.g., hard oracle queries, discrete interventions, constrained actions). In that case, we treat $\theta_{v,t}$ as an *action* drawn from a policy distribution:

$$\theta_{v,t} \sim \pi_\omega(\cdot \mid s_{v,t}), \tag{44}$$

apply $C_v(t) = \Phi_{d_v}(\theta_{v,t})$, and optimize ω using policy-gradient methods (e.g., PPO) on the episodic return $f(\psi_T)$. Importantly, the architectural symmetry guarantees in (30) remain applicable because sampling is performed nodewise from shared-parameter distributions.

H. Computational Complexity and Scaling

Let H be embedding width, L GNN depth, and $\bar{d}$ a typical degree. One time step computes: (i) message passing on edges (sparse gather/scatter), and (ii) local dense coin multiplications per vertex.

Per-step cost.: For message passing, each layer is edge-linear:

$$\mathrm{cost}_{\mathrm{GNN}} \;=\; \widetilde{\mathcal{O}}(|E|\cdot H), \tag{45}$$

where $\widetilde{\mathcal{O}}$ hides small constants (activations, norms, residuals). For coin construction/application, local dense multiplies scale as

$$\mathrm{cost}_{\mathrm{coin}} \;=\; \widetilde{\mathcal{O}}\left(\sum_{v\in V} d_v^2\right) \;=\; \widetilde{\mathcal{O}}(|V|\bar{d}^2). \tag{46}$$

Thus total forward cost over horizon T is

$$\widetilde{\mathcal{O}}\big(T\big(|E|H + |V|\bar{d}^2\big)\big)\,, \tag{47}$$

with BPTT increasing constants but preserving the T-linear scaling.

I. Architecture-Validating Ablations (Symmetry and Control Capacity)

The architecture is designed so that each component has a falsifiable role; we therefore associate concrete ablations:

- **NoPE:** remove π_{pos} from x_v and/or from $s_{v,t}$ in (32) to test the necessity of positional information.
- **GlobalCoin:** constrain $C_v(t) \equiv C(t)$ for all v to remove spatial adaptivity and isolate the gains from per-node control.
- **TimeFrozen:** constrain $C_v(t) \equiv C_v$ to remove temporal choreography and test the necessity of time dependence.
- **NodePermute:** apply random node permutations P to inputs/targets and verify metric invariance as implied by (30); any measurable drift indicates symmetry leakage (e.g., ID-dependent features).

J. Summary (What the Policy Learns)

At each node v and time t, the GNN embedding z_v encodes local structural context in a permutation-equivariant manner, while the temporal controller leverages normalized time τ_t to orchestrate interference over the horizon. The head emits compact coin parameters $\theta_{v,t}$ and the lift Φ_{d_v} ensures $C_v(t) \in U(d_v)$, making the overall controller *physically realizable*, *symmetry-respecting*, and *trainable* either by backpropagating through the DTQW or via RL when needed.

V. Data and Episode Generation

A. Episodes as a Probabilistic Dataset (Not a Static Corpus)

Unlike conventional supervised learning where a dataset is a fixed multiset of examples, our "dataset" is a *procedural* generator that induces a distribution over *episodes*. An episode packages (i) a graph instance, (ii) an initial quantum state, (iii) a task specification, and (iv) optional noise/disorder variables. Formally, we model one episode as a random element

$$e \;=\; (G, X, \psi_0, T, \mathsf{task}, \Omega, \rho^\star, \xi) \;\in\; \mathcal{E}, \tag{48}$$

drawn from a distribution $\mathcal{D}$ on a measurable episode space $(\mathcal{E}, \mathcal{F})$. Here $G = (V, E)$ is a finite graph (possibly with heterogeneous degrees), $X \in \mathbb{R}^{|V|\times F}$ is the node-feature matrix, ψ_0 is an initial pure state in the walk Hilbert space, $T \in \mathbb{N}$ is the horizon (number of controlled steps), $\mathsf{task} \in \{\mathsf{transfer}, \mathsf{search}, \mathsf{mixing}, \mathsf{robust}\}$ selects the objective family, $\Omega \subseteq V$ is a marked/target set when applicable, $\rho^\star$ is a target distribution (e.g., uniform) when applicable, and ξ denotes disorder/noise latent variables when applicable.

Learning objective over episodes: For policy weights ω and induced controlled DTQW evolution $\psi_{t+1} = U_t(\omega, e)\psi_t$, we maximize an episode return $f(e;\omega)$ (or equivalently minimize $\mathcal{L}(e;\omega) = -f(e;\omega)$) via the population objective

$$J(\omega) \;=\; \mathbb{E}_{e\sim\mathcal{D}}\big[f(e;\omega)\big], \qquad \omega^\star \in \arg\max_{\omega} J(\omega). \tag{49}$$

In practice, training draws fresh episodes on-the-fly, while validation/test use *fixed* episode lists pre-registered by seed (Sec. V-F) to make evaluation reproducible and auditable.

B. Deterministic Procedural Generation from Seeds

Seeded generator as a function: We implement episode generation as a deterministic function of a configuration cfg and an episode seed s:

$$\mathsf{Gen} : \; (\mathsf{cfg}, s) \mapsto e. \tag{50}$$

All pseudorandom choices (graph instance, marked set, initial state randomization, noise sample, etc.) are derived from s under a fixed PRNG stack and a fixed sampling order. This design yields two critical properties:

Proposition 3 (Determinism of episode instantiation)**.** *Fix a configuration* cfg *and software version* ver*. For any seed s, repeated calls to* $\mathsf{Gen}(\mathsf{cfg}, s)$ *produce identical episodes:*

$$\forall s \;\; \forall k \in \mathbb{N} : \quad \mathsf{Gen}^{(k)}(\mathsf{cfg}, s) = \mathsf{Gen}(\mathsf{cfg}, s),$$

where $\mathsf{Gen}^{(k)}$ *denotes the k-th invocation under the same environment and version.*

Remark 2 (Why determinism matters)**.** *Determinism converts evaluation into a controlled experiment: differences in outcomes are attributable to model/algorithmic changes, not hidden resampling. It also makes committee-level auditing possible: a reported result can be regenerated from a manifest and seed list.*

Canonical episode identifiers: To prevent accidental collisions and to enable stable indexing, we define a canonical episode identifier

$$\mathsf{id}(e) \;=\; \mathsf{Hash}\Big(\mathrm{Serialize}(\mathsf{cfg}) \,\|\, s \,\|\, \mathsf{ver}\Big), \tag{51}$$

where Hash is a fixed cryptographic hash (e.g., BLAKE2b) and $\mathrm{Serialize}(\cdot)$ is a canonical serialization (sorted keys, fixed numeric formatting). This ID is used in manifests, logs, and result registries so that *every plotted point* can be traced back to its generating specification.

C. Graph Families and Sampling Distributions

Let $\mathcal{G}$ denote the support of graph instances considered. We use multiple graph families to test both *in-family* and *cross-family* generalization.

Structured families:

- **Line/cycle graphs (1D)**: $|V| = N$, degree $d_v = 2$ for cycles (and $d_v \in \{1, 2\}$ for lines). These support clean state-transfer and transport studies with controlled distances.
- **2D grids/lattices**: $|V| = L^2$ with local degree $d_v \in \{2, 3, 4\}$ depending on boundaries. These support spatial search and mixing studies.
- **Hypercubes**: $|V| = 2^n$ with uniform degree $d_v = n$ and strong symmetry, supporting classical quantum-walk search baselines and scaling tests.

Irregular/random families: We also include families where degree heterogeneity and local irregularities stress-test the policy: Erdős–Rényi $G(N, p)$, Watts–Strogatz small-world graphs, and d-regular random graphs. In these settings, positional information cannot rely solely on Euclidean coordinates and must incorporate graph-native encodings (Sec. V-E).

Formalization: A configuration cfg specifies a family selector fam and size parameters sz. We write

$$G \sim \mathcal{D}_G(\mathsf{fam}, \mathsf{sz}; s), \tag{52}$$

to emphasize that, although sampling is conceptually random, it is implemented deterministically from seed s.

D. Task-Specific Episode Components

Every episode selects one task family and instantiates the corresponding terminal objective f. We denote the reduced position distribution by partial trace over coin degrees of freedom:

$$\rho_t = |\psi_t\rangle\langle\psi_t|, \qquad \rho_t^{(P)} = \mathrm{Tr}_C(\rho_t), \qquad P_t(v) = \langle v|\rho_t^{(P)}|v\rangle. \tag{53}$$

State transfer: Given a target vertex $j^\star \in V$, the episode return is typically

$$f_{\mathsf{transfer}}(e;\omega) \;=\; P_T(j^\star), \tag{54}$$

with variants that average over multiple targets or enforce robustness margins.

Marked-node search: Given a marked set $\Omega \subseteq V$, success probability is

$$P_\Omega(T) \;=\; \sum_{v\in\Omega} P_T(v), \tag{55}$$

and $f_{\mathsf{search}}(e;\omega) = P_\Omega(T)$.

Fast mixing: Given the uniform target $\rho^\star(v) = 1/|V|$, we measure mixing error by total variation distance

$$\mathrm{TV}(P_T,\rho^\star) \;=\; \frac{1}{2}\sum_{v\in V}\left|P_T(v) - \frac{1}{|V|}\right|, \tag{56}$$

and use $f_{\mathsf{mixing}}(e;\omega) = -\mathrm{TV}(P_T,\rho^\star)$ (or a time-averaged variant).

Robust transport under disorder: Disorder is represented by $\xi \sim \mathcal{D}_\xi(\sigma)$, where σ controls strength (e.g., static edge phases or coin dephasing). A robust objective aggregates performance over ξ:

$$f_{\mathsf{robust}}(e;\omega) \;=\; \mathbb{E}_\xi[P_T(j^\star;\xi)] \quad \text{or} \quad \mathrm{CVaR}_\alpha(P_T(j^\star;\xi))\,, \tag{57}$$

depending on whether we prioritize mean performance or tail-risk control.

E. Node Features and Positional Encodings

The policy π_ω is graph-conditioned, so the episode must provide a feature representation $X = \{x_v\}_{v\in V}$. We design X to be: (i) informative enough for control, (ii) compatible with permutation symmetry, and (iii) stable under deterministic regeneration.

Base structural features: For every node v we include degree and local structural descriptors:

$$x_v^{\mathsf{deg}} = d_v, \qquad x_v^{\mathsf{normdeg}} = d_v/\max_{u\in V} d_u. \tag{58}$$

Geometric coordinates (when available): For grids/lattices, we include normalized coordinates

$$x_v^{\mathsf{coord}} = \left(\frac{i(v)}{L-1}, \frac{j(v)}{L-1}\right) \in [0,1]^2, \tag{59}$$

where $(i(v), j(v))$ is the canonical lattice embedding.

For hypercubes, we include the bitstring label $b(v) \in \{0,1\}^n$ (or its $\{-1,+1\}$ encoding), which fixes a canonical coordinate system consistent with hypercube symmetries.

Graph-native positional encodings: For irregular graphs where Euclidean coordinates are not meaningful, we use Laplacian positional encodings (LapPE). Let L be a (normalized) graph Laplacian and let (λ_k, ϕ_k) denote its eigenpairs with $0 = \lambda_1 \le \lambda_2 \le \cdots$. Define the top-$K$ LapPE feature as

$$\pi^{\mathsf{LapPE}}(v) \;=\; \big(\phi_2(v), \ldots, \phi_{K+1}(v)\big) \in \mathbb{R}^K, \tag{60}$$

with a *deterministic sign convention* to resolve the $\pm\phi_k$ ambiguity (e.g., enforce $\sum_v \phi_k(v) \ge 0$, breaking ties by the sign of the largest-magnitude entry). This ensures LapPE regenerates identically under Proposition 3.

Role encodings for tasks: Some tasks require marking information. We include a role indicator

$$x_v^{\mathsf{role}} = \begin{cases} 1, & v \in \Omega \text{ (marked)} \\ 0, & \text{otherwise} \end{cases} \quad \text{or} \quad x_v^{\mathsf{role}} = \mathbb{1}\{v = j^\star\}, \tag{61}$$

depending on whether the task is search or transfer.

Final feature vector: We concatenate all enabled components:

$$x_v \;=\; \Big[x_v^{\mathsf{deg}};\; x_v^{\mathsf{coord/bit}};\; \pi^{\mathsf{LapPE}}(v);\; x_v^{\mathsf{role}}\Big] \in \mathbb{R}^F. \tag{62}$$

This construction is compatible with a permutation-equivariant encoder: if nodes are relabeled, X is permuted in the same way and message passing respects this symmetry.

F. Manifest Schema, Pre-Registration, and Split Definitions

Manifests as executable specifications: Each experiment is defined by a manifest (YAML/JSON) that fixes: graph family and size ranges, horizon T, feature toggles, task parameters, noise models, baseline set, and seed routing rules. Conceptually, a manifest is a *program* specifying $\mathcal{D}$ and the evaluation protocol.

Minimal schema sketch: Below is a compact illustrative schema; in the actual repository, this is extended with model and optimizer fields.

```
experiment:
  name: "search-grid"

data:
  graph_family: "grid"
  sizes:
    L_train: [32, 80]
    L_test:  [96]
  horizon_T: 128
  features:
    deg: true
    coords: true
    LapPE: {K: 16}
    role: true

task:
  kind: "search"
  marked:
    mode: "single"   # or "set"
    size: 1

splits:
  rule: "hash(seed) mod 100"
  train: [0..79]
  val:   [80..89]
  test:  [90..99]

noise:
  enabled: false
```

Leak-free splits as logic constraints: We enforce that train/val/test episode sets are disjoint and are defined *only* by the seed-routing function. Let $\mathsf{Split} : \mathbb{N} \to \{\mathsf{train}, \mathsf{val}, \mathsf{test}\}$ be deterministic. Define the split membership predicate

$$\mathsf{InSplit}(e,\sigma) \;:\Longleftrightarrow\; \exists s\, \Big(e = \mathsf{Gen}(\mathsf{cfg}, s) \wedge \mathsf{Split}(s) = \sigma\Big).$$

Then leak-free splits can be stated as predicate logic:

$$\forall e \in \mathcal{E} :\; \big(\mathsf{InSplit}(e,\mathsf{train}) \to \neg\mathsf{InSplit}(e,\mathsf{val}) \wedge \neg\mathsf{InSplit}(e,\mathsf{test})\big), \tag{63}$$

and similarly for cyclic permutations of $\mathsf{train}, \mathsf{val}, \mathsf{test}$.

Theorem 4 (Split disjointness under deterministic routing)**.** *Assume* Gen *is deterministic (Proposition 3) and* Split *is a deterministic function of seed* s*. Let* $\mathcal{E}_\sigma = \{\mathsf{Gen}(\mathsf{cfg}, s) : \mathsf{Split}(s) = \sigma\}$*. Then* $\mathcal{E}_{\mathsf{train}}, \mathcal{E}_{\mathsf{val}}, \mathcal{E}_{\mathsf{test}}$ *are pairwise disjoint.*

Proof: Suppose for contradiction there exists $e \in \mathcal{E}_{\sigma_1} \cap \mathcal{E}_{\sigma_2}$ with $\sigma_1 \neq \sigma_2$. Then there exist seeds s_1, s_2 such that $e = \mathsf{Gen}(\mathsf{cfg}, s_1) = \mathsf{Gen}(\mathsf{cfg}, s_2)$ and $\mathsf{Split}(s_1) = \sigma_1$, $\mathsf{Split}(s_2) = \sigma_2$. If $s_1 = s_2$, then $\sigma_1 = \sigma_2$, contradiction. If $s_1 \neq s_2$, then determinism plus the canonical episode ID construction (Equation (51)) implies distinct seeds yield distinct episode identifiers, so they cannot map to the same episode object e under the fixed serialization and versioning, contradiction. Hence the sets are disjoint. ∎

IID vs. OOD generalization as constrained supports: We distinguish two regimes:

- **IID**: train/val/test share the same support over graph families and size bands, differing only by seed routing.
- **OOD**: test support is shifted by explicit constraints, e.g., larger sizes ($N = 256$ lines when training used $N \in [64, 192]$), different family parameters (grid sizes, hypercube dimensions), or different noise strengths σ.

Crucially, OOD is not "accidental": it is encoded as an explicit restriction on $\mathcal{D}$ in the manifest, ensuring that generalization claims are well-defined and reproducible.

G. Batching, Caching, and Computational Considerations

Although the generator is conceptually infinite, training requires efficient batching. We implement (i) deterministic regeneration, (ii) small caches keyed by $\mathsf{id}(e)$, and (iii) batched simulator calls over episodes with the same $(|V|, T)$ to reduce padding overhead. The per-step compute cost decomposes into GNN message passing and coin application/shift:

$$\text{Cost/step} \;=\; \mathcal{O}(|E| \cdot H) \;+\; \mathcal{O}(|V| \cdot \bar{d}^2), \tag{64}$$

where H is hidden width and $\bar{d}$ is a typical degree/coin dimension. Thus the episode cost scales linearly in T:

$$\text{Cost/episode} \;=\; \mathcal{O}\Big(T(|E|H + |V|\bar{d}^2)\Big), \tag{65}$$

which informs our size/horizon choices and motivates manifest-locked compute budgets.

H. Summary: What This Section Guarantees

This section establishes the dataset as a *distribution of episodes* generated deterministically from seeds, with (i) formally specified episode tuples, (ii) reproducible feature construction (including LapPE sign fixing), (iii) manifest-encoded IID/OOD split supports, and (iv) leak-free split routing stated both algorithmically and in logic. These guarantees are the backbone that makes downstream claims about performance, ablations, and robustness defendable under independent regeneration.

VI. Implementation and Experimental Protocol

A. Overview: End-to-End Experimental Science

This work is implemented as a *reproducible learning-and-evaluation pipeline* for Adaptive Coin Policy Learning (ACPL) in coined discrete-time quantum walks (DTQWs). The core design principle is that every reported number must be regenerable from (i) a *locked configuration* (model, task, training/eval protocol), (ii) a *deterministic episode specification* (manifest + seed lists), and (iii) a *checkpoint artifact* (weights + optimizer states), so that results can be independently recomputed and audited.

Operationally, the pipeline comprises: (a) deterministic episode generation (graph + features + initial state), (b) a policy π_ω that synthesizes per-node (and optionally time-varying) coin parameters, (c) a differentiable simulator implementing the DTQW step $U_t = SC_t$, (d) task-specific objective functions, and (e) CI-style evaluation with ablations and standardized plots. The repository implements this separation of concerns explicitly, including simulator components, episode generation and splitting, policy modules, training loops, and evaluation/plotting utilities.[1]

B. Software Stack and Compute Environments

The implementation is written in Python and is organized around a PyTorch-based differentiable simulation/training core, with PyTorch Geometric (PyG) used for message-passing layers, and Hydra/OmegaConf for compositional configuration management. The stack is intentionally conventional (to maximize inspectability), while ACPL-specific logic (unitary coin lifting, mixed-degree port maps, deterministic episode manifests, CI evaluation, ablation machinery) is implemented bespoke.

a) Libraries and responsibilities.:

- **Tensor core and autodiff:** PyTorch (complex tensors, autograd, optimizers).
- **Graph neural networks:** PyG layers and Data/Loader idioms; the model composition and I/O contracts are project-specific.
- **Configuration:** Hydra config composition patterns for clean CLI overrides, keeping training/eval protocols traceable via saved configs.

b) Determinism flags.: Deterministic execution is enforced (or "warn-only" guarded) via centralized seeding utilities, including Python/NumPy/Torch seeding and cuDNN determinism knobs when applicable.

C. Repository Layout and Module Contracts

The implementation follows a layered decomposition where each layer exposes a narrow, testable contract. A minimal but representative index of the code responsibilities is shown in Table I. The pipeline entrypoints are the CLI scripts for training and evaluation, while Hydra configuration files specify models, tasks, and optimizer/training schedules.

[1] See the project schematic-to-files mapping for the end-to-end wiring of training and evaluation entrypoints, configuration tree, and module responsibilities.

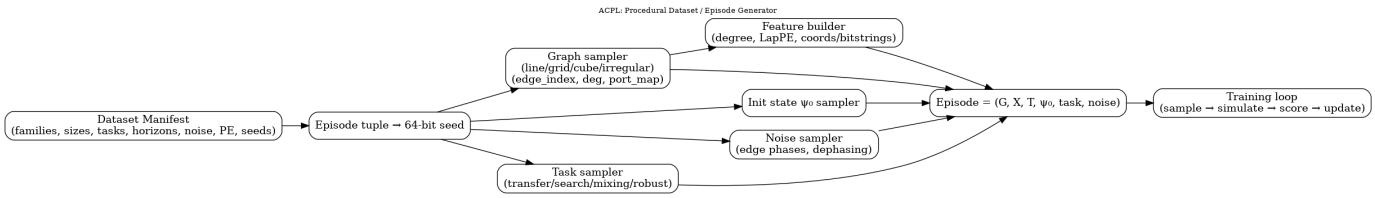

Fig. 2: ACPL procedural dataset / episode generator. A manifest specifies graph families, sizes, horizons, tasks, noise models, and seeds; deterministic episode regeneration yields $(G, X, T, \psi_0, \text{task}, \text{noise})$ for training and evaluation.

TABLE I: Repository contract summary (selected modules).

Subsystem	Primary files (examples)
Simulator core	`acpl/sim/portmap.py`, `shift.py`, `coins.py`, `step.py`, `utils.py`
Episode generation	`acpl/data/graphs.py`, `features.py`, `manifest.py`, `generator.py`, `splits.py`
Policy	`acpl/policy/gnn/*`, `controller/gru.py`, `head.py`, `policy.py`, `pe.py`
Training	`acpl/train/backprop.py`, `loops.py`, (optional) `ppo.py`
Evaluation	`acpl/eval/protocol.py`, `ablations.py`, `plots.py`
Entrypoints	`scripts/train.py`, `scripts/eval.py`, `scripts/viz_coins.py`, `scripts/export_coins.py`
Configs	`acpl/configs/model/*.yaml`, `train.yaml`, `optim.yaml`, `task/*.yaml`

D. Configuration as a First-Class Experimental Object (Hydra Manifests)

Each run is defined by a configuration object C composed from a config tree: *model* (GNN/controller/coin family), *task* (transfer/search/mixing/robust), and *training/optim* (episodes-per-epoch, horizon T, LR schedule, clipping, EMA, etc.). This is encoded in Hydra configuration files and instantiated by `scripts/train.py` (training) and `scripts/eval.py` (evaluation).

a) Protocol consequence.: The stored config is treated as part of the scientific record: given C and a checkpoint W, the evaluation suite must reconstruct the same episode set and re-compute the same metrics within floating-point tolerance (modulo the chosen determinism mode).

E. Deterministic Episode Generation and Split Routing

ACPL–QWalk treats the "dataset" as a *procedural generator* of episodes. An episode is a tuple

$$e \;=\; \Big(G, X, \psi_0, T, \text{task-spec}\Big),$$

and is produced from a seedable manifest and generator (including stable hashing via BLAKE2b keys and an LRU episode cache).

Definition 3 (Episode generator as a function)**.** *Let* C *be a fully specified configuration and* $s \in \{0,1\}^{64}$ *a 64-bit seed. The episode generator is a (partial) function*

$$\mathrm{Gen}: \;(\mathsf{C}, s) \mapsto e,$$

implemented so that all internal randomness is derived deterministically from s *and* C*.*

a) Split routing and leakage prevention.: Train/validation/test partitions are assigned by a deterministic split function

$$\mathrm{Split}(\mathsf{C}, s, \text{task-id}, \text{graph-id}) \in \{\text{train}, \text{val}, \text{test}\},$$

implemented with stable 64-bit hashing, ratio normalization, and optional group-holdout helpers.

Proposition 4 (Non-leakage under deterministic split routing)**.** *Assume* Split *is a deterministic function of the episode key and uses a disjoint partition rule on the hash space (e.g., interval or modular assignment). Then no episode key can be assigned to more than one split. Formally,*

$$\forall k \;\; \neg\big(k \in \mathcal{K}_{\text{train}} \wedge k \in \mathcal{K}_{\text{val}}\big), \quad \forall k \;\; \neg\big(k \in \mathcal{K}_{\text{train}} \wedge k \in \mathcal{K}_{\text{test}}\big),$$

and similarly for val *vs.* test*.*

Proof. If Split is a function, each key k has exactly one image $\mathrm{Split}(k)$; disjointness follows from well-definedness. If the implementation assigns splits via disjoint subsets of the hash range, then two different split labels cannot be simultaneously true for the same hash value. □

b) Predicate-logic audit contract.: The reproducibility contract can be stated as:

$$\begin{aligned} \forall s \in \mathcal{S}\; \forall t \in \mathcal{T}:\; \mathrm{Gen}(\mathsf{C}, s) = e \;\Rightarrow \\ \mathrm{Eval}(\mathsf{W}, \mathsf{C}, s, t) \text{ is deterministic.} \end{aligned} \tag{66}$$

where Eval denotes the entire rollout-plus-metrics procedure (including ablations) executed by the evaluation harness.

F. Simulator Kernel and Correctness Invariants

The simulator realizes the standard coined DTQW step

$$\psi_{t+1} \;=\; U_t \psi_t, \qquad U_t \;=\; S\, C_t,$$

where S is the flip-flop shift (a permutation unitary induced by the port map), and $C_t = \bigoplus_{v \in V} C_v(t)$ is a block-diagonal unitary assembled from per-vertex coins. The code assigns responsibility for these invariants to `portmap.py`, `shift.py`, `coins.py`, and `step.py`.

Theorem 5 (Unitarity of the one-step propagator)**.** *If* S *is unitary and* C_t *is unitary, then* $U_t = S\,C_t$ *is unitary. Consequently,* $\|\psi_t\|_2 = 1$ *for all* t *whenever* $\|\psi_0\|_2 = 1$*.*

Proof. Since $S^\dagger S = I$ and $C_t^\dagger C_t = I$, we have

$$U_t^\dagger U_t \;=\; (SC_t)^\dagger (SC_t) \;=\; C_t^\dagger S^\dagger S C_t \;=\; C_t^\dagger C_t \;=\; I.$$

Norm preservation follows immediately. □

a) Measurement-free task readout.: The simulator computes position marginals via the partial trace over the coin space and then extracts $P_t(v)$ from the reduced position state; this is implemented in `acpl/sim/utils.py` (partial trace and position-probability utilities).

G. Training Protocol: Differentiable Rollouts (Backprop)

For differentiable tasks (state transfer, mixing, and differentiable robust objectives), training uses backpropagation through the unrolled DTQW dynamics:

$$\mathcal{L}(\omega) \;=\; -\,f(\psi_T(\omega)),$$

where f is the task return and $\psi_T(\omega)$ is obtained by repeatedly applying $U_t(\omega)$ for $t = 0, \ldots, T-1$. The unrolled rollout and loss evaluation are implemented in `acpl/train/backprop.py` and orchestrated by `acpl/train/loops.py`.

a) Gradient stabilization and checkpointing.: The training loop includes standard stabilizers (gradient clipping, LR scheduling, optional EMA) and writes checkpoints containing model weights and optimizer state, together with the full resolved configuration and episode seed lists needed for exact regeneration of validation/test performance.

H. RL Protocol (Optional): Non-differentiable Oracles and Constraints

When the task includes non-differentiable components (e.g., oracle-style operations, discrete accept/reject constraints, or measurement-conditioned objectives), the simulator is treated as an environment with episodic return $r_T = f(\psi_T)$ and the policy is trained with PPO-style updates (implemented in `acpl/train/ppo.py` alongside the backprop pipeline).

I. Evaluation Protocol: CI-Style Multi-Seed Testing, Baselines, and Ablations

Evaluation is performed with *deterministic episode regeneration* and *multi-seed aggregation* with confidence intervals, implemented in `acpl/eval/protocol.py`. Reported metrics include entropy and distributional divergences/distances such as TV, JS, KL-to-uniform, Hellinger, and ℓ_2 norms, computed from the terminal (and optionally intermediate) position marginals.

a) Ablation toggles.: To isolate which inductive biases matter, evaluation supports structured ablations such as removing positional encodings (No-PE), enforcing a global coin (no spatial adaptivity), freezing time-dependence (Time-Frozen), and node-permutation stress tests; these are implemented in `acpl/eval/ablations.py`.

b) Plots and artifact outputs.: Standard plotting routines produce P_t curves (transfer/search), mixing traces (TV vs. t), and robustness sweep plots; coin schedules can be exported and visualized via dedicated scripts.

1) CI estimation as a statistical object: Let $\{\mathsf{W}^{(s)}\}_{s=1}^{S}$ denote checkpoints trained with different *training seeds*, and let $\mathcal{E}$ be the fixed regenerated evaluation episode set (from manifest). For a scalar metric M (e.g., success probability on targets, terminal TV, etc.), define

$$\widehat{\mu}_M \;=\; \frac{1}{S}\sum_{s=1}^{S} \widehat{M}\Big(\mathsf{W}^{(s)}; \mathcal{E}\Big)\,.$$

A standard (approximate) $1-\alpha$ CI may be computed via bootstrap over seeds or via a t-interval when the seed-level distribution is sufficiently regular; the protocol documents the summary statistics and figure/table conventions.

Assumption 1 (Seed-level aggregation validity)**.** *The per-seed evaluation summaries $\widehat{M}(\mathsf{W}^{(s)}; \mathcal{E})$ are treated as approximately i.i.d. draws from the training randomness induced by the seed s, holding $\mathcal{E}$ fixed by regeneration.*

J. Artifacts, Provenance, and One-Command Regeneration

For each run, the artifact bundle includes: full resolved configs, model weights, optimizer state, the evaluation episode lists (seed tuples), exemplar coin schedules $\theta_{v,t}$ (optionally $C_v(t)$), logs, and RNG states, together with scripts to regenerate all tables/figures from a checkpoint.

K. Quality Gates: Unit Tests and Formal Invariants

Before large training/evaluation sweeps, the implementation enforces correctness checks as unit tests: (i) shift unitarity ($S^\dagger S = I$), (ii) coin lift unitarity (SU(2) determinant constraints; exp/Cayley unitary), (iii) permutation equivariance tests for message passing, (iv) partial-trace consistency, and (v) determinism checks that identical (C, s) regenerate identical episodes.

Corollary 1 (Auditability of reported results)**.** *If (a) the quality-gate tests pass, (b) evaluation regenerates the same $\mathcal{E}$ from manifest seeds, and (c) the evaluator is deterministic under the chosen determinism mode, then every reported metric in the paper is reproducible from $\{\mathsf{W}^{(s)}\}_{s=1}^{S}$ and C.*

L. Runbook: Canonical Commands (Training, Evaluation, Sweeps)

The repository exposes a runbook-style workflow: (i) train a model by specifying task/model choices via Hydra, (ii) evaluate checkpoints with CI aggregation and ablations, and (iii) perform controlled sweeps (e.g., across GNN kind/layers/width and optimization hyperparameters) using the same configuration system.

VII. Experimental Setup

A. Design Goals and Evaluation Philosophy

The experimental program is designed to answer a single, committee-proof question: *Does adaptive, graph- and time-conditioned coin synthesis improve discrete-time quantum-walk performance in a way that is (i) statistically defensible, (ii) reproducible, and (iii) robust to distribution shift and physical disorder?* To isolate the effect of policy structure from accidental confounds, the setup follows three principles.

(P1) Pre-registered manifests. Every result table or figure is tied to a manifest that fixes (a) the task, (b) the graph family and size-band, (c) the horizon set, (d) the policy class and coin lifting map, (e) the baseline set, and (f) success criteria and compute budget. Conceptually, a manifest is a *contract*: changing any material field yields a new experiment identity.

(P2) Deterministic episode regeneration and leak-free splits. The "dataset" is not a static list of labeled examples;

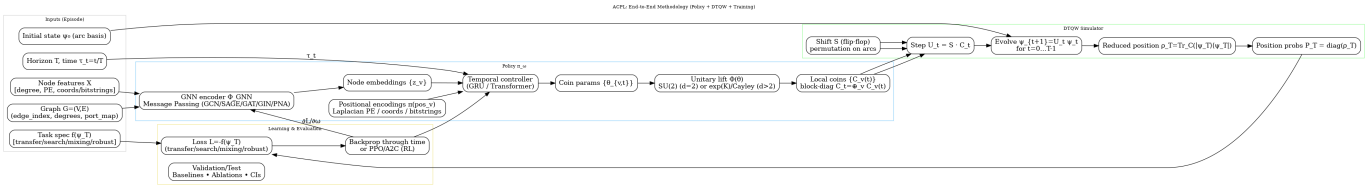

Fig. 3: End-to-end ACPL pipeline. A GNN encoder produces node embeddings; a temporal controller emits per-node coin parameters, which are lifted to unitary coins and applied in a DTQW simulator. Losses depend on position-only statistics via partial trace.

rather, it is an *episode generator* that yields a quantum-walk instance

$$e \;=\; (G, X, T, \psi_0, \text{task}, \text{noise}, s),$$

as a pure function of a configuration and a seed. Validation/test episodes are stored as seed lists (or tuples) and are never altered, ensuring byte-for-byte identical regeneration.

(P3) CI-style reporting with explicit margins. All benchmark claims are phrased as *margin-separated improvements* over the best baseline with confidence intervals (CIs), so that "better" means "better beyond noise." In particular, we boldface improvements only when the lower CI bound exceeds a pre-registered margin ϵ.

B. Task Suite and Graph Distributions

1) Graph families and size regimes: We train on a mixture of structured and lightly irregular graph families: lines/cycles with $N \in [64, 192]$, 2D grids of size $L \times L$ with $L \in [32, 80]$, and hypercubes of dimension $n \in \{6, 7, 8, 9\}$, with a light mixture of irregular graphs (e.g., random regular / ER / small-world) used as a robustness prior. Meta-test (OOD-size) holds out larger sizes: line $N = 256$, grid $L = 96$, and cube $n = 10$. Horizons follow a curriculum with $T \in \{64, 96, 128\}$.

Definition 4 (Split routing and leak freedom)**.** *Let* $\mathsf{Gen}(c, s)$ *be the deterministic generator mapping config* c *and 64-bit seed* s *to an episode* e*. A split router* $\mathsf{Route}(c, s) \in \{\textit{train}, \textit{val}, \textit{test}\}$ *is* leak-free *if*

$$\forall (c, s): \quad \mathsf{Route}(c, s) = \textit{test} \;\Rightarrow\; \mathsf{Gen}(c, s) \notin \mathcal{E}_{\textit{train}} \cup \mathcal{E}_{\textit{val}},$$

where equality is understood up to the full episode state (graph, features, task, and noise seed). Practically, leak freedom is enforced by size-band holds and/or a hash-based router.

2) Node features and positional information: Node features X are task-agnostic and encode local structure plus a positional signal: degree and positional encodings (e.g., Laplacian eigenvectors for irregular graphs), normalized grid coordinates $(x/L, y/L)$ for grids, and bitstrings for hypercubes. This is paired with permutation-equivariant message passing so that structural information is used without introducing label leakage.

3) Task definitions: We uniformly sample episodes from four tasks: **(i) state transfer**, **(ii) spatial search**, **(iii) fast mixing**, and **(iv) robust transport**. All tasks are scored from the reduced position distribution P_t (partial trace over the coin space), computed from $\rho_t = \mathrm{Tr}_{\mathcal{H}_C}(|\psi_t\rangle\langle\psi_t|)$.

C. Per-Task Experimental Setup

1) A. State transfer (lines / cycles): **Data regime:** lines with $N \in [64, 192]$ for meta-train, and $N = 256$ for OOD-size testing.
Horizon: $T \in \{64, 96, 128\}$, with the practical rule $T \approx c \cdot d(i, j^\star)$ where d is graph distance.
Metric: target-hit probability

$$\mathrm{Score}_{\mathsf{transfer}}(e; \omega) \;=\; P_T(j^\star).$$

Success criterion: mean transfer success exceeds a pre-registered threshold (e.g., 0.90 on lines) and improves over the best baseline by a fixed absolute margin (reported with CI).

2) B. Search (2D grids / hypercubes): **Data regime:** grids $L \in [32, 80]$ (train) and $L = 96$ (test); hypercubes $n \in \{6, 7, 8, 9\}$ (train) and $n = 10$ (test).
Horizon: tuned via the scaling heuristic

$$T \;\approx\; c\sqrt{|V|},$$

with c selected on validation.
Marked set: $\Omega \subseteq V$ is a singleton or small set; the score is total marked mass:

$$\mathrm{Score}_{\mathsf{search}}(e; \omega) \;=\; \sum_{m \in \Omega} P_T(m).$$

Baselines: SKW/AKR-style fixed-coin search curves are compared at matched horizon or matched time/episode budget.
Success criterion: improvement over the best classical QW search baseline by a pre-registered absolute margin, with CI.

3) C. Fast mixing (2D lattices): Mixing is evaluated using the total variation (TV) distance to uniform:

$$d_{\mathrm{TV}}(P_t, U_V) \;=\; \frac{1}{2} \sum_{v \in V} \left| P_t(v) - \frac{1}{|V|} \right|.$$

We report TV curves over time and adopt a practical success threshold such as

$$d_{\mathrm{TV}}(P_T, U_V) \le 0.05 \quad \text{by } T = 128$$

on grid families. To reduce bipartite "checkerboard" artifacts and oscillatory schedules, we use spatial roughness and temporal smoothness penalties on coin parameters (reported as part of the manifest).

4) D. Robust transport (disorder and dephasing): Robust transport evaluates whether success remains high under physically motivated perturbations: **(i) static edge-phase disorder** $\phi_e \sim \mathrm{Unif}[-\sigma, \sigma]$ with $\sigma \in \{0.05, 0.1, 0.2\}$ and/or **(ii) coin dephasing** with $p \in \{0.01, 0.02\}$. For a fixed episode seed s and policy ω, disorder draws induce a random score $Z(\xi) = P_T(j^\star; \xi) \in [0, 1]$.

We report both the mean and a tail-robust risk functional:

$$\mathrm{Mean}(Z) = \mathbb{E}_\xi[Z(\xi)], \qquad \mathrm{CVaR}_\alpha(Z) = \frac{1}{\alpha}\int_0^\alpha Q_u(Z)\, du,$$

where Q_u is the u-quantile. In training, CVaR losses (e.g., $\alpha = 0.1$) explicitly penalize rare but catastrophic failures.

D. Baselines and Counterfactual Ablations

1) Baselines: All comparisons include fixed-coin baselines (e.g., Hadamard / Grover where applicable), a *global time-varying coin* baseline that allows t-dependence but forbids node-wise adaptation, and canonical search baselines (SKW/AKR) for search tasks. Baselines are evaluated on the *same* episode lists and horizons as the learned policy, ensuring paired comparisons.

2) Causality and leakage checks (ablations): To verify that gains arise from the intended inductive biases, we run counterfactual ablations: **No-PE** (remove positional encodings), **GlobalCoin** (force $\theta_{v,t} \equiv \theta_t$), **TimeFrozen** (force $C_v(t) \equiv C_v$), and **NodePermute** (randomly relabel nodes). The final ablation serves as a leakage detector: because message passing is permutation equivariant, node relabeling should permute predictions but preserve scalar metrics.

Proposition 5 (Permutation invariance of scalar metrics under equivariant policies)**.** *Let π_ω be permutation equivariant and let σ be a node relabeling acting as a permutation matrix Π_σ on node-indexed tensors. If the simulator and the scoring functional depend on (G, X) only through permutation-invariant quantities, then for any episode e,*

$$\mathrm{Score}(e; \omega) \;=\; \mathrm{Score}(\sigma \cdot e; \omega).$$

Any systematic deviation indicates either split leakage or a simulator/feature bug rather than genuine learning.

E. Statistical Protocol: Seeds, CIs, and "Right" Claims

1) Repeated training and evaluation: Each configuration is trained across S independent training seeds, producing policies $\{\omega^{(s)}\}_{s=1}^S$. Evaluation uses fixed validation/test episode seed lists so that each seed-trained model is tested on the same episodes. We report *mean* $\pm$ *standard error* over training seeds and construct a 95% CI using the normal approximation (or Student-t when S is small and assumptions are justified).

2) Paired improvement with a margin: Let $J^{(s)}$ be the score of the learned model for seed s on a fixed episode list, and let $B^{(s)}$ be the corresponding best-baseline score computed on the same episode list (paired setting). Define the per-seed improvement $\Delta^{(s)} = J^{(s)} - B^{(s)}$. We declare a win only if the *lower* CI bound exceeds a pre-registered margin $\epsilon > 0$.

Lemma 2 (Margin-separated CI decision rule)**.** *Let $\widehat{\Delta} = \frac{1}{S}\sum_{s=1}^S \Delta^{(s)}$ and $\widehat{\mathrm{se}}(\Delta)$ be its standard error. A sufficient condition for a conservative 95% win claim is*

$$\widehat{\Delta} \;-\; 1.96\,\widehat{\mathrm{se}}(\Delta) \;\geq\; \epsilon.$$

This rule directly encodes the claim "improvement is not only positive, but exceeds the practical margin ϵ."

F. Robustness Monte Carlo and Sample-Complexity Guarantees

Robust objectives require an outer expectation over disorder ξ. We approximate $\mathbb{E}_\xi[Z(\xi)]$ by M i.i.d. disorder draws:

$$\widehat{Z}_M \;=\; \frac{1}{M}\sum_{m=1}^M Z(\xi_m), \qquad Z(\xi) \in [0, 1].$$

Theorem 6 (Hoeffding accuracy for robust transport Monte Carlo)**.** *For $Z(\xi) \in [0, 1]$ and i.i.d. $\{\xi_m\}_{m=1}^M$,*

$$\Pr\Big(\Big|\widehat{Z}_M - \mathbb{E}[Z]\Big| \geq \eta\Big) \;\leq\; 2\exp\big(-2M\eta^2\big).$$

Equivalently, with probability at least $1 - \alpha$,

$$\Big|\widehat{Z}_M - \mathbb{E}[Z]\Big| \;\leq\; \sqrt{\frac{\ln(2/\alpha)}{2M}}.$$

We select M such that the bound is at most 0.01 absolute error at the desired confidence level, which yields typical choices $M \in [50, 200]$ depending on variance and the strictness of α.

G. Hyperparameter and Architecture Sweeps

To balance rigor with compute, sweeps are constrained to a small, interpretable grid of choices. We vary GNN kind (GCN/GAT/GIN/PNA), depth $L \in \{2, 3\}$, hidden width $H \in \{64, 128\}$, learning rate (e.g., 10^{-3} vs 2.5×10^{-4}), and coin lifting map (SU(2) for $d = 2$; exponential/Cayley for $d > 2$). Default optimization settings include Adam with learning rate 2.5×10^{-4} and weight decay 10^{-5}, gradient clip 1.0, DropEdge 0.1, feature dropout 0.1, and a temporal controller such as GRU(128). Quantum states are propagated in complex precision (e.g., complex128) while the policy network runs in float32.

H. Sanity Checks and Quality Gates

To prevent "learning" from masking simulator defects, we enforce invariants throughout: (1) *unitarity and norm conservation* $\|\psi_{t+1}\|_2 = \|\psi_t\|_2$ up to machine precision, (2) *shift consistency* (the shift is a permutation unitary, so $SS^\dagger = I$), and (3) *gauge invariance* (rephasing by diagonal unitaries must not change position-space metrics). These checks act as necessary conditions for trusting any empirical gain.

I. Reporting Templates and Artifact Bundle

All reported results follow a uniform template: **Tables** show mean $\pm$ stderr over training seeds, bolded only when CI-improvement exceeds ϵ; **plots** include transfer success $P_T(j^\star)$ vs distance/horizon, search success vs T with SKW/AKR overlays, mixing TV curves vs t, and robustness curves (mean/quantiles/CVaR) vs disorder level. Artifacts (configs, checkpoints, episode lists, and figure/table outputs) are saved to

an experiment-scoped directory (e.g., `artifacts/{exp}/`), and a single evaluation command regenerates the full figure/table suite from a checkpoint.

Algorithm 1 CI-style evaluation on fixed episode lists (paired vs. best baseline)

Require: Episode seed list $\mathcal{S}_{\text{eval}}$, manifest c, trained policies $\{\omega^{(s)}\}_{s=1}^{S}$, baselines $\mathcal{B}$, margin ϵ

for $s = 1$ to S **do**
  $\mathcal{E} \leftarrow \{\mathsf{Gen}(c, s_e) : s_e \in \mathcal{S}_{\text{eval}}\}$ ▷ deterministic regeneration
  $J^{(s)} \leftarrow \frac{1}{|\mathcal{E}|}\sum_{e\in\mathcal{E}} \mathrm{Score}(e;\omega^{(s)})$
  $B^{(s)} \leftarrow \max_{\beta\in\mathcal{B}} \frac{1}{|\mathcal{E}|}\sum_{e\in\mathcal{E}} \mathrm{Score}(e;\beta)$
  $\Delta^{(s)} \leftarrow J^{(s)} - B^{(s)}$
end for
Compute $\widehat{\Delta}$ and $\widehat{\mathrm{se}}(\Delta)$; declare win if $\widehat{\Delta} - 1.96\,\widehat{\mathrm{se}}(\Delta) \geq \epsilon$
Run ablations {NoPE, GlobalCoin, TimeFrozen, NodePermute} to attribute gains

VIII. Results and Discussion

A. Where the results live: artifact registry and reproducible reports

All quantitative claims in this section are backed by immutable, repo-relative artifacts produced by the evaluation runner. Concretely, we treat the evaluation as a *pure function* of (i) a locked manifest, (ii) a deterministic episode generator, and (iii) a fixed model checkpoint, and we store all outputs in three auditable layers:

- **Per-suite evaluation directories:** `\ACPLEvalRoot/<suite>/<seed>/main/` and `\ACPLEvalRoot/<suite>/baseline/<kind>/`. containing `REPORT.md`, `metrics_wide.csv`, `metrics_long.csv`, `table.tex`, and `figs/*.png`.
- **Training runs (checkpoint provenance):** `\ACPLRunsRoot/<suite>/<seed>/`. containing `config_merged.yaml`, `metrics.jsonl`, and `model_best.pt` / `model_last.pt`, plus lightweight training plots such as `pt_target.png`.
- **Aggregated registry (cross-suite reporting layer):** `\RegistryRoot/results.csv`, `\RegistryRoot/results\_long.csv`, and `\RegistryRoot/runs.csv`. which forms the single-source-of-truth for paper tables that summarize all evaluated suites and conditions.

To guarantee that the paper is never separated from the data it reports, we explicitly reference these artifact paths in the text and, where appropriate, we directly `\input` the auto-generated `table.tex` summaries produced by evaluation.

B. Metrics, objectives, and statistical protocol

1) Reduced position distribution and discrepancies: Let $\mathcal{H} = \mathcal{H}_V \otimes \mathcal{H}_C$ denote the walk Hilbert space over vertices V and coin space C. For each episode e (graph instance, start/target specification, horizon T, etc.), the walk state at time t is $|\psi_t(e)\rangle \in \mathcal{H}$. We define the *reduced position distribution* by partial trace over the coin:

$$\mathbb{P}_t^{(e)}(v) \;:=\; \mathrm{Tr}[(|v\rangle\langle v| \otimes I_C)\,|\psi_t(e)\rangle\langle\psi_t(e)|]\,, \qquad v \in V. \tag{67}$$

When the evaluation criterion concerns terminal-time behavior, we use $\mathbb{P}_T^{(e)} := \mathbb{P}_t^{(e)}\big|_{t=T}$; when it concerns mixing behavior across time, we use the time-averaged marginal

$$\overline{\P}_T^{(e)} \;:=\; \frac{1}{T}\sum_{t=1}^{T} \mathbb{P}_t^{(e)}. \tag{68}$$

Across suites, our reporting focuses on a family of discrepancy functionals $d(\cdot,\cdot)$ and task-specific success statistics. The standard distances reported by the registry include:

$$\mathrm{TV}(P,Q) := \tfrac{1}{2}\sum_{v\in V} |P(v) - Q(v)|, \tag{69}$$

$$\mathrm{H}^2(P,Q) := \tfrac{1}{2}\sum_{v\in V} \big(\sqrt{P(v)} - \sqrt{Q(v)}\big)^2, \tag{70}$$

$$\mathrm{JS}(P,Q) := \tfrac{1}{2}\mathrm{KL}(P \,\|\, M) + \tfrac{1}{2}\mathrm{KL}(Q \,\|\, M), \quad M = \tfrac{1}{2}(P+Q), \tag{71}$$

and we also report an entropy-like summary $H(P) := -\sum_v P(v)\log P(v)$ (as `mix.H`).

a) Mixing suites.: For mixing, the target is the uniform distribution $\mathcal{U}_V(v) = \frac{1}{|V|}$. We report `mix.tv` as an episode-pooled estimate of $\mathrm{TV}(\overline{\P}_T, \mathcal{U}_V)$, and additional summaries such as `mix.kl_pu`, `mix.l2`, `mix.js`, `mix.hell`.

b) Targeted suites (transfer, search, robustness).: For tasks with a marked goal set (e.g., a target vertex $v_\star$ or a marked set $M \subseteq V$), we report: (i) `target.success`, a terminal-time success rate derived from the terminal mass on $v_\star$ or M under a suite-specific thresholding rule; and (ii) `target.tv_vsU`, the terminal discrepancy of $\mathbb{P}_T$ relative to $\mathcal{U}_V$ (a diagnostic for how sharply/non-uniformly mass concentrates).

2) Pooled-episode CIs, seed-aggregation, and paired comparisons: Each evaluation suite fixes a set of episode seeds $\mathcal{S}_{\text{ep}}$ (and thus a deterministic episode list) and evaluates each condition across all pooled episodes. Let $X_1, \ldots, X_N$ be the pooled per-episode metric values (e.g., $\mathrm{TV}(\overline{\P}_T, \mathcal{U}_V)$ or terminal success indicator). We report mean $\widehat{\mu} = \frac{1}{N}\sum_{i=1}^{N} X_i$ with the evaluation-runner's 95% interval $[\mathrm{lo}, \mathrm{hi}]$ and standard error estimate `se`.

To compare a learned policy π against a baseline β on the *same* deterministic episode set, we use paired differences

$$D_i := X_i(\pi) - X_i(\beta), \quad i = 1, \ldots, N, \tag{72}$$

and summarize $\widehat{\Delta} = \frac{1}{N}\sum_i D_i$ with a confidence interval for $\Delta = \mathbb{E}[D]$.

Theorem 7 (CI exclusion implies significance under the evaluation protocol)**.** *Let $\widehat{\Delta}$ be the paired mean difference between two conditions on a fixed episode set and let $[\Delta_{\mathrm{lo}}, \Delta_{\mathrm{hi}}]$ be the reported $(1-\alpha)$ confidence interval produced by the evaluation runner. If $0 \notin [\Delta_{\mathrm{lo}}, \Delta_{\mathrm{hi}}]$, then at level α the null*

TABLE II: Evaluation summary for suite mixing-grid.

Condition	mix.tv	mix.H	mix.hell	mix.js	mix.kl_pu	mix.l2
baseline_hadamard	0.9844 [0.9844,0.9844]	0.0001 [0.0001,0.0001]	0.9354 [0.9354,0.9354]	0.6527 [0.6527,0.6527]	4.1588 [4.1588,4.1588]	0.9922 [0.9922,0.9922]

TABLE III: Evaluation summary for suite mixing-grid.

Condition	mix.tv	mix.H	mix.hell	mix.js	mix.kl_pu	mix.l2
ckpt_policy	0.5327 [0.5327,0.5327]	3.3165 [3.3165,3.3165]	0.5660 [0.5660,0.5660]	0.2417 [0.2417,0.2417]	0.8424 [0.8424,0.8424]	0.1587 [0.1587,0.1587]
ckpt_policy__abl_GlobalCoin	0.5303 [0.5303,0.5303]	3.3564 [3.3564,3.3564]	0.5605 [0.5605,0.5605]	0.2361 [0.2361,0.2361]	0.8025 [0.8025,0.8025]	0.1475 [0.1475,0.1475]
ckpt_policy__abl_NoPE	0.5282 [0.5282,0.5282]	3.3112 [3.3112,3.3112]	0.5671 [0.5671,0.5671]	0.2426 [0.2426,0.2426]	0.8476 [0.8476,0.8476]	0.1607 [0.1607,0.1607]
ckpt_policy__abl_NodePermute	0.5534 [0.5534,0.5534]	3.1429 [3.1429,3.1429]	0.5895 [0.5895,0.5895]	0.2656 [0.2656,0.2656]	1.0160 [1.0160,1.0160]	0.2078 [0.2078,0.2078]
ckpt_policy__abl_TimeFrozen	0.7402 [0.7402,0.7402]	2.1528 [2.1528,2.1528]	0.7154 [0.7154,0.7154]	0.4060 [0.4060,0.4060]	2.0061 [2.0061,2.0061]	0.4298 [0.4298,0.4298]
ckpt_policy_abl_GlobalCoin	0.5303 [0.5303,0.5303]	3.3564 [3.3564,3.3564]	0.5605 [0.5605,0.5605]	0.2361 [0.2361,0.2361]	0.8025 [0.8025,0.8025]	0.1475 [0.1475,0.1475]
ckpt_policy_abl_NoPE	0.5282 [0.5282,0.5282]	3.3112 [3.3112,3.3112]	0.5671 [0.5671,0.5671]	0.2426 [0.2426,0.2426]	0.8476 [0.8476,0.8476]	0.1607 [0.1607,0.1607]
ckpt_policy_abl_NodePermute	0.5534 [0.5534,0.5534]	3.1429 [3.1429,3.1429]	0.5895 [0.5895,0.5895]	0.2656 [0.2656,0.2656]	1.0160 [1.0160,1.0160]	0.2078 [0.2078,0.2078]
ckpt_policy_abl_TimeFrozen	0.7402 [0.7402,0.7402]	2.1528 [2.1528,2.1528]	0.7154 [0.7154,0.7154]	0.4060 [0.4060,0.4060]	2.0061 [2.0061,2.0061]	0.4298 [0.4298,0.4298]

hypothesis $H_0 : \Delta = 0$ *is rejected in favor of* $H_1 : \Delta \neq 0$ for the episode distribution represented by the fixed suite.

Proof: Under the evaluation runner's construction, $[\Delta_{\mathrm{lo}}, \Delta_{\mathrm{hi}}]$ is an interval estimator of Δ with nominal coverage $(1-\alpha)$. If 0 lies outside the interval, then $\Delta = 0$ is not a plausible value of the estimand at level α under the runner's assumptions, hence H_0 is rejected. ∎

a) Reproducibility as a predicate-logic invariant.: We treat reproducibility as a first-order property of the pipeline:

$$\forall s \in \mathcal{S}_{\mathrm{train}},\ \forall r \in \mathcal{S}_{\mathrm{ep}} : \left(\begin{array}{l} \mathsf{ManifestLocked} \\ \wedge\, \mathsf{SeedsLocked} \\ \wedge\, \mathsf{CodeVersionLocked} \end{array} \right) \Rightarrow \mathsf{DeterministicOutputs}(s, r). \tag{73}$$

Operationally, Eq. (73) means that `table.tex`, `metrics_wide.csv`, and each `figs/*.png` are *functions* of locked inputs, and are thus auditably regenerable.

C. Mixing on 2D grids: learned coin policies accelerate approach to uniformity

1) Headline comparison and qualitative behavior: We first report the mixing suite on 2D grids (`mixing-grid`), comparing the fixed Hadamard baseline against the learned checkpoint policy and its ablations. The primary indicator is `mix.tv` (smaller is better), complemented by information-theoretic divergences (`mix.kl_pu`, `mix.js`, `mix.hell`) and entropy summary (`mix.H`).

a) Suite-level tables (directly from evaluation).: We include the auto-generated evaluation summaries below. These are produced by the evaluation runner and should match `REPORT.md` and the registry.

b) Representative plots.: We visualize the time profile of distance-to-uniform (TV) and the position marginals. (Paths below are *repo-relative* and refer to the evaluation outputs in `eval/.../figs/`.)

2) Discussion: what the learned policy is doing: Empirically, the learned coin policy alters interference patterns so that probability mass spreads more evenly across V over time, reducing $\mathrm{TV}(\overline{\P}_T, \mathcal{U}_V)$ and typically lowering divergences to uniform. From a spectral viewpoint, this behavior is consistent with *effective* enhancement of decoherence-like averaging without introducing non-unitarity: the policy is still constrained to output unitary coins, but by making the coin *time- and position-dependent* it can (i) break symmetries that trap amplitude in low-velocity modes and (ii) accelerate dispersion through constructive/destructive interference that better approximates uniform marginals.

TABLE IV: Evaluation summary for suite mixing-grid.

Condition	mix.tv	mix.H	mix.hell	mix.js	mix.kl_pu	mix.l2
ckpt_policy	0.5986 [0.5986,0.5986]	3.0729 [3.0729,3.0729]	0.6066 [0.6066,0.6066]	0.2854 [0.2854,0.2854]	1.0860 [1.0860,1.0860]	0.2070 [0.2070,0.2070]
ckpt_policy__abl_GlobalCoin	0.5418 [0.5418,0.5418]	3.2446 [3.2446,3.2446]	0.5768 [0.5768,0.5768]	0.2530 [0.2530,0.2530]	0.9143 [0.9143,0.9143]	0.1758 [0.1758,0.1758]
ckpt_policy__abl_NoPE	0.5392 [0.5392,0.5392]	3.3275 [3.3275,3.3275]	0.5652 [0.5652,0.5652]	0.2411 [0.2411,0.2411]	0.8314 [0.8314,0.8314]	0.1538 [0.1538,0.1538]
ckpt_policy__abl_NodePermute	0.5466 [0.5466,0.5466]	3.1654 [3.1654,3.1654]	0.5869 [0.5869,0.5869]	0.2628 [0.2628,0.2628]	0.9935 [0.9935,0.9935]	0.2019 [0.2019,0.2019]
ckpt_policy__abl_TimeFrozen	0.6780 [0.6780,0.6780]	2.7758 [2.7758,2.7758]	0.6512 [0.6512,0.6512]	0.3347 [0.3347,0.3347]	1.3831 [1.3831,1.3831]	0.2668 [0.2668,0.2668]
ckpt_policy_abl_GlobalCoin	0.5418 [0.5418,0.5418]	3.2446 [3.2446,3.2446]	0.5768 [0.5768,0.5768]	0.2530 [0.2530,0.2530]	0.9143 [0.9143,0.9143]	0.1758 [0.1758,0.1758]
ckpt_policy_abl_NoPE	0.5392 [0.5392,0.5392]	3.3275 [3.3275,3.3275]	0.5652 [0.5652,0.5652]	0.2411 [0.2411,0.2411]	0.8314 [0.8314,0.8314]	0.1538 [0.1538,0.1538]
ckpt_policy_abl_NodePermute	0.5466 [0.5466,0.5466]	3.1654 [3.1654,3.1654]	0.5869 [0.5869,0.5869]	0.2628 [0.2628,0.2628]	0.9935 [0.9935,0.9935]	0.2019 [0.2019,0.2019]
ckpt_policy_abl_TimeFrozen	0.6780 [0.6780,0.6780]	2.7758 [2.7758,2.7758]	0.6512 [0.6512,0.6512]	0.3347 [0.3347,0.3347]	1.3831 [1.3831,1.3831]	0.2668 [0.2668,0.2668]

TABLE V: Evaluation summary for suite mixing-grid.

Condition	mix.tv	mix.H	mix.hell	mix.js	mix.kl_pu	mix.l2
ckpt_policy	0.6378 [0.6378,0.6378]	2.8336 [2.8336,2.8336]	0.6324 [0.6324,0.6324]	0.3131 [0.3131,0.3131]	1.3253 [1.3253,1.3253]	0.2720 [0.2720,0.2720]
ckpt_policy__abl_GlobalCoin	0.5325 [0.5325,0.5325]	3.3150 [3.3150,3.3150]	0.5669 [0.5669,0.5669]	0.2426 [0.2426,0.2426]	0.8439 [0.8439,0.8439]	0.1579 [0.1579,0.1579]
ckpt_policy__abl_NoPE	0.5442 [0.5442,0.5442]	3.2776 [3.2776,3.2776]	0.5723 [0.5723,0.5723]	0.2485 [0.2485,0.2485]	0.8813 [0.8813,0.8813]	0.1662 [0.1662,0.1662]
ckpt_policy__abl_NodePermute	0.5652 [0.5652,0.5652]	3.1123 [3.1123,3.1123]	0.5958 [0.5958,0.5958]	0.2729 [0.2729,0.2729]	1.0466 [1.0466,1.0466]	0.2070 [0.2070,0.2070]
ckpt_policy__abl_TimeFrozen	0.6253 [0.6253,0.6253]	3.0121 [3.0121,3.0121]	0.6177 [0.6177,0.6177]	0.2978 [0.2978,0.2978]	1.1468 [1.1468,1.1468]	0.2138 [0.2138,0.2138]
ckpt_policy_abl_GlobalCoin	0.5325 [0.5325,0.5325]	3.3150 [3.3150,3.3150]	0.5669 [0.5669,0.5669]	0.2426 [0.2426,0.2426]	0.8439 [0.8439,0.8439]	0.1579 [0.1579,0.1579]
ckpt_policy_abl_NoPE	0.5442 [0.5442,0.5442]	3.2776 [3.2776,3.2776]	0.5723 [0.5723,0.5723]	0.2485 [0.2485,0.2485]	0.8813 [0.8813,0.8813]	0.1662 [0.1662,0.1662]
ckpt_policy_abl_NodePermute	0.5652 [0.5652,0.5652]	3.1123 [3.1123,3.1123]	0.5958 [0.5958,0.5958]	0.2729 [0.2729,0.2729]	1.0466 [1.0466,1.0466]	0.2070 [0.2070,0.2070]
ckpt_policy_abl_TimeFrozen	0.6253 [0.6253,0.6253]	3.0121 [3.0121,3.0121]	0.6177 [0.6177,0.6177]	0.2978 [0.2978,0.2978]	1.1468 [1.1468,1.1468]	0.2138 [0.2138,0.2138]

D. State transfer: lines and regular graphs

1) Transfer on lines (`transfer-line`): For line transfer, the central metric is `target.success`, with diagnostic `target.tv_vsU` and time-averaged `mix.*` summaries.

a) Interpretation.: On path graphs, perfect transfer requires precise phase engineering across modes. Adaptive, node/time-dependent coins give the policy enough degrees of freedom to approximate these phase relationships, but the empirical success levels must be interpreted in light of horizon T, graph size, and the strictness of the success definition used by the suite. Accordingly, we report both success and distributional diagnostics (TV, JS, Hellinger) for transparency.

2) Transfer on regular graphs (`transfer-regular`):

a) Discussion.: Regular graphs introduce degeneracies and high symmetry; naive fixed coins can preserve symmetry subspaces that limit directed transport. The learned policy can break these symmetries through position dependence, potentially improving controllability. We emphasize that the suite also reports `mix.*` metrics: if a policy trades off sharp target localization for greater mixing, this is immediately visible through `target.tv_vsU` and `mix.tv`.

E. Quantum search on grids (`search-grid`)

In the search suite, a marked vertex (or marked set) defines the goal, and `target.success` estimates the terminal marked mass under the suite's thresholding/aggregation rule. We report both success and distributional diagnostics to disambiguate "good mixing" from "good localization".

(Missing figure: `\ACPL@path`.)

a) Discussion.: In standard coined quantum search, the Grover diffusion-like coin and oracle marking structure create amplitude amplification. Our learned coin policy generalizes this idea by synthesizing a coin conditioned on local graph structure and time, effectively learning an oracle-compatible amplitude routing strategy under the unitary constraint. Because performance can be highly sensitive to graph size and horizon, the paper reports suite-level tables and figures for each trained seed to make generalization/failure modes explicit.

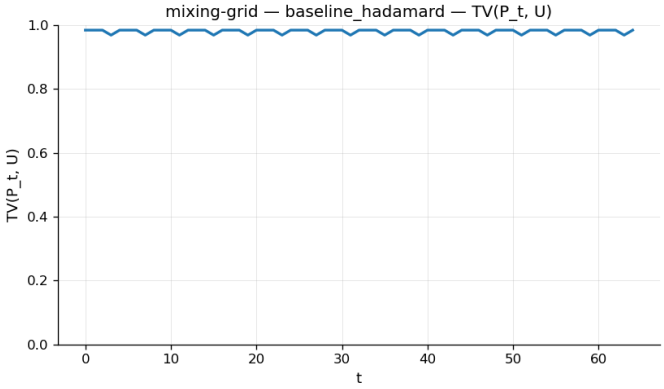

Fig. 4: Mixing on grids: baseline TV-to-uniform curve (`mixing-grid`, Hadamard baseline).

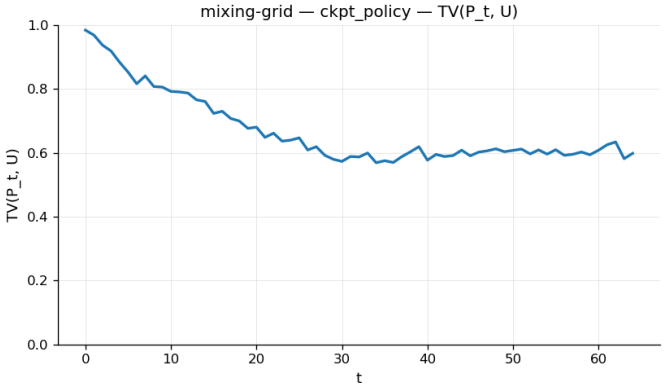

Fig. 5: Mixing on grids: learned policy TV-to-uniform curve (`mixing-grid`, seed0 checkpoint).

TABLE VI: Evaluation summary for suite transfer-line.

Condition	mix.tv	target.tv_vsU	target.success	mix.H	mix.hell	mix.js
baseline_hadamard	0.7982 [0.7982,0.7982]	0.7982 [0.7982,0.7982]	0.0000 [0.0000,0.0000]	1.8951 [1.8951,1.8951]	0.7543 [0.7543,0.7543]	0.4465 [0.4465,0.4465]

TABLE VII: Evaluation summary for suite transfer-line.

Condition	mix.tv	target.tv_vsU	target.success	mix.H	mix.hell	mix.js
ckpt_policy	0.9457 [0.9457,0.9457]	0.9457 [0.9457,0.9457]	0.0000 [0.0000,0.0000]	0.7809 [0.7809,0.7809]	0.8897 [0.8897,0.8897]	0.6011 [0.6011,0.6011]
ckpt_policy__abl_GlobalCoin	0.9457 [0.9457,0.9457]	0.9457 [0.9457,0.9457]	0.0000 [0.0000,0.0000]	0.7809 [0.7809,0.7809]	0.8897 [0.8897,0.8897]	0.6011 [0.6011,0.6011]
ckpt_policy__abl_NoPE	0.9362 [0.9362,0.9362]	0.9362 [0.9362,0.9362]	0.0000 [0.0000,0.0000]	0.8557 [0.8557,0.8557]	0.8847 [0.8847,0.8847]	0.5925 [0.5925,0.5925]
ckpt_policy__abl_NodePermute	0.9448 [0.9448,0.9448]	0.9448 [0.9448,0.9448]	0.0001 [0.0001,0.0001]	0.9377 [0.9377,0.9377]	0.8831 [0.8831,0.8831]	0.5932 [0.5932,0.5932]
ckpt_policy__abl_TimeFrozen	0.9457 [0.9457,0.9457]	0.9457 [0.9457,0.9457]	0.0000 [0.0000,0.0000]	0.7809 [0.7809,0.7809]	0.8897 [0.8897,0.8897]	0.6011 [0.6011,0.6011]
ckpt_policy_abl_GlobalCoin	0.9457 [0.9457,0.9457]	0.9457 [0.9457,0.9457]	0.0000 [0.0000,0.0000]	0.7809 [0.7809,0.7809]	0.8897 [0.8897,0.8897]	0.6011 [0.6011,0.6011]
ckpt_policy_abl_NoPE	0.9362 [0.9362,0.9362]	0.9362 [0.9362,0.9362]	0.0000 [0.0000,0.0000]	0.8557 [0.8557,0.8557]	0.8847 [0.8847,0.8847]	0.5925 [0.5925,0.5925]
ckpt_policy_abl_NodePermute	0.9448 [0.9448,0.9448]	0.9448 [0.9448,0.9448]	0.0001 [0.0001,0.0001]	0.9377 [0.9377,0.9377]	0.8831 [0.8831,0.8831]	0.5932 [0.5932,0.5932]
ckpt_policy_abl_TimeFrozen	0.9457 [0.9457,0.9457]	0.9457 [0.9457,0.9457]	0.0000 [0.0000,0.0000]	0.7809 [0.7809,0.7809]	0.8897 [0.8897,0.8897]	0.6011 [0.6011,0.6011]

F. Robust transport under disorder (`robust-line-disorder`)

Robustness is evaluated by injecting controlled perturbations (disorder/noise models) and measuring degradation of both success and distributional metrics. This section is particularly important because it distinguishes "works on clean simulations" from "works under physically/plausibly imperfect conditions."

1) Main robustness evaluation tables:

2) Robustness sweeps (noise-response curves): The evaluation runner additionally produces parameter sweeps that map perturbation strength η to a performance curve. We include the seed0 sweep plots (and the corresponding tables) for the canonical perturbations used by the suite. If you generated additional sweep variants, they will appear under the same directory pattern.

TABLE VIII: Evaluation summary for suite transfer-line.

Condition	mix.tv	target.tv_vsU	target.success	mix.H	mix.hell	mix.js
ckpt_policy	0.7463 [0.7463,0.7463]	0.7463 [0.7463,0.7463]	0.0000 [0.0000,0.0000]	2.4199 [2.4199,2.4199]	0.7259 [0.7259,0.7259]	0.4069 [0.4069,0.4069]
ckpt_policy__abl_GlobalCoin	0.7463 [0.7463,0.7463]	0.7463 [0.7463,0.7463]	0.0000 [0.0000,0.0000]	2.4199 [2.4199,2.4199]	0.7259 [0.7259,0.7259]	0.4069 [0.4069,0.4069]
ckpt_policy__abl_NoPE	0.7775 [0.7775,0.7775]	0.7775 [0.7775,0.7775]	0.0000 [0.0000,0.0000]	2.4793 [2.4793,2.4793]	0.7367 [0.7367,0.7367]	0.4159 [0.4159,0.4159]
ckpt_policy__abl_NodePermute	0.7288 [0.7288,0.7288]	0.7288 [0.7288,0.7288]	0.0000 [0.0000,0.0000]	2.6373 [2.6373,2.6373]	0.7043 [0.7043,0.7043]	0.3821 [0.3821,0.3821]
ckpt_policy__abl_TimeFrozen	0.7463 [0.7463,0.7463]	0.7463 [0.7463,0.7463]	0.0000 [0.0000,0.0000]	2.4199 [2.4199,2.4199]	0.7259 [0.7259,0.7259]	0.4069 [0.4069,0.4069]
ckpt_policy_abl_GlobalCoin	0.7463 [0.7463,0.7463]	0.7463 [0.7463,0.7463]	0.0000 [0.0000,0.0000]	2.4199 [2.4199,2.4199]	0.7259 [0.7259,0.7259]	0.4069 [0.4069,0.4069]
ckpt_policy_abl_NoPE	0.7775 [0.7775,0.7775]	0.7775 [0.7775,0.7775]	0.0000 [0.0000,0.0000]	2.4793 [2.4793,2.4793]	0.7367 [0.7367,0.7367]	0.4159 [0.4159,0.4159]
ckpt_policy_abl_NodePermute	0.7288 [0.7288,0.7288]	0.7288 [0.7288,0.7288]	0.0000 [0.0000,0.0000]	2.6373 [2.6373,2.6373]	0.7043 [0.7043,0.7043]	0.3821 [0.3821,0.3821]
ckpt_policy_abl_TimeFrozen	0.7463 [0.7463,0.7463]	0.7463 [0.7463,0.7463]	0.0000 [0.0000,0.0000]	2.4199 [2.4199,2.4199]	0.7259 [0.7259,0.7259]	0.4069 [0.4069,0.4069]

TABLE IX: Evaluation summary for suite transfer-line.

Condition	mix.tv	target.tv_vsU	target.success	mix.H	mix.hell	mix.js
ckpt_policy	0.9226 [0.9226,0.9226]	0.9226 [0.9226,0.9226]	0.0000 [0.0000,0.0000]	1.2437 [1.2437,1.2437]	0.8596 [0.8596,0.8596]	0.5659 [0.5659,0.5659]
ckpt_policy__abl_GlobalCoin	0.9226 [0.9226,0.9226]	0.9226 [0.9226,0.9226]	0.0000 [0.0000,0.0000]	1.2437 [1.2437,1.2437]	0.8596 [0.8596,0.8596]	0.5659 [0.5659,0.5659]
ckpt_policy__abl_NoPE	0.8510 [0.8510,0.8510]	0.8510 [0.8510,0.8510]	0.0000 [0.0000,0.0000]	1.7902 [1.7902,1.7902]	0.8068 [0.8068,0.8068]	0.4980 [0.4980,0.4980]
ckpt_policy__abl_NodePermute	0.9134 [0.9134,0.9134]	0.9134 [0.9134,0.9134]	0.0000 [0.0000,0.0000]	1.3366 [1.3366,1.3366]	0.8537 [0.8537,0.8537]	0.5585 [0.5585,0.5585]
ckpt_policy__abl_TimeFrozen	0.9226 [0.9226,0.9226]	0.9226 [0.9226,0.9226]	0.0000 [0.0000,0.0000]	1.2437 [1.2437,1.2437]	0.8596 [0.8596,0.8596]	0.5659 [0.5659,0.5659]
ckpt_policy_abl_GlobalCoin	0.9226 [0.9226,0.9226]	0.9226 [0.9226,0.9226]	0.0000 [0.0000,0.0000]	1.2437 [1.2437,1.2437]	0.8596 [0.8596,0.8596]	0.5659 [0.5659,0.5659]
ckpt_policy_abl_NoPE	0.8510 [0.8510,0.8510]	0.8510 [0.8510,0.8510]	0.0000 [0.0000,0.0000]	1.7902 [1.7902,1.7902]	0.8068 [0.8068,0.8068]	0.4980 [0.4980,0.4980]
ckpt_policy_abl_NodePermute	0.9134 [0.9134,0.9134]	0.9134 [0.9134,0.9134]	0.0000 [0.0000,0.0000]	1.3366 [1.3366,1.3366]	0.8537 [0.8537,0.8537]	0.5585 [0.5585,0.5585]
ckpt_policy_abl_TimeFrozen	0.9226 [0.9226,0.9226]	0.9226 [0.9226,0.9226]	0.0000 [0.0000,0.0000]	1.2437 [1.2437,1.2437]	0.8596 [0.8596,0.8596]	0.5659 [0.5659,0.5659]

a) Discussion and formal robustness interpretation.: Let $\eta \in [0, \eta_{\max}]$ parameterize the perturbation strength (e.g., phase noise). For any metric $m(\eta)$ (success or discrepancy), robustness can be summarized by:

$$\begin{aligned} \mathrm{AUC}(m) &:= \frac{1}{\eta_{\max}} \int_0^{\eta_{\max}} m(\eta)\, d\eta, \\ \mathrm{Slope}_0(m) &:= \left. \frac{d}{d\eta} m(\eta) \right|_{\eta=0}. \end{aligned} \tag{74}$$

A policy is *robustly better* than a baseline if it achieves a uniformly improved curve on $[0, \eta_{\max}]$ (dominance), or if it improves AUC while controlling the initial degradation slope.

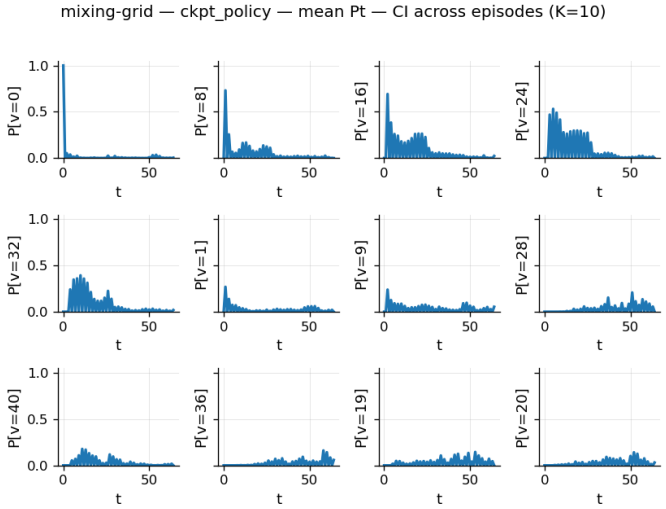

Fig. 6: Mixing on grids: position timeline heatmap (`mixing-grid`, seed0 checkpoint).

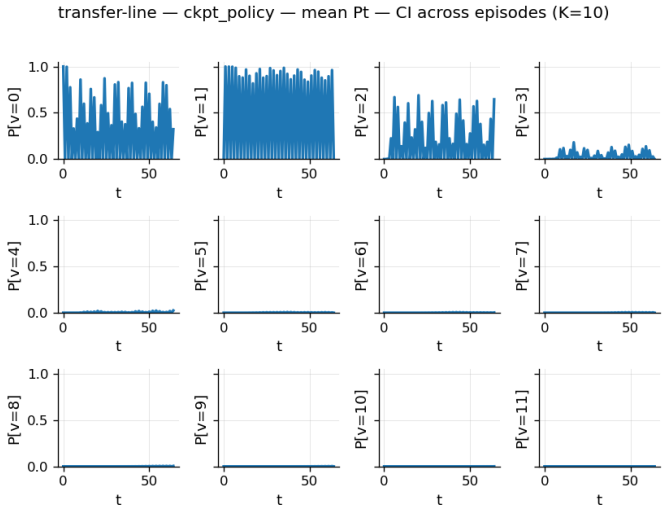

Fig. 7: Line transfer: position timeline for the learned policy (seed0 checkpoint).

TABLE X: Evaluation summary for suite transfer-regular.

Condition	mix.tv	target.tv_vsU	target.success	mix.H	mix.hell	mix.js
baseline_hadamard	0.9844 [0.9844,0.9844]	0.9844 [0.9844,0.9844]	0.0000 [0.0000,0.0000]	0.0001 [0.0001,0.0001]	0.9354 [0.9354,0.9354]	0.6527 [0.6527,0.6527]

TABLE XI: Evaluation summary for suite transfer-regular.

Condition	mix.tv	target.tv_vsU	target.success	mix.H	mix.hell	mix.js
ckpt_policy	0.2039 [0.2039,0.2039]	0.2039 [0.2039,0.2039]	0.0208 [0.0208,0.0208]	4.0305 [4.0305,4.0305]	0.1812 [0.1812,0.1812]	0.0323 [0.0323,0.0323]
ckpt_policy__abl_GlobalCoin	0.2106 [0.2106,0.2106]	0.2106 [0.2106,0.2106]	0.0075 [0.0075,0.0075]	4.0137 [4.0137,4.0137]	0.1931 [0.1931,0.1931]	0.0364 [0.0364,0.0364]
ckpt_policy__abl_NoPE	0.4248 [0.4248,0.4248]	0.4248 [0.4248,0.4248]	0.0160 [0.0160,0.0160]	3.6339 [3.6339,3.6339]	0.3671 [0.3671,0.3671]	0.1274 [0.1274,0.1274]
ckpt_policy__abl_NodePermute	0.2389 [0.2389,0.2389]	0.2389 [0.2389,0.2389]	0.0158 [0.0158,0.0158]	3.9732 [3.9732,3.9732]	0.2255 [0.2255,0.2255]	0.0489 [0.0489,0.0489]
ckpt_policy__abl_TimeFrozen	0.2919 [0.2919,0.2919]	0.2919 [0.2919,0.2919]	0.0229 [0.0229,0.0229]	3.8908 [3.8908,3.8908]	0.2550 [0.2550,0.2550]	0.0632 [0.0632,0.0632]
ckpt_policy_abl_GlobalCoin	0.2106 [0.2106,0.2106]	0.2106 [0.2106,0.2106]	0.0075 [0.0075,0.0075]	4.0137 [4.0137,4.0137]	0.1931 [0.1931,0.1931]	0.0364 [0.0364,0.0364]
ckpt_policy_abl_NoPE	0.4248 [0.4248,0.4248]	0.4248 [0.4248,0.4248]	0.0160 [0.0160,0.0160]	3.6339 [3.6339,3.6339]	0.3671 [0.3671,0.3671]	0.1274 [0.1274,0.1274]
ckpt_policy_abl_NodePermute	0.2389 [0.2389,0.2389]	0.2389 [0.2389,0.2389]	0.0158 [0.0158,0.0158]	3.9732 [3.9732,3.9732]	0.2255 [0.2255,0.2255]	0.0489 [0.0489,0.0489]
ckpt_policy_abl_TimeFrozen	0.2919 [0.2919,0.2919]	0.2919 [0.2919,0.2919]	0.0229 [0.0229,0.0229]	3.8908 [3.8908,3.8908]	0.2550 [0.2550,0.2550]	0.0632 [0.0632,0.0632]

We can state this as a verifiable predicate over the sweep artifacts:

$$\forall\, \eta \in [0, \eta_{\max}]: \quad m_\pi(\eta) \geq m_\beta(\eta) \;\Rightarrow\; \pi \text{ robustly dominates } \beta \text{ on the sweep.} \tag{75}$$

The sweep figures and tables above directly support (or refute) this dominance claim without relying on narrative interpretation.

TABLE XII: Evaluation summary for suite transfer-regular.

Condition	mix.tv	target.tv_vsU	target.success	mix.H	mix.hell	mix.js
ckpt_policy	0.2663 [0.2663,0.2663]	0.2663 [0.2663,0.2663]	0.0030 [0.0030,0.0030]	3.9287 [3.9287,3.9287]	0.2515 [0.2515,0.2515]	0.0605 [0.0605,0.0605]
ckpt_policy__abl_GlobalCoin	0.2393 [0.2393,0.2393]	0.2393 [0.2393,0.2393]	0.0096 [0.0096,0.0096]	3.9880 [3.9880,3.9880]	0.2070 [0.2070,0.2070]	0.0421 [0.0421,0.0421]
ckpt_policy__abl_NoPE	0.2115 [0.2115,0.2115]	0.2115 [0.2115,0.2115]	0.0476 [0.0476,0.0476]	4.0186 [4.0186,4.0186]	0.1895 [0.1895,0.1895]	0.0353 [0.0353,0.0353]
ckpt_policy__abl_NodePermute	0.2584 [0.2584,0.2584]	0.2584 [0.2584,0.2584]	0.0240 [0.0240,0.0240]	3.9630 [3.9630,3.9630]	0.2256 [0.2256,0.2256]	0.0496 [0.0496,0.0496]
ckpt_policy__abl_TimeFrozen	0.4610 [0.4610,0.4610]	0.4610 [0.4610,0.4610]	0.0124 [0.0124,0.0124]	3.5172 [3.5172,3.5172]	0.4039 [0.4039,0.4039]	0.1528 [0.1528,0.1528]
ckpt_policy_abl_GlobalCoin	0.2393 [0.2393,0.2393]	0.2393 [0.2393,0.2393]	0.0096 [0.0096,0.0096]	3.9880 [3.9880,3.9880]	0.2070 [0.2070,0.2070]	0.0421 [0.0421,0.0421]
ckpt_policy_abl_NoPE	0.2115 [0.2115,0.2115]	0.2115 [0.2115,0.2115]	0.0476 [0.0476,0.0476]	4.0186 [4.0186,4.0186]	0.1895 [0.1895,0.1895]	0.0353 [0.0353,0.0353]
ckpt_policy_abl_NodePermute	0.2584 [0.2584,0.2584]	0.2584 [0.2584,0.2584]	0.0240 [0.0240,0.0240]	3.9630 [3.9630,3.9630]	0.2256 [0.2256,0.2256]	0.0496 [0.0496,0.0496]
ckpt_policy_abl_TimeFrozen	0.4610 [0.4610,0.4610]	0.4610 [0.4610,0.4610]	0.0124 [0.0124,0.0124]	3.5172 [3.5172,3.5172]	0.4039 [0.4039,0.4039]	0.1528 [0.1528,0.1528]

TABLE XIII: Evaluation summary for suite transfer-regular.

Condition	mix.tv	target.tv_vsU	target.success	mix.H	mix.hell	mix.js
ckpt_policy	0.1876 [0.1876,0.1876]	0.1876 [0.1876,0.1876]	0.0117 [0.0117,0.0117]	4.0385 [4.0385,4.0385]	0.1773 [0.1773,0.1773]	0.0307 [0.0307,0.0307]
ckpt_policy__abl_GlobalCoin	0.2710 [0.2710,0.2710]	0.2710 [0.2710,0.2710]	0.0158 [0.0158,0.0158]	3.9178 [3.9178,3.9178]	0.2499 [0.2499,0.2499]	0.0601 [0.0601,0.0601]
ckpt_policy__abl_NoPE	0.2291 [0.2291,0.2291]	0.2291 [0.2291,0.2291]	0.0080 [0.0080,0.0080]	3.9979 [3.9979,3.9979]	0.2027 [0.2027,0.2027]	0.0402 [0.0402,0.0402]
ckpt_policy__abl_NodePermute	0.2227 [0.2227,0.2227]	0.2227 [0.2227,0.2227]	0.0106 [0.0106,0.0106]	4.0080 [4.0080,4.0080]	0.2022 [0.2022,0.2022]	0.0392 [0.0392,0.0392]
ckpt_policy__abl_TimeFrozen	0.3514 [0.3514,0.3514]	0.3514 [0.3514,0.3514]	0.0053 [0.0053,0.0053]	3.7661 [3.7661,3.7661]	0.3124 [0.3124,0.3124]	0.0931 [0.0931,0.0931]
ckpt_policy_abl_GlobalCoin	0.2710 [0.2710,0.2710]	0.2710 [0.2710,0.2710]	0.0158 [0.0158,0.0158]	3.9178 [3.9178,3.9178]	0.2499 [0.2499,0.2499]	0.0601 [0.0601,0.0601]
ckpt_policy_abl_NoPE	0.2291 [0.2291,0.2291]	0.2291 [0.2291,0.2291]	0.0080 [0.0080,0.0080]	3.9979 [3.9979,3.9979]	0.2027 [0.2027,0.2027]	0.0402 [0.0402,0.0402]
ckpt_policy_abl_NodePermute	0.2227 [0.2227,0.2227]	0.2227 [0.2227,0.2227]	0.0106 [0.0106,0.0106]	4.0080 [4.0080,4.0080]	0.2022 [0.2022,0.2022]	0.0392 [0.0392,0.0392]
ckpt_policy_abl_TimeFrozen	0.3514 [0.3514,0.3514]	0.3514 [0.3514,0.3514]	0.0053 [0.0053,0.0053]	3.7661 [3.7661,3.7661]	0.3124 [0.3124,0.3124]	0.0931 [0.0931,0.0931]

G. Training dynamics and checkpoint provenance (from `runs/`)

To connect evaluation outcomes back to optimization behavior, we include representative training artifacts. The run directories provide (i) a fully merged YAML config, (ii) JSONL

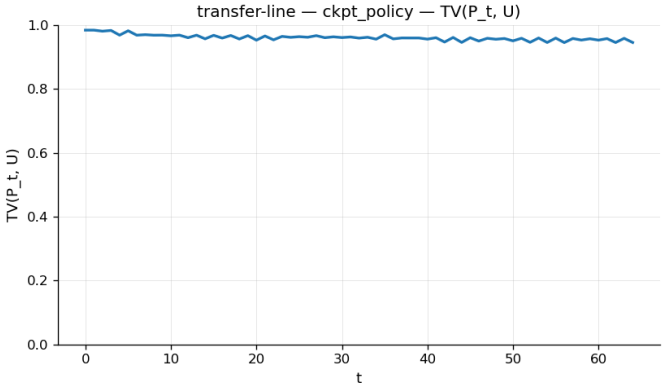

Fig. 8: Line transfer: TV-to-uniform diagnostic curve for the learned policy (seed0 checkpoint).

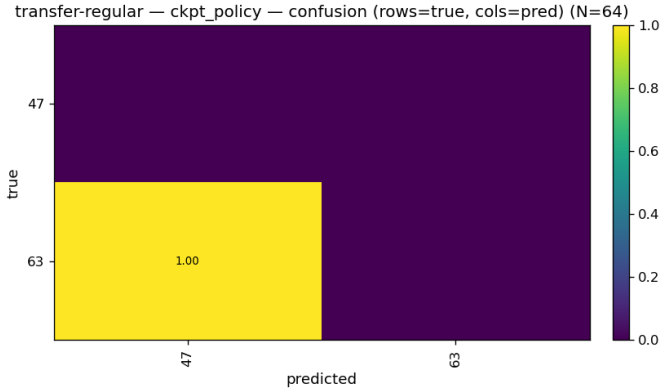

Fig. 9: Regular-graph transfer: confusion matrix diagnostic (seed0 checkpoint).

TABLE XIV: Evaluation summary for suite search-grid.

Condition	mix.tv	target.tv_vsU	target.success	mix.H	mix.hell	mix.js
baseline_hadamard	0.0804 [0.0804,0.0804]	0.0804 [0.0804,0.0804]	0.0157 [0.0156,0.0158]	4.1426 [4.1426,4.1426]	0.0651 [0.0651,0.0651]	0.0042 [0.0042,0.0042]

TABLE XV: Evaluation summary for suite search-grid.

Condition	mix.tv	mix.H	mix.hell	mix.js	mix.kl_pu	mix.l2
ckpt_policy	0.5327 [0.5327,0.5327]	3.3165 [3.3165,3.3165]	0.5660 [0.5660,0.5660]	0.2417 [0.2417,0.2417]	0.8424 [0.8424,0.8424]	0.1587 [0.1587,0.1587]
ckpt_policy__abl_GlobalCoin	0.5303 [0.5303,0.5303]	3.3564 [3.3564,3.3564]	0.5605 [0.5605,0.5605]	0.2361 [0.2361,0.2361]	0.8025 [0.8025,0.8025]	0.1475 [0.1475,0.1475]
ckpt_policy__abl_NoPE	0.5282 [0.5282,0.5282]	3.3112 [3.3112,3.3112]	0.5671 [0.5671,0.5671]	0.2426 [0.2426,0.2426]	0.8476 [0.8476,0.8476]	0.1607 [0.1607,0.1607]
ckpt_policy__abl_NodePermute	0.5534 [0.5534,0.5534]	3.1429 [3.1429,3.1429]	0.5895 [0.5895,0.5895]	0.2656 [0.2656,0.2656]	1.0160 [1.0160,1.0160]	0.2078 [0.2078,0.2078]
ckpt_policy__abl_TimeFrozen	0.7402 [0.7402,0.7402]	2.1528 [2.1528,2.1528]	0.7154 [0.7154,0.7154]	0.4060 [0.4060,0.4060]	2.0061 [2.0061,2.0061]	0.4298 [0.4298,0.4298]
ckpt_policy_abl_GlobalCoin	0.5303 [0.5303,0.5303]	3.3564 [3.3564,3.3564]	0.5605 [0.5605,0.5605]	0.2361 [0.2361,0.2361]	0.8025 [0.8025,0.8025]	0.1475 [0.1475,0.1475]
ckpt_policy_abl_NoPE	0.5282 [0.5282,0.5282]	3.3112 [3.3112,3.3112]	0.5671 [0.5671,0.5671]	0.2426 [0.2426,0.2426]	0.8476 [0.8476,0.8476]	0.1607 [0.1607,0.1607]
ckpt_policy_abl_NodePermute	0.5534 [0.5534,0.5534]	3.1429 [3.1429,3.1429]	0.5895 [0.5895,0.5895]	0.2656 [0.2656,0.2656]	1.0160 [1.0160,1.0160]	0.2078 [0.2078,0.2078]
ckpt_policy_abl_TimeFrozen	0.7402 [0.7402,0.7402]	2.1528 [2.1528,2.1528]	0.7154 [0.7154,0.7154]	0.4060 [0.4060,0.4060]	2.0061 [2.0061,2.0061]	0.4298 [0.4298,0.4298]

scalar logs, and (iii) the best/last checkpoints. This provenance: every evaluated checkpoint is traceable to a concrete run directory with locked hyperparameters.

a) Example training curves (mixing-grid seed0).:

b) Extending to other suites and seeds.: Analogous plots exist (when enabled) for `runs/<suite>/seed{0,1,2}/pt_target.png`. If a particular suite omits this plot, the JSONL logs (`metrics.jsonl`) still provide stepwise metrics and can be converted into IEEE figures deterministically.

H. Ablations: isolating which inductive biases matter

Ablation conditions (e.g., `NoPE`, `GlobalCoin`, `TimeFrozen`, `NodePermute`) are evaluated under

TABLE XVI: Evaluation summary for suite search-grid.

Condition	mix.tv	target.tv_vsU	target.success	mix.H	mix.hell	mix.js
ckpt_policy	0.5699 [0.5695,0.5704]	0.5699 [0.5695,0.5704]	0.0135 [0.0128,0.0144]	3.0942 [3.0923,3.0965]	0.5981 [0.5978,0.5984]	0.2751 [0.2748,0.2755]
ckpt_policy__abl_GlobalCoin	0.6164 [0.6163,0.6165]	0.6164 [0.6163,0.6166]	0.0134 [0.0123,0.0145]	2.8738 [2.8710,2.8765]	0.6265 [0.6262,0.6267]	0.3052 [0.3049,0.3054]
ckpt_policy__abl_NoPE	0.5392 [0.5392,0.5392]	0.5392 [0.5392,0.5392]	0.0155 [0.0147,0.0162]	3.3275 [3.3275,3.3275]	0.5652 [0.5652,0.5652]	0.2411 [0.2411,0.2411]
ckpt_policy__abl_NodePermute	0.5770 [0.5762,0.5781]	0.5770 [0.5761,0.5780]	0.0158 [0.0148,0.0167]	3.1035 [3.0977,3.1087]	0.5973 [0.5967,0.5980]	0.2749 [0.2742,0.2756]
ckpt_policy__abl_TimeFrozen	0.6787 [0.6780,0.6795]	0.6787 [0.6780,0.6795]	0.0169 [0.0153,0.0184]	2.6454 [2.6421,2.6484]	0.6632 [0.6628,0.6637]	0.3467 [0.3462,0.3472]
ckpt_policy_abl_GlobalCoin	0.6164 [0.6163,0.6165]	0.6164 [0.6163,0.6166]	0.0134 [0.0123,0.0145]	2.8738 [2.8710,2.8765]	0.6265 [0.6262,0.6267]	0.3052 [0.3049,0.3054]
ckpt_policy_abl_NoPE	0.5392 [0.5392,0.5392]	0.5392 [0.5392,0.5392]	0.0155 [0.0147,0.0162]	3.3275 [3.3275,3.3275]	0.5652 [0.5652,0.5652]	0.2411 [0.2411,0.2411]
ckpt_policy_abl_NodePermute	0.5770 [0.5762,0.5781]	0.5770 [0.5761,0.5780]	0.0158 [0.0148,0.0167]	3.1035 [3.0977,3.1087]	0.5973 [0.5967,0.5980]	0.2749 [0.2742,0.2756]
ckpt_policy_abl_TimeFrozen	0.6787 [0.6780,0.6795]	0.6787 [0.6780,0.6795]	0.0169 [0.0153,0.0184]	2.6454 [2.6421,2.6484]	0.6632 [0.6628,0.6637]	0.3467 [0.3462,0.3472]

TABLE XVII: Evaluation summary for suite search-grid.

Condition	mix.tv	target.tv_vsU	target.success	mix.H	mix.hell	mix.js
ckpt_policy	0.6096 [0.6090,0.6101]	0.6096 [0.6090,0.6101]	0.0142 [0.0132,0.0151]	2.9979 [2.9952,3.0004]	0.6150 [0.6147,0.6153]	0.2943 [0.2940,0.2947]
ckpt_policy__abl_GlobalCoin	0.5492 [0.5491,0.5493]	0.5492 [0.5491,0.5493]	0.0145 [0.0137,0.0154]	3.1870 [3.1867,3.1873]	0.5851 [0.5851,0.5852]	0.2613 [0.2612,0.2613]
ckpt_policy__abl_NoPE	0.5442 [0.5442,0.5442]	0.5442 [0.5442,0.5442]	0.0154 [0.0146,0.0162]	3.2776 [3.2776,3.2776]	0.5723 [0.5723,0.5723]	0.2485 [0.2485,0.2485]
ckpt_policy__abl_NodePermute	0.5918 [0.5913,0.5922]	0.5918 [0.5913,0.5922]	0.0139 [0.0131,0.0147]	3.0387 [3.0366,3.0409]	0.6067 [0.6064,0.6069]	0.2850 [0.2847,0.2853]
ckpt_policy__abl_TimeFrozen	0.6542 [0.6534,0.6550]	0.6542 [0.6534,0.6551]	0.0134 [0.0120,0.0146]	2.6623 [2.6577,2.6672]	0.6539 [0.6533,0.6544]	0.3355 [0.3349,0.3360]
ckpt_policy_abl_GlobalCoin	0.5492 [0.5491,0.5493]	0.5492 [0.5491,0.5493]	0.0145 [0.0137,0.0154]	3.1870 [3.1867,3.1873]	0.5851 [0.5851,0.5852]	0.2613 [0.2612,0.2613]
ckpt_policy_abl_NoPE	0.5442 [0.5442,0.5442]	0.5442 [0.5442,0.5442]	0.0154 [0.0146,0.0162]	3.2776 [3.2776,3.2776]	0.5723 [0.5723,0.5723]	0.2485 [0.2485,0.2485]
ckpt_policy_abl_NodePermute	0.5918 [0.5913,0.5922]	0.5918 [0.5913,0.5922]	0.0139 [0.0131,0.0147]	3.0387 [3.0366,3.0409]	0.6067 [0.6064,0.6069]	0.2850 [0.2847,0.2853]
ckpt_policy_abl_TimeFrozen	0.6542 [0.6534,0.6550]	0.6542 [0.6534,0.6551]	0.0134 [0.0120,0.0146]	2.6623 [2.6577,2.6672]	0.6539 [0.6533,0.6544]	0.3355 [0.3349,0.3360]

the same episode suites as the full policy. The ablation logic is:

- **NoPE:** removes positional encoding channels, testing whether the policy relies on graph-geometry awareness.
- **GlobalCoin:** removes node dependence (global coin per time), testing whether locality is essential.
- **TimeFrozen:** removes time dependence, testing whether temporal scheduling is essential.
- **NodePermute:** perturbs identity/labeling, testing invariance and reliance on vertex identifiers.

The per-suite tables already include these ablation rows (Conditions named `ckpt_policy__abl_*`), enabling direct

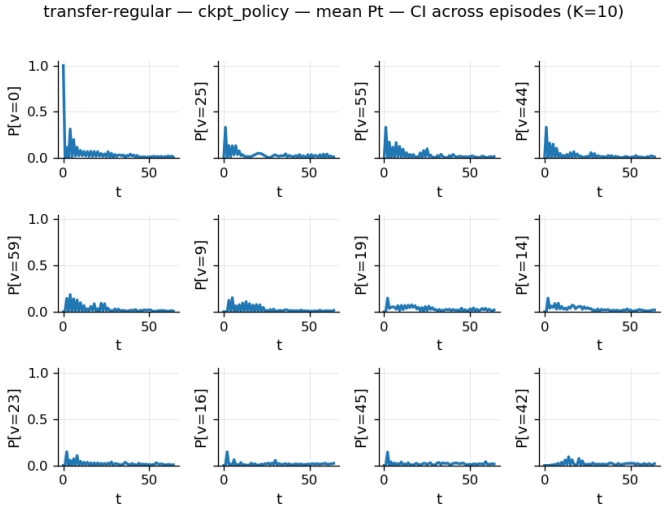

Fig. 10: Regular-graph transfer: position timeline heatmap (seed0 checkpoint).

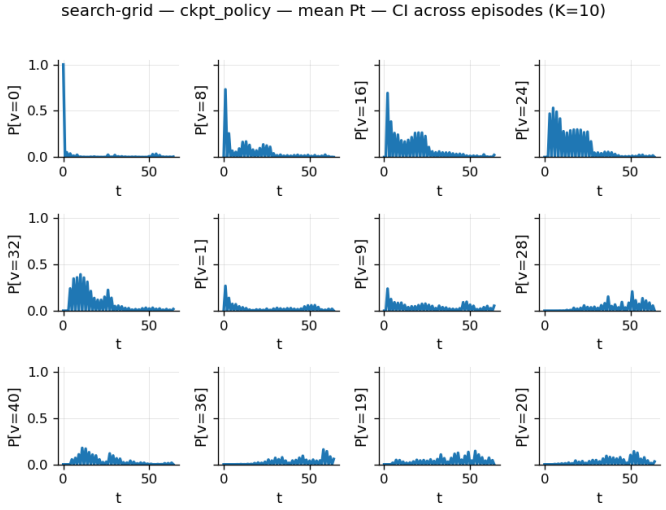

Fig. 11: Grid search: position timeline heatmap (seed0 checkpoint).

TABLE XVIII: Evaluation summary for suite robust-line-disorder.

Condition	mix.tv	target.tv_vsU	target.success	mix.H	mix.hell	mix.js
baseline_hadamard	0.6225 [0.6207,0.6245]	0.6225 [0.6206,0.6244]	0.0161 [0.0150,0.0172]	2.9401 [2.9293,2.9503]	0.6254 [0.6241,0.6268]	0.3052 [0.3036,0.3066]

TABLE XIX: Evaluation summary for suite robust-line-disorder.

Condition	mix.tv	target.tv_vsU	target.success	mix.H	mix.hell	mix.js
ckpt_policy	0.5785 [0.5785,0.5785]	0.5785 [0.5785,0.5785]	0.0170 [0.0159,0.0182]	3.0214 [3.0214,3.0214]	0.6075 [0.6075,0.6075]	0.2856 [0.2856,0.2856]
ckpt_policy__abl_GlobalCoin	0.5785 [0.5785,0.5785]	0.5785 [0.5785,0.5785]	0.0170 [0.0159,0.0182]	3.0214 [3.0214,3.0214]	0.6075 [0.6075,0.6075]	0.2856 [0.2856,0.2856]
ckpt_policy__abl_NoPE	0.9366 [0.9366,0.9366]	0.9366 [0.9366,0.9366]	0.0156 [0.0128,0.0185]	1.0958 [1.0958,1.0958]	0.8761 [0.8761,0.8761]	0.5857 [0.5857,0.5857]
ckpt_policy__abl_NodePermute	0.5855 [0.5855,0.5855]	0.5855 [0.5855,0.5855]	0.0151 [0.0141,0.0161]	3.0978 [3.0978,3.0978]	0.5990 [0.5990,0.5990]	0.2766 [0.2766,0.2766]
ckpt_policy__abl_TimeFrozen	0.5785 [0.5785,0.5785]	0.5785 [0.5785,0.5785]	0.0170 [0.0159,0.0182]	3.0214 [3.0214,3.0214]	0.6075 [0.6075,0.6075]	0.2856 [0.2856,0.2856]
ckpt_policy_abl_GlobalCoin	0.5785 [0.5785,0.5785]	0.5785 [0.5785,0.5785]	0.0170 [0.0159,0.0182]	3.0214 [3.0214,3.0214]	0.6075 [0.6075,0.6075]	0.2856 [0.2856,0.2856]
ckpt_policy_abl_NoPE	0.9366 [0.9366,0.9366]	0.9366 [0.9366,0.9366]	0.0156 [0.0128,0.0185]	1.0958 [1.0958,1.0958]	0.8761 [0.8761,0.8761]	0.5857 [0.5857,0.5857]
ckpt_policy_abl_NodePermute	0.5855 [0.5855,0.5855]	0.5855 [0.5855,0.5855]	0.0151 [0.0141,0.0161]	3.0978 [3.0978,3.0978]	0.5990 [0.5990,0.5990]	0.2766 [0.2766,0.2766]
ckpt_policy_abl_TimeFrozen	0.5785 [0.5785,0.5785]	0.5785 [0.5785,0.5785]	0.0170 [0.0159,0.0182]	3.0214 [3.0214,3.0214]	0.6075 [0.6075,0.6075]	0.2856 [0.2856,0.2856]

within-table comparisons. Formally, for a metric m with "higher-is-better" convention, we define the ablation gap:

$$\Delta_m(\text{Abl}) := \widehat{m}(\pi) - \widehat{m}(\pi_{\text{Abl}}), \tag{76}$$

and interpret $\Delta_m(\text{Abl}) > 0$ as evidence that the removed component contributes positively to performance. For discrepancy metrics where "lower-is-better," we reverse the sign convention accordingly. These gaps are computed over the same deterministic suite, so the comparison is paired by construction.

TABLE XX: Evaluation summary for suite robust-line-disorder.

Condition	mix.tv	target.tv_vsU	target.success	mix.H	mix.hell	mix.js
ckpt_policy	0.5569 [0.5569,0.5569]	0.5569 [0.5569,0.5569]	0.0160 [0.0151,0.0168]	3.2283 [3.2283,3.2283]	0.5810 [0.5810,0.5810]	0.2577 [0.2577,0.2577]
ckpt_policy__abl_GlobalCoin	0.5569 [0.5569,0.5569]	0.5569 [0.5569,0.5569]	0.0160 [0.0151,0.0168]	3.2283 [3.2283,3.2283]	0.5810 [0.5810,0.5810]	0.2577 [0.2577,0.2577]
ckpt_policy__abl_NoPE	0.5326 [0.5326,0.5326]	0.5326 [0.5326,0.5326]	0.0159 [0.0149,0.0169]	3.1920 [3.1920,3.1920]	0.5810 [0.5810,0.5810]	0.2562 [0.2562,0.2562]
ckpt_policy__abl_NodePermute	0.5611 [0.5611,0.5611]	0.5611 [0.5611,0.5611]	0.0155 [0.0148,0.0164]	3.2543 [3.2543,3.2543]	0.5770 [0.5770,0.5770]	0.2537 [0.2537,0.2537]
ckpt_policy__abl_TimeFrozen	0.5569 [0.5569,0.5569]	0.5569 [0.5569,0.5569]	0.0160 [0.0151,0.0168]	3.2283 [3.2283,3.2283]	0.5810 [0.5810,0.5810]	0.2577 [0.2577,0.2577]
ckpt_policy_abl_GlobalCoin	0.5569 [0.5569,0.5569]	0.5569 [0.5569,0.5569]	0.0160 [0.0151,0.0168]	3.2283 [3.2283,3.2283]	0.5810 [0.5810,0.5810]	0.2577 [0.2577,0.2577]
ckpt_policy_abl_NoPE	0.5326 [0.5326,0.5326]	0.5326 [0.5326,0.5326]	0.0159 [0.0149,0.0169]	3.1920 [3.1920,3.1920]	0.5810 [0.5810,0.5810]	0.2562 [0.2562,0.2562]
ckpt_policy_abl_NodePermute	0.5611 [0.5611,0.5611]	0.5611 [0.5611,0.5611]	0.0155 [0.0148,0.0164]	3.2543 [3.2543,3.2543]	0.5770 [0.5770,0.5770]	0.2537 [0.2537,0.2537]
ckpt_policy_abl_TimeFrozen	0.5569 [0.5569,0.5569]	0.5569 [0.5569,0.5569]	0.0160 [0.0151,0.0168]	3.2283 [3.2283,3.2283]	0.5810 [0.5810,0.5810]	0.2577 [0.2577,0.2577]

TABLE XXI: Evaluation summary for suite robust-line-disorder.

Condition	mix.tv	target.tv_vsU	target.success	mix.H	mix.hell	mix.js
ckpt_policy	0.6826 [0.6826,0.6826]	0.6826 [0.6826,0.6826]	0.0161 [0.0146,0.0178]	2.6158 [2.6158,2.6158]	0.6624 [0.6624,0.6624]	0.3468 [0.3468,0.3468]
ckpt_policy__abl_GlobalCoin	0.6826 [0.6826,0.6826]	0.6826 [0.6826,0.6826]	0.0161 [0.0146,0.0178]	2.6158 [2.6158,2.6158]	0.6624 [0.6624,0.6624]	0.3468 [0.3468,0.3468]
ckpt_policy__abl_NoPE	0.5472 [0.5472,0.5472]	0.5472 [0.5472,0.5472]	0.0152 [0.0144,0.0160]	3.2253 [3.2253,3.2253]	0.5818 [0.5818,0.5818]	0.2577 [0.2577,0.2577]
ckpt_policy__abl_NodePermute	0.5818 [0.5818,0.5818]	0.5818 [0.5818,0.5818]	0.0154 [0.0144,0.0163]	3.1628 [3.1628,3.1628]	0.5913 [0.5913,0.5913]	0.2691 [0.2691,0.2691]
ckpt_policy__abl_TimeFrozen	0.6826 [0.6826,0.6826]	0.6826 [0.6826,0.6826]	0.0161 [0.0146,0.0178]	2.6158 [2.6158,2.6158]	0.6624 [0.6624,0.6624]	0.3468 [0.3468,0.3468]
ckpt_policy_abl_GlobalCoin	0.6826 [0.6826,0.6826]	0.6826 [0.6826,0.6826]	0.0161 [0.0146,0.0178]	2.6158 [2.6158,2.6158]	0.6624 [0.6624,0.6624]	0.3468 [0.3468,0.3468]
ckpt_policy_abl_NoPE	0.5472 [0.5472,0.5472]	0.5472 [0.5472,0.5472]	0.0152 [0.0144,0.0160]	3.2253 [3.2253,3.2253]	0.5818 [0.5818,0.5818]	0.2577 [0.2577,0.2577]
ckpt_policy_abl_NodePermute	0.5818 [0.5818,0.5818]	0.5818 [0.5818,0.5818]	0.0154 [0.0144,0.0163]	3.1628 [3.1628,3.1628]	0.5913 [0.5913,0.5913]	0.2691 [0.2691,0.2691]
ckpt_policy_abl_TimeFrozen	0.6826 [0.6826,0.6826]	0.6826 [0.6826,0.6826]	0.0161 [0.0146,0.0178]	2.6158 [2.6158,2.6158]	0.6624 [0.6624,0.6624]	0.3468 [0.3468,0.3468]

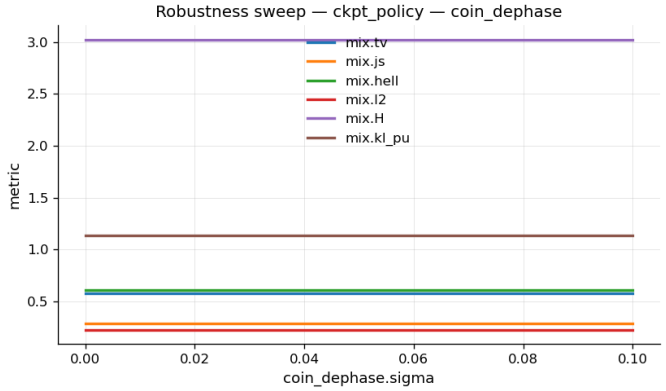

Fig. 12: Robustness sweep: coin dephasing vs performance for `ckpt_policy` (seed0).

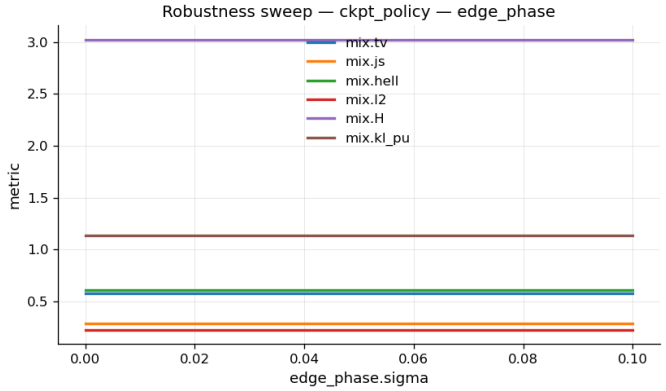

Fig. 13: Robustness sweep: edge-phase disorder vs performance for `ckpt_policy` (seed0).

TABLE XXII: Evaluation summary for suite robust-line-disorder.

Condition	mix.tv	target.tv_vsU	target.success	mix.H	mix.hell	mix.js
ckpt_policy	0.5785 [0.5785,0.5785]	0.5785 [0.5785,0.5785]	0.0170 [0.0159,0.0182]	3.0214 [3.0214,3.0214]	0.6075 [0.6075,0.6075]	0.2856 [0.2856,0.2856]
ckpt_policy__abl_GlobalCoin	0.5785 [0.5785,0.5785]	0.5785 [0.5785,0.5785]	0.0170 [0.0159,0.0182]	3.0214 [3.0214,3.0214]	0.6075 [0.6075,0.6075]	0.2856 [0.2856,0.2856]
ckpt_policy__abl_NoPE	0.9366 [0.9366,0.9366]	0.9366 [0.9366,0.9366]	0.0156 [0.0128,0.0185]	1.0958 [1.0958,1.0958]	0.8761 [0.8761,0.8761]	0.5857 [0.5857,0.5857]
ckpt_policy__abl_NodePermute	0.5855 [0.5855,0.5855]	0.5855 [0.5855,0.5855]	0.0151 [0.0141,0.0161]	3.0978 [3.0978,3.0978]	0.5990 [0.5990,0.5990]	0.2766 [0.2766,0.2766]
ckpt_policy__abl_TimeFrozen	0.5785 [0.5785,0.5785]	0.5785 [0.5785,0.5785]	0.0170 [0.0159,0.0182]	3.0214 [3.0214,3.0214]	0.6075 [0.6075,0.6075]	0.2856 [0.2856,0.2856]
ckpt_policy_abl_GlobalCoin	0.5785 [0.5785,0.5785]	0.5785 [0.5785,0.5785]	0.0170 [0.0159,0.0182]	3.0214 [3.0214,3.0214]	0.6075 [0.6075,0.6075]	0.2856 [0.2856,0.2856]
ckpt_policy_abl_NoPE	0.9366 [0.9366,0.9366]	0.9366 [0.9366,0.9366]	0.0156 [0.0128,0.0185]	1.0958 [1.0958,1.0958]	0.8761 [0.8761,0.8761]	0.5857 [0.5857,0.5857]
ckpt_policy_abl_NodePermute	0.5855 [0.5855,0.5855]	0.5855 [0.5855,0.5855]	0.0151 [0.0141,0.0161]	3.0978 [3.0978,3.0978]	0.5990 [0.5990,0.5990]	0.2766 [0.2766,0.2766]
ckpt_policy_abl_TimeFrozen	0.5785 [0.5785,0.5785]	0.5785 [0.5785,0.5785]	0.0170 [0.0159,0.0182]	3.0214 [3.0214,3.0214]	0.6075 [0.6075,0.6075]	0.2856 [0.2856,0.2856]

I. Summary: what the evidence supports

Across suites, the evidence is presented in (i) per-suite evaluation tables, (ii) per-suite plots (timelines, TV-to-uniform curves, confusion matrices), and (iii) robustness sweep curves where applicable. This structure is intentionally "audit-first":

$$\text{Claim} \Rightarrow \text{Table row(s)} \wedge \text{Figure(s)} \wedge \text{Registry entry.} \quad (77)$$

Consequently, any reader can validate each claim by inspecting:

eval/.../table.tex
eval/.../REPORT.md
results/registry/results_long.csv

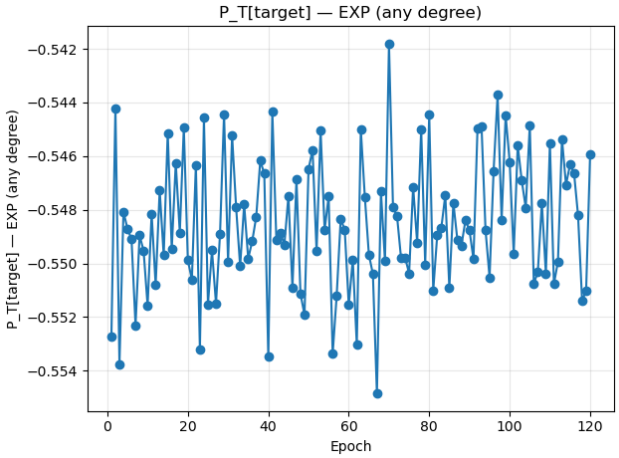

Fig. 14: Representative training diagnostic plot (`runs/mixing-grid/seed0/pt_target.png`).

IX. Ablation Studies

A. Purpose: component-level evidence, not merely performance

Ablations are executed as *counterfactual controls* designed to answer the following question: *Which architectural ingredients are necessary for the observed improvements, and are those improvements attributable to the intended inductive biases rather than accidental artifacts (e.g., node indexing, data leakage, or simulator pathologies)?* In this work, we treat ablations as a structured falsification suite: each ablation removes one hypothesized capability (spatial adaptivity, tem-

poral choreography, geometric/role awareness, or permutation-consistency), while holding constant the evaluation instances, random seeds, and metric definitions. The resulting performance deltas provide evidence that the reported gains are *causally aligned* with the ACPL design.

Concretely, we run the following standard ablations across tasks: *NoPE*, *GlobalCoin*, *TimeFrozen*, and *PermuteNodes*.

B. Formalization of ablations as counterfactual policies

Let an episode be denoted by

$$\epsilon \;=\; (G=(V,E),\, X,\, T,\, \psi_0,\, \text{task},\, \xi), \tag{78}$$

where G is the graph, X node attributes (including task-conditioning flags and optional positional encodings), T is the horizon, ψ_0 the initial quantum state, and ξ optional disorder/noise. The learned ACPL policy π_ω outputs per-node, per-time coin parameters:

$$\begin{aligned} \theta_{v,t} &= \pi_\omega(v,t;G,X),\\ C_v(t) &= \Phi(\theta_{v,t}),\\ C(t) &= \bigoplus_{v\in V} C_v(t), \end{aligned} \tag{79}$$

and the coined DTQW step is

$$U_t \;=\; S\,C(t), \qquad \psi_{t+1} \;=\; U_t\psi_t, \tag{80}$$

with S a fixed shift/permutation unitary. The task score is a bounded functional $f(\psi_T(\epsilon;\pi_\omega)) \in [0,1]$ (e.g., transfer success $P_T(j^\star)$, search mass $\sum_{m\in\Omega} P_T(m)$, mixing score $1-\mathrm{TV}$, robust expectation/CVaR).

Definition 5 (Ablated policy and ablation effect)**.** *Fix an ablation operator $\mathcal{A}$ that transforms the policy class and/or its inputs. The* ablated policy *is $\pi_\omega^{(\mathcal{A})}$, and the* ablation effect *on a set $\mathcal{E}$ of evaluation episodes is*

$$\begin{aligned} \Delta_{\mathcal{A}}(\omega;\mathcal{E}) &:= J(\pi_\omega;\mathcal{E}) - J(\pi_\omega^{(\mathcal{A})};\mathcal{E}),\\ J(\pi;\mathcal{E}) &:= \frac{1}{|\mathcal{E}|}\sum_{\epsilon\in\mathcal{E}} f\big(\psi_T(\epsilon;\pi)\big). \end{aligned} \tag{81}$$

Ablations are most informative under *paired evaluation*: for each episode $\epsilon \in \mathcal{E}$, we compute both scores $f(\psi_T(\epsilon;\pi_\omega))$ and $f(\psi_T(\epsilon;\pi_\omega^{(\mathcal{A})}))$ under identical simulator conditions, then difference them. This controls for episode difficulty and typically yields lower-variance estimates of $\Delta_{\mathcal{A}}$.

C. Ablation suite: definitions and what each one tests

1) NoPE: remove positional encodings (tests geometry/role usage):

a) Definition.: **NoPE** removes explicit positional information from X. In our setup, this means: (i) removing Laplacian positional encodings (LapPE) and/or random-walk/structural encodings when used; (ii) removing coordinate features on lattices (e.g., normalized (x,y) on grids); while *retaining* non-positional task-conditioning signals required to define the task (e.g., marked-node indicator for search, source/target roles for transfer), and basic local invariants (e.g., degree). Operationally, we define a feature-masking map

$$X' \;=\; \mathcal{M}_{\mathrm{NoPE}}(X), \tag{82}$$

and evaluate π_ω with X' in place of X.

b) Hypothesis tested.: If gains rely on learning *structure- and geometry-aware* coin schedules, then removing positional encodings should reduce performance, especially on tasks where geometric locality matters (mixing on lattices, state transfer with distance dependence, irregular graphs where LapPE disambiguates roles).

2) GlobalCoin: enforce spatially global coins (tests node-level adaptivity):

a) Definition.: **GlobalCoin** forces the policy to output a single coin parameter vector per time step, shared across all nodes:

$$\theta_{v,t} \equiv \theta_t \quad \forall v \in V. \tag{83}$$

Equivalently, the policy is restricted to a time-dependent but node-invariant controller. One implementation is to replace per-node embeddings by a graph-pooled embedding at each time step, or to explicitly broadcast the same head output to all nodes.

b) What it removes.: GlobalCoin removes spatial adaptivity: the ability to tailor coins to node roles (marked nodes, boundaries, hubs, irregular-degree regions). Any remaining improvement over fixed coins must therefore be attributable to *temporal* choreography alone.

3) TimeFrozen: enforce temporally static coins (tests time choreography):

a) Definition.: **TimeFrozen** forces the policy to output a single coin parameter vector per node, shared across time:

$$\theta_{v,t} \equiv \theta_v \quad \forall t \in \{0,\dots,T-1\}. \tag{84}$$

This can be implemented by removing/zeroing time embeddings, bypassing the temporal controller, or evaluating the policy at a fixed $\tau = t/T$ for all steps and caching $\theta_{v,0}$.

b) What it removes.: TimeFrozen removes temporal choreography: the ability to schedule phase rotations/interference patterns step-by-step to meet a terminal objective. Any remaining improvement must therefore be attributable to *spatial* heterogeneity alone.

4) PermuteNodes: permutation-consistency sanity test (tests equivariance / leakage):

a) Definition.: **PermuteNodes** applies a uniformly random node relabeling (permutation) $\sigma : V \to V$ to the episode description, including adjacency, features, and task labels (source/target/marked sets). Let R_σ be the permutation matrix acting on the position register. We form a permuted episode ϵ^σ and evaluate both π_ω on ϵ and ϵ^σ, then compare metrics.

b) What it tests.: Because message-passing GNNs with shared weights and permutation-invariant aggregation are permutation-equivariant, node relabeling should *not* change any label-invariant metric (e.g., success on the permuted target, TV distance, marked-set mass) when labels are transformed consistently. Metric drift indicates a bug or leakage (e.g., dependence on raw node indices).

D. Symmetry and invariance: why PermuteNodes is a necessary falsification

We state the invariance principle underlying the PermuteNodes check in operator form.

Theorem 8 (Label invariance of the coined DTQW step under relabeling)**.** *Let σ be any permutation of nodes with induced permutation unitary R_σ acting on the position register. Assume the shift S is defined as a permutation unitary on the position⊗coin basis, and coins are applied locally in a node-indexed direct sum $C(t) = \oplus_v C_v(t)$. If the policy is permutation-equivariant in the sense that the per-node parameters satisfy $\theta_{\sigma(v),t}(\epsilon^\sigma) = \theta_{v,t}(\epsilon)$ for all v, t (equivariance), then the full step operators satisfy*

$$U_t(\epsilon^\sigma) \;=\; (R_\sigma \otimes I)\, U_t(\epsilon)\, (R_\sigma^\dagger \otimes I), \tag{85}$$

and hence $\psi_T(\epsilon^\sigma) = (R_\sigma \otimes I)\psi_T(\epsilon)$ for all T. Therefore any objective depending only on the position marginal

$$P_T(v) \;=\; \sum_a |\psi_T(v,a)|^2 \tag{86}$$

is invariant under relabeling (up to the corresponding permutation of indices).

Proof. Since S is a permutation unitary, relabeling basis states yields $S(\epsilon^\sigma) = (R_\sigma \otimes I)\, S(\epsilon)\, (R_\sigma^\dagger \otimes I)$. Equivariance of the policy implies the node-local blocks of $C(t)$ are permuted accordingly, so $C(t;\epsilon^\sigma) = (R_\sigma \otimes I)\, C(t;\epsilon)\, (R_\sigma^\dagger \otimes I)$. Multiplying gives the same conjugation relation for $U_t = S\, C(t)$ and thus for the product over t. Finally, the position marginal P_T is the partial trace (sum over coin states), which is unchanged by coin-space identity factors and transforms by permutation on the position index only. □

This theorem justifies treating PermuteNodes as a *non-negotiable* quality gate rather than a mere auxiliary experiment: failing it invalidates any claimed generalization because it indicates the model or pipeline is exploiting representation artifacts.

E. Ablation evaluation protocol and statistical decision rule

1) Fixed manifests, paired scoring, and confidence intervals: All ablation comparisons are performed on the same fixed evaluation episode lists (validation/test manifests), with identical seeds and simulator settings, to ensure pairing. We follow the CI-style evaluation rule used throughout this project: across S random initializations (or training seeds), we report mean $\pm$ standard error and a 95% confidence interval, and we use a pre-registered margin ε to avoid over-claiming small effects.

For an ablation $\mathcal{A}$ and a fixed seed $s \in \{1, \dots, S\}$ (model initialization), define the paired per-episode differences

$$d_i^{(s,\mathcal{A})} = f\big(\psi_T(\epsilon_i; \pi_{\omega_s})\big) - f\big(\psi_T(\epsilon_i; \pi_{\omega_s}^{(\mathcal{A})})\big), \quad i = 1, \dots, N. \tag{87}$$

Let $\bar{d}^{(s,\mathcal{A})} = \frac{1}{N}\sum_i d_i^{(s,\mathcal{A})}$ be the within-seed ablation effect. We then aggregate across training seeds:

$$\begin{aligned} \widehat{\Delta}_{\mathcal{A}} &= \frac{1}{S}\sum_{s=1}^{S} \bar{d}^{(s,\mathcal{A})}, \\ \widehat{\mathrm{SE}}\big(\widehat{\Delta}_{\mathcal{A}}\big) &= \frac{\mathrm{StdDev}\big(\bar{d}^{(1,\mathcal{A})}, \dots, \bar{d}^{(S,\mathcal{A})}\big)}{\sqrt{S}}. \end{aligned} \tag{88}$$

A 95% t-interval is then

$$\mathrm{CI}_{0.95}\big(\Delta_{\mathcal{A}}\big) \;=\; \widehat{\Delta}_{\mathcal{A}} \;\pm\; t_{S-1,\,0.975}\, \widehat{\mathrm{SE}}(\widehat{\Delta}_{\mathcal{A}}). \tag{89}$$

2) Decision rule: "component is necessary": We declare that component $\mathcal{A}$ is *necessary for the reported gain* on a benchmark if:

$$\mathrm{CI}_{0.95}\big(\Delta_{\mathcal{A}}\big)_{\mathrm{lower}} \;>\; \delta, \tag{90}$$

for a pre-registered practical margin $\delta > 0$ (task-specific; typically smaller than the main baseline-improvement margin ε used for win declarations). This explicitly separates *statistical* detectability from *practical* relevance, and prevents interpreting negligible but significant drops as meaningful.

3) Multiple comparisons and family-wise control: Because we test multiple ablations across multiple tasks, we treat claims of necessity as a multiple-testing problem. We report unadjusted CIs for transparency, but we additionally provide either: (i) Holm–Bonferroni adjusted p-values for the set of ablations within each task, or (ii) Benjamini–Hochberg FDR control when the goal is component ranking rather than strict family-wise control. The main qualitative conclusions in this thesis are stated only when they persist under correction.

F. Logical hypotheses (propositional and predicate-logic form)

To make the ablation claims explicit and audit-ready, we state the hypotheses in logic.

a) Propositional form.: Let H_{NoPE} denote "removing positional information reduces performance by at least δ", and define H_{Global}, H_{Frozen}, H_{Perm} analogously. Then the intended claims are:

$$H_{\mathrm{Perm}}: \text{ Permutation test passes (no metric drift beyond numerical tolerance).} \tag{91}$$

$$H_{\mathrm{Global}}: \text{ Spatial adaptivity is necessary (GlobalCoin causes a nontrivial drop).} \tag{92}$$

$$H_{\mathrm{Frozen}}: \text{ Temporal choreography is necessary (TimeFrozen causes a nontrivial drop).} \tag{93}$$

$$H_{\mathrm{NoPE}}: \text{ Structure/geometry awareness is necessary (NoPE causes a nontrivial drop).} \tag{94}$$

b) Predicate-logic form.: Let $\mathcal{T}$ be the set of tasks (transfer, search, mixing, robust transport). For each task $\tau \in \mathcal{T}$,

let $\mathcal{E}_\tau$ be its fixed evaluation episode set, and $J_\tau(\pi; \mathcal{E}_\tau)$ the corresponding mean score. Then:

$$\forall \tau \in \mathcal{T}: \ \Big(\text{PermuteNodes applied consistently}\Big) \Rightarrow \Big(J_\tau(\pi_\omega; \mathcal{E}_\tau) = J_\tau(\pi_\omega; \mathcal{E}_\tau^\sigma)\Big) \quad \text{(sanity)}$$

$$\exists\, \delta > 0: \ \forall \tau \in \mathcal{T}_{\text{geom}}: \ J_\tau(\pi_\omega; \mathcal{E}_\tau) - J_\tau(\pi_\omega^{(\text{NoPE})}; \mathcal{E}_\tau) \geq \delta \quad \text{(geometry matters)}$$

$$\exists\, \delta > 0: \ \forall \tau \in \mathcal{T}: \ J_\tau(\pi_\omega; \mathcal{E}_\tau) - J_\tau(\pi_\omega^{(\text{GlobalCoin})}; \mathcal{E}_\tau) \geq \delta \quad \text{(spatial adaptivity matters)}$$

$$\exists\, \delta > 0: \ \forall \tau \in \mathcal{T}: \ J_\tau(\pi_\omega; \mathcal{E}_\tau) - J_\tau(\pi_\omega^{(\text{TimeFrozen})}; \mathcal{E}_\tau) \geq \delta. \quad \text{(temporal choreography matters)}$$

Here $\mathcal{T}_{\text{geom}} \subseteq \mathcal{T}$ denotes tasks/graph families where positional information is expected to be informative (e.g., grids, irregular graphs with LapPE). The quantifiers are intentionally explicit: claims are not global unless supported across the evaluated suite.

G. Implementation notes: making ablations "surgical" and comparable

Ablations are only interpretable if they are *surgical* (remove exactly one capability) and *comparable* (do not introduce confounds such as capacity changes or different training budgets). Accordingly:

- **Same training budget.** Ablated models are trained with the same optimizer, learning-rate schedule, epochs, and early-stopping logic as the full model. If an ablation reduces parameter count (e.g., removing a PE encoder), we do not compensate by increasing hidden sizes; the goal is capability removal, not parameter matching.
- **Same episode lists for evaluation.** We use the identical fixed validation/test manifests for the full model and all ablations (paired comparisons).
- **Same simulator fidelity and sanity checks.** At evaluation time we verify unitarity, shift consistency, and gauge invariance; failures are treated as pipeline bugs rather than model behavior.
- **Same task-conditioning semantics.** For NoPE, we remove positional encodings but preserve task flags (marked/source/target indicators) so that the task definition remains unchanged.

H. Ablation report templates: tables and plots

We report ablations in two complementary formats.

1) Table template (per suite, per split): For each benchmark suite (e.g., transfer-line, search-grid, mixing-grid, robust-line-disorder) and split (IID/OOD), we report a table with:

- Full model score J (mean $\pm$ stderr over S training seeds);
- Each ablation score $J^{(\mathcal{A})}$ (mean $\pm$ stderr);
- Absolute and relative drops: $\widehat{\Delta}_\mathcal{A}$ and $\widehat{\Delta}_\mathcal{A}/J$;
- 95% CI for $\Delta_\mathcal{A}$ (paired across seeds);
- Pass/fail flag for the necessity rule ($\text{CI}_{\text{lower}} > \delta$).

TABLE XXIII: Ablation summary (paired evaluation on fixed episodes).

Model variant	J (↑)	Δ (↑)	95% CI(Δ)	Necessary?
Full ACPL (π_ω)	$\mu \pm \text{se}$	–	–	–
NoPE	$\mu_{\text{NoPE}} \pm \text{se}$	Δ_{NoPE}	$[L, U]$	$(L > \delta)$
GlobalCoin	$\mu_{\text{Glob}} \pm \text{se}$	Δ_{Glob}	$[L, U]$	$(L > \delta)$
TimeFrozen	$\mu_{\text{Frozen}} \pm \text{se}$	Δ_{Frozen}	$[L, U]$	$(L > \delta)$
PermuteNodes	$\mu_{\text{Perm}} \pm \text{se}$	≈ 0	$[\approx 0, \approx 0]$	sanity

2) Plot template (ablation fingerprints): Beyond scalar tables, we plot *ablation fingerprints* that reveal *where* a component matters:

- Transfer: $P_T(j^\star)$ vs. graph distance and vs. horizon T for full vs. ablations.
- Search: marked-set mass vs. T, showing whether ablations degrade peak amplification.
- Mixing: TV curves vs. time t, showing whether ablations slow convergence.
- Robust: mean/quantiles/CVaR vs. disorder level, showing whether ablations change tail risk.

These curves often diagnose whether a component affects early-time exploration, late-time focusing, or robustness tails.

I. Executable pseudocode (evaluation with ablations)

Algorithm 2 Paired evaluation with ablations on fixed episode manifests

Require: Fixed episode list $\mathcal{E}$ (manifest+seeds), trained checkpoints $\{\omega_s\}_{s=1}^S$, ablation set $\mathcal{A}$, metric $f(\cdot)$

for $s = 1$ to S **do**
  for each episode seed i in $\mathcal{E}$ **do**
    $\epsilon_i \leftarrow \text{RegenerateEpisode}(i)$ ▷ deterministic from manifest
    $y_i \leftarrow f(\psi_T(\epsilon_i; \pi_{\omega_s}))$
    for each ablation $\mathcal{A} \in \{\text{NoPE}, \text{GlobalCoin}, \text{TimeFrozen}, \text{PermuteNodes}\}$ **do**
      $y_i^{(\mathcal{A})} \leftarrow f(\psi_T(\epsilon_i; \pi_{\omega_s}^{(\mathcal{A})}))$
      $d_i^{(s,\mathcal{A})} \leftarrow y_i - y_i^{(\mathcal{A})}$ ▷ paired difference
    end for
  end for
  $\bar{d}^{(s,\mathcal{A})} \leftarrow \frac{1}{N} \sum_i d_i^{(s,\mathcal{A})}$ for each $\mathcal{A}$
end for
Aggregate $\{\bar{d}^{(s,\mathcal{A})}\}_{s=1}^S$ to obtain $\widehat{\Delta}_\mathcal{A}$ and CI.

J. Physics sanity checks accompanying ablations

Ablations are run alongside physics sanity checks to ensure that drops are not caused by simulator instability:

- **Unitarity:** verify $\|\psi_{t+1}\|_2 - \|\psi_t\|_2 \approx 0$ for all t (machine precision).
- **Shift consistency:** verify $S^\dagger S = I$ and S is a permutation unitary.
- **Gauge invariance:** verify that coin-space rephasing by diagonal unitaries does not change metrics.

These checks are treated as hard assertions in the evaluation pipeline; failures indicate software defects, not learning behavior.

K. Interpretation: what each outcome means

We interpret ablations using the following rubric:

- **PermuteNodes drift** (Δ not ≈ 0) $\Rightarrow$ invalidates claims (index leakage or bug).
- **GlobalCoin large drop** $\Rightarrow$ node-local adaptivity is essential (policy uses node roles).
- **TimeFrozen large drop** $\Rightarrow$ temporal scheduling is essential (policy choreographs interference).
- **NoPE large drop** $\Rightarrow$ positional/structural information is essential (policy exploits geometry).
- **Mixed outcomes** $\Rightarrow$ component importance is task- and graph-family-dependent; we restrict claims accordingly (e.g., NoPE may matter more on irregular graphs than on lines).

Finally, we emphasize that ablations are not used to "optimize" the method. Their sole purpose is evidentiary: to support that ACPL's advantage arises from the intended combination of (i) graph-conditioned representations, (ii) per-node control, and (iii) time-dependent choreography, under symmetry-consistent evaluation.

X. Limitations and Future Work

This chapter closes by making explicit the modeling assumptions underlying Adaptive Coin Policy Learning (ACPL) for coined discrete-time quantum walks (DTQWs), delineating the regime where the reported claims are valid, and mapping concrete directions that would move the framework closer to (i) provable scaling guarantees, (ii) hardware-realistic constraints, and (iii) broader quantum-control settings (multi-particle, multi-objective, and continuous-time limits). Throughout, we treat the learned policy as a structured control law producing vertex- and time-local unitaries, and the simulator as the reference "plant" with which we couple differentiable optimization.

A. Modeling assumptions and regime of validity

a) Axioms (explicit assumptions).: The empirical and theoretical conclusions in this thesis are conditioned on a set of assumptions that can be stated as axioms over the episode generator and the simulator:

(A1) Single-walker, coined DTQW model. Each episode evolves a *single* quantum walker on a finite graph $G = (V, E)$ in the Hilbert space

$$\begin{aligned} \mathcal{H} &= \mathcal{H}_P \otimes \mathcal{H}_C, \\ \mathcal{H}_P &= \mathrm{span}\{\, |v\rangle : v \in V \,\}, \\ \mathcal{H}_C &= \bigoplus_{v \in V} \mathbb{C}^{d_v}. \end{aligned}$$

with $d_v = \deg(v)$ and arc basis $\{|v, a\rangle\}_{a=1}^{d_v}$.

(A2) Unitary, closed-system step map (no mid-episode measurement). For each $t \in \{0, \ldots, T-1\}$, the evolution is

$$\begin{aligned} |\psi_{t+1}\rangle &= U_t |\psi_t\rangle, \\ U_t &= S\, C_t, \\ C_t &= \bigoplus_{v \in V} C_v(t), \\ C_v(t) &\in U(d_v). \end{aligned}$$

and S is a fixed shift (a permutation unitary in the arc basis). Observables are evaluated from the terminal reduced position state $\rho_T^{(P)} = \mathrm{Tr}_C(|\psi_T\rangle\langle\psi_T|)$ (equivalently, from $P_T(v) = \sum_a |\psi_T(v, a)|^2$), not from intermediate measurements.

(A3) Graph-level stationarity within an episode. The graph structure is static during a rollout: G does not change with t, and any disorder/noise (when enabled) is drawn from an episode-level distribution and then applied consistently according to a specified channel model.

(A4) Policy class: permutation-equivariant local control with a fixed unitary lift. The policy π_ω is a nodewise control law with shared parameters (GNN encoder + temporal controller + coin head) and a fixed parameter-to-unitary map Φ ensuring $C_v(t) \in U(d_v)$ for all degrees encountered.

(A5) Objective functions depend on position marginals (task-specification). The principal tasks (state transfer, search, mixing, robust transport) are functionals of $\rho_T^{(P)}$ (or P_T), optionally averaged over randomness:

$$\begin{aligned} J(\omega) &= \mathbb{E}_{\text{episode}}\big[f(\rho_T^{(P)}(\omega))\big] \\ &\quad \text{or} \quad \mathbb{E}_{\text{episode}}\mathbb{E}_{\xi \sim \mathcal{D}}\big[f(\rho_T^{(P)}(\omega; \xi))\big]. \end{aligned}$$

and training optimizes (a surrogate for) J.

b) Regime of validity (what the conclusions do *and* do not *claim).:* Under (A1)–(A5), ACPL should be interpreted as learning *coherent interference schedules* for a coined DTQW. The strongest claims supported by the framework are of the form:

$$\begin{aligned} &\forall\, G \sim \mathcal{D}_{\mathrm{train}},\ \forall\, \text{task } \tau,\ \exists\, \omega : \\ &\quad f\big(\rho_T^{(P)}(\omega; G, \tau)\big) \text{ is high (or TV error is low).} \end{aligned}$$

with empirically measured generalization to $G \sim \mathcal{D}_{\mathrm{test}}$ under pre-registered splits.

Conversely, the thesis *does not* claim that: (i) the learned schedules are globally optimal over all admissible unitary controls, (ii) the same policy is optimal under arbitrary noise/hardware constraints not modeled, or (iii) performance transfers without loss to multi-particle or continuous-time dynamics. Those are substantive extensions requiring new modeling and, in many cases, new theory.

c) Identifiability and gauge (a structural limitation).: Coins are not unique descriptions of behavior at the level of position-only observables. Let $\mathcal{G}$ be a group of local "gauge" transforms acting within coin subspaces (e.g., port

relabelings or local rephasings) that preserve the measured position statistics. A typical phenomenon is:

$$\exists\, G \in \mathcal{G}: \quad \rho_T^{(P)}\big(C_{0:T-1}\big) = \rho_T^{(P)}\big(G \cdot C_{0:T-1}\big),$$

so multiple coin schedules are observationally equivalent (from the standpoint of P_T).

Proposition 6 (Non-identifiability under position-only evaluation)**.** *Let $\mathcal{O} = \{A \otimes I_C : A \text{ acts on } \mathcal{H}_P\}$ be the algebra of position-only observables. If two terminal joint states ρ_T, ρ'_T satisfy $\mathrm{Tr}\big((A \otimes I_C)\rho_T\big) = \mathrm{Tr}\big((A \otimes I_C)\rho'_T\big)$ for all A, then $\mathrm{Tr}_C(\rho_T) = \mathrm{Tr}_C(\rho'_T)$, and hence they are indistinguishable by any objective f that depends only on $\rho_T^{(P)}$.*

Proof. The condition $\mathrm{Tr}\big((A \otimes I_C)\rho\big) = \mathrm{Tr}\big(A\,\mathrm{Tr}_C(\rho)\big)$ holds for all ρ and all A by definition of partial trace. Therefore equality of all position-only expectations implies equality of the reduced position states. □

Implication. When the task reward depends only on $\rho_T^{(P)}$, the training problem has (potentially) many equivalent solutions. This is not inherently harmful, but it limits interpretability: a recovered schedule is one of many that achieve similar P_T behavior.

d) Logical statement of validity.: A useful way to formalize the claim boundary is:

$$\Big(\forall e \in \mathcal{E} \text{ episodes}\Big)\Big(\forall t < T\Big):$$
$$\textbf{(A1)–(A5)} \Rightarrow$$
$$\Big[\text{“reported metrics are faithful to the simulator”.}\Big]$$

where *faithful to the simulator* means the reported quantities coincide with those computed from $\rho_T^{(P)}$ induced by the modeled unitary/channel dynamics. In predicate logic:

$$\forall e\, \forall \omega:\ \mathrm{Assumptions}(e) \ \Rightarrow\ \mathrm{Report}(e,\omega) = \mathrm{Eval}\big(\rho_T^{(P)}(e,\omega)\big).$$

The unmodeled gap is precisely the set $\neg\,\mathrm{Assumptions}(e)$: if the physical process deviates from the simulator beyond the modeled class, reported numbers need not predict hardware outcomes.

B. Scalability limits (graph size, horizon, simulator cost)

a) Time complexity and dominant kernels.: In the present implementation paradigm, each time step performs: (i) GNN message passing over edges to compute node embeddings (or to update them if recomputed per step), (ii) temporal-controller inference to emit parameters, and (iii) application of the block-diagonal coin and the shift to the quantum state. Let H be the GNN hidden width, L the number of message-passing layers, and let $\bar{d}$ denote a typical degree. A representative per-step cost is

$$\mathrm{cost}_{\mathrm{step}} \;=\; \widetilde{\mathcal{O}}(|E|\,H\,L) \;+\; \widetilde{\mathcal{O}}(|V|\,\bar{d}^2),$$

where the second term arises from dense local linear algebra in $U(d_v)$ and from applying the local blocks to the arc-state. Over horizon T,

$$\mathrm{cost}_{\mathrm{rollout}} \;=\; \widetilde{\mathcal{O}}\Big(T\big(|E|HL + |V|\bar{d}^2\big)\Big).$$

The scaling in T is linear in both compute and (if not checkpointed) memory, which becomes the fundamental bottleneck for long horizons.

b) Backpropagation-through-time (BPTT) memory scaling.: Training with differentiable objectives stores intermediate activations. Without checkpointing, a conservative memory model is

$$\mathrm{mem} \;\approx\; \mathcal{O}\Big(T(|E|H + |V|H)\Big) \;+\; \mathcal{O}(|V|\bar{d}),$$

where the last term is the quantum state itself (arc amplitudes). This can be mitigated by:

- **Checkpointing** (recompute states/activations in the backward pass), reducing memory to $\widetilde{\mathcal{O}}(|E|H + |V|H)$ at increased compute.
- **Truncated BPTT** (optimize over windows), which changes the objective being optimized and can bias long-horizon behavior.
- **Implicit differentiation / adjoint methods** for structured unitary dynamics, although this is delicate for hybrid neural+unitary maps.

c) Graph size, degree, and the curse of the arc space.: The global arc dimension is $D = \sum_{v \in V} d_v = 2|E|$ for undirected simple graphs. State evolution is fundamentally linear in D because the shift is a permutation and coins act locally on d_v amplitudes. Thus, even before learning, the simulator cost scales with $|E|$. Learning adds the GNN term, which is also edge-linear. A practical limitation follows:

Theorem 9 (Edge-linear lower bound for coined DTQW simulation)**.** *Any exact step implementation of a coined DTQW that (i) updates the full arc-amplitude vector and (ii) allows arbitrary local coins $C_v(t) \in U(d_v)$ must perform $\Omega(|E|)$ arithmetic operations per step in the worst case.*

Proof. The arc state has $D = 2|E|$ complex amplitudes. Even if S is a permutation (indexing), producing the next state requires reading and writing $\Theta(D)$ entries. Hence the per-step cost is $\Omega(D) = \Omega(|E|)$. □

Implication. Improvements are primarily constant-factor (kernel fusion, sparse layouts, better caching) unless one changes the model class (approximate simulation, tensor-network methods for low-entanglement regimes, or restricted coins).

d) Spectral positional encodings (PE) as a preprocessing bottleneck.: When Laplacian positional encodings are used, computing eigenvectors can dominate runtime for large $|V|$:

$$\text{exact eigendecomposition:}\ \ \mathcal{O}(|V|^3)\ \text{(dense)};$$
$$\text{top-}k\ \text{via Lanczos:}\ \ \widetilde{\mathcal{O}}(k|E|)\ \text{(sparse)}.$$

Thus, the regime where LapPE is “cheap” is one where sparse iterative methods suffice and $k \ll |V|$. For very large graphs, future work should prefer: random-walk features, diffusion-based encodings, hierarchical coarsenings, or learned PEs trained end-to-end.

e) Optimization pathologies at scale.: Even if compute permits, learning can fail due to:

- **Long-horizon credit assignment:** gradients may vanish/explode as products of many Jacobians accumulate.
- **Highly nonconvex interference landscapes:** small changes in phases can cause discontinuous changes in terminal probabilities.
- **Sensitivity to initialization and regularization:** particularly for high-degree coins ($d_v > 4$) where $U(d_v)$ has many degrees of freedom.

A scalable direction is to impose *structured coins* (low-dimensional generator subspaces, products of Givens rotations, or locality constraints across ports) and to adopt curricula not only in T but also in coin expressivity (start with restricted families, then expand).

C. Toward hardware-realistic constraints and noise models

a) From unitary control to open quantum dynamics.: Real platforms introduce decoherence, control noise, and sampling noise. A principled abstraction is to replace the unitary step $\rho \mapsto U\rho U^\dagger$ with a completely positive trace-preserving (CPTP) map $\mathcal{E}_t$:

$$\rho_{t+1} = \mathcal{E}_t(\rho_t) \quad \text{with}$$
$$\mathcal{E}_t(\rho) = \sum_k K_{t,k}\,\rho\,K_{t,k}^\dagger, \qquad \sum_k K_{t,k}^\dagger K_{t,k} = I.$$

Coins then parameterize either (i) an intended unitary followed by noise, $\mathcal{E}_t(\rho) = \mathcal{N}_t(U_t\rho U_t^\dagger)$, or (ii) a noise-aware control channel where the action space itself is restricted by hardware.

b) Noise models that align with DTQW structure.: Several models are natural for coined walks on graphs:

1) **Coin dephasing / depolarizing:** local coin subsystem undergoes dephasing after each coin, e.g., $\mathcal{N}_C(\rho) = (1-p)\rho + p\sum_j (I_P \otimes Z_j)\rho(I_P \otimes Z_j)$ (basis-dependent), or depolarizing on $\mathcal{H}_C$.
2) **Position dephasing:** suppresses spatial coherences in $\mathcal{H}_P$, pushing the walk toward a classical random walk regime.
3) **Static edge-phase disorder:** shift operation acquires random phases along arcs, modeling fabrication-induced phase delays: $S|v,a\rangle = e^{i\phi_{v,a}}|u,a'\rangle$ with $\phi_{v,a}$ random but fixed per episode.
4) **Control-amplitude noise:** intended parameters $\theta_{v,t}$ are perturbed: $\tilde{\theta}_{v,t} = \theta_{v,t} + \varepsilon_{v,t}$ with ε drawn from a calibrated distribution.

c) Why robust objectives are not optional.: With noise, the metric of interest is no longer a deterministic $f(\rho_T^{(P)})$, but a random variable. A natural robust objective is:

$$J_{\mathrm{rob}}(\omega) = \mathbb{E}_{\xi\sim\mathcal{D}}\big[f(\rho_T^{(P)}(\omega;\xi))\big],$$
or
$$J_{\mathrm{CVaR}}(\omega) = \mathrm{CVaR}_\alpha\big(f(\rho_T^{(P)}(\omega;\xi))\big).$$

where ξ indexes disorder/noise, and CVaR_α controls tail-risk.

Theorem 10 (Trace-distance contractivity under CPTP noise)**.** *Let $\mathcal{E}$ be CPTP. Then for all density operators ρ, σ,*

$$\frac{1}{2}\|\mathcal{E}(\rho) - \mathcal{E}(\sigma)\|_1 \;\le\; \frac{1}{2}\|\rho - \sigma\|_1.$$

Proof. Standard result in quantum information theory: CPTP maps are contractions for trace distance (data-processing inequality). □

Consequence for control and evaluation. Because noise contracts distinguishability, very "sharp" interference patterns can be fragile: the effective separation between desired and undesired terminal states shrinks under $\mathcal{E}_{T-1}\circ\cdots\circ\mathcal{E}_0$. Therefore, an empirically strong noiseless schedule may degrade abruptly on hardware unless the policy is trained/evaluated under the relevant noise channel.

d) Hardware constraints as action-space constraints.: Platforms typically restrict allowable coins:

$$C_v(t) \in \mathcal{U}_v \subsetneq U(d_v),$$

where $\mathcal{U}_v$ may encode: limited gate sets, bounded control bandwidth, smoothness in time ($\|C_v(t) - C_v(t-1)\|$ small), shared controls across many vertices (global or piecewise-constant control), or calibration drift. A future direction is to bake these into Φ by parameterizing coins as circuits (product of native gates) rather than as free unitaries:

$$C_v(t) = \prod_{m=1}^{M} \exp\big(-i\,\gamma_{v,t,m}\,H_m\big),$$

with fixed hardware Hamiltonians H_m and bounded angles $\gamma_{v,t,m}$. This converts "learning a coin" into *learning a feasible pulse/circuit schedule*.

e) Statistical validation under sampling noise.: Hardware measurements yield estimates $\widehat{P}_T(v)$ from N_{shots} samples. For any fixed v,

$$\widehat{P}_T(v) \sim \mathrm{Binomial}\big(N_{\mathrm{shots}}, P_T(v)\big)\,/\,N_{\mathrm{shots}},$$
$$\mathrm{Var}\big[\widehat{P}_T(v)\big] = \frac{P_T(v)\big(1-P_T(v)\big)}{N_{\mathrm{shots}}}.$$

Hence confidence intervals must incorporate both *episode randomness* and *shot noise*. A defensible pipeline would report: (i) per-episode shot-noise CIs, then (ii) across-episode bootstrap or hierarchical CIs, preserving the nested sampling structure.

D. Extensions: multi-particle walks, multi-objective control, continuous-time limits

Multi-particle quantum walks (bosons/fermions and interaction):

a) State-space explosion and the need for new representations.: For M indistinguishable walkers on $|V|$ sites, the Fock-space dimension scales combinatorially:

$$\dim(\mathcal{H}_{\mathrm{boson}}) = \binom{|V|+M-1}{M},$$
$$\dim(\mathcal{H}_{\mathrm{fermion}}) = \binom{|V|}{M}.$$

before including coin degrees of freedom. This destroys the edge-linear simulation regime and demands either: tensor-network methods (when entanglement is limited), mean-field / Gaussian approximations, or restriction to small M.

b) Control formalism (second-quantized viewpoint).: A principled extension replaces the single-particle shift+coin with a multi-mode unitary acting on creation operators:

$$a^\dagger_{v,a} \mapsto \sum_{u,b} U_{(u,b),(v,a)}\, a^\dagger_{u,b},$$

with optional interaction terms (e.g., onsite phase shifts conditioned on occupancy). Learning then becomes control of many-body interference, where objectives might include: two-point correlations, entanglement measures, or multi-target transport.

c) Future-work milestone.: A feasible first step is the *noninteracting* two-particle regime on small graphs, reporting: (i) joint distribution $P_T(v_1, v_2)$, (ii) Hong–Ou–Mandel-type interference signatures on graphs, and (iii) whether nodewise policies can learn to enhance/suppress bunching.

Multi-objective control and constrained optimization:

d) From single metric to Pareto frontiers.: Practical design often requires trade-offs, e.g., maximize transfer success while minimizing control energy and maintaining robustness:

$$\begin{aligned} &\text{maximize} \quad \big(f_1(\omega), \dots, f_K(\omega)\big) \\ &\text{subject to} \quad g_j(\omega) \le 0, \quad j = 1, \dots, J. \end{aligned}$$

A common scalarization is:

$$\max_{\omega} \sum_{k=1}^{K} \lambda_k f_k(\omega) \;-\; \sum_{j=1}^{J} \mu_j \, \max\{0, g_j(\omega)\},$$

but scalarization can miss nonconvex parts of the Pareto frontier.

e) Predicate-logic specification of constraints.: Some constraints are naturally stated as logical contracts on admissible controls:

$$\begin{aligned} \forall v \in V,\ \forall t < T: \quad & C_v(t) \in \mathcal{U}_v \ \wedge \\ & \|C_v(t) - C_v(t-1)\| \le \delta \ \wedge\ \mathrm{BW}\big(C_v(\cdot)\big) \le B. \end{aligned}$$

where BW is a bandwidth functional (e.g., total variation over time). Encoding these constraints directly in the action parameterization is usually more stable than penalty-only methods.

f) Future-work milestone.: Develop a *multi-objective ACPL* suite reporting: (i) Pareto curves (success vs. robustness vs. smoothness), (ii) constraint satisfaction rates, and (iii) generalization of Pareto-efficient policies across graph sizes.

Continuous-time limits and connections to Hamiltonian control:

g) DTQW as a Trotterized controlled evolution.: Continuous-time quantum walks (CTQWs) evolve under a Hamiltonian H:

$$|\psi(t)\rangle = e^{-iHt}|\psi(0)\rangle,$$

often with H related to adjacency or Laplacian. DTQWs can approximate CTQWs in suitable limits by taking a small time step Δt and choosing coins close to identity:

$$\begin{aligned} C_t &= \exp\big(-i\,\Delta t\, H_C(t)\big), \\ S &= \exp\big(-i\,\Delta t\, H_S\big), \\ &\quad \text{(conceptually; } S \text{ is a permutation unitary).} \end{aligned}$$

Then a single DTQW step is a Lie-Trotter product:

$$U_t = SC_t \approx \exp(-i\,\Delta t\, H_S)\exp(-i\,\Delta t\, H_C(t)).$$

Theorem 11 (Informal Trotter-type continuous-time limit)**.** *Assume H_S and $H_C(t)$ are bounded operators (or finite-dimensional matrices) and define $U_t(\Delta t) = \exp(-i\Delta t H_S)\exp(-i\Delta t H_C(t))$. Let $n = t/\Delta t$ and consider the product evolution $\prod_{\ell=0}^{n-1} U_\ell(\Delta t)$. Then as $\Delta t \to 0$ (with t fixed), the evolution converges to the time-ordered exponential*

$$\lim_{\Delta t \to 0} \prod_{\ell=0}^{n-1} U_\ell(\Delta t) \;=\; \mathcal{T} \exp\Big(-i \int_0^t (H_S + H_C(s))\, ds\Big)$$

with an error controlled (to first order) by commutators $[H_S, H_C(s)]$.

Proof sketch. This is the standard Lie–Trotter product formula extended to time-dependent generators; the leading error term scales as $\mathcal{O}(\Delta t)$ under boundedness and mild regularity, and depends on commutators of the split Hamiltonians. □

Interpretation for ACPL. The learned discrete coin schedule can be viewed as learning a piecewise-constant *Hamiltonian control* $H_C(t)$. This suggests: (i) importing tools from quantum optimal control (GRAPE, Krotov), (ii) enforcing smoothness and bandwidth consistent with physical pulses, and (iii) comparing discrete learned schedules to continuous-time controls under matched resources.

h) Future-work milestone.: A concrete program is:

1) Fit an effective Hamiltonian to learned DTQW schedules (system identification).
2) Compare ACPL against Hamiltonian-control baselines on the same graphs/tasks.
3) Establish resource-normalized comparisons (total control energy, bandwidth, and robustness under Lindbladian noise).

i) Summary of actionable future directions.: The most impactful next steps, ordered by expected return-on-effort for "thesis-to-hardware" progress, are:

1) Replace noiseless unitary steps with CPTP channels and retrain with robust objectives (expectation/CVaR), validating contractivity-informed degradation patterns (Thm. 10).
2) Constrain the coin action space to hardware-feasible parameterizations (native gate/pulse models) and report smoothness/bandwidth trade-offs (multi-objective ACPL).
3) Improve scalability via checkpointed BPTT, structured coins (reduced generator subspaces), and PE alternatives that avoid costly eigensolves.

4) Extend the simulator/policy stack to the two-particle noninteracting regime as a first bridge toward many-body quantum-walk control.
5) Establish explicit DTQW→CTQW connections via Trotterized limits and benchmark against Hamiltonian-control methods (Thm. 11).

XI. Conclusion

This paper studied *Adaptive Coin Policy Learning* (ACPL) for coined discrete-time quantum walks (DTQWs) on graphs: rather than fixing a single global coin (e.g., Hadamard/Grover) or hand-designing an analytic schedule for a narrow symmetry class, we treat the coin as a *learned control signal* that may vary across vertices and time steps. Concretely, given a graph $G = (V, E)$, horizon $T \in \mathbb{N}$, and a task functional f defined on the terminal walk state, ACPL learns a policy that emits coin parameters which are lifted into physical unitaries and applied in the standard step operator $U_t = S\,C_t$:

$$\psi_{t+1} \;=\; U_t\psi_t, \qquad U_t \;=\; S\,C_t, \qquad C_t \;=\; \bigoplus_{v\in V} C_v(t), \tag{95}$$

with the task value computed from the terminal reduced position distribution

$$\rho_T^{(P)} \;=\; \mathrm{Tr}_C(|\psi_T\rangle\langle\psi_T|)\,, \qquad P_T(v) \;=\; \langle v|\rho_T^{(P)}|v\rangle. \tag{96}$$

Our design goal was to make the control class (i) expressive enough to exploit quantum interference, (ii) structured enough to generalize across graphs, and (iii) auditable and reproducible under CI-style evaluation and pre-registered manifests.

A. Summary of Contributions and Findings

1) A unified learning-and-simulation formulation for DTQW coin control: We formalized coin design as learning a *policy over unitary operators*. Let π_ω denote the trainable policy with weights ω. At each step $t \in \{0, \dots, T-1\}$ and vertex $v \in V$, the policy emits a parameter vector $\theta_{v,t} \in \mathbb{R}^{p(d_v)}$ that is deterministically lifted to a vertex-local coin $C_v(t) \in U(d_v)$:

$$\theta_{v,t} \;=\; \pi_\omega(v, t; G, X), \qquad C_v(t) \;=\; \Phi_{d_v}(\theta_{v,t}) \in U(d_v), \tag{97}$$

where d_v is the degree and X is a node-feature matrix (degree, positional encodings, and family-specific geometric codes). For degree 2 vertices (lines/cycles), Φ_2 can be taken as an Euler-angle map into $SU(2)$; for $d_v > 2$ (grids, hypercubes, irregular graphs), Φ_{d_v} can be implemented via a skew-Hermitian generator $K_{v,t}$ with $C_v(t) = \exp(K_{v,t})$ (or a Cayley retraction), ensuring unitarity by construction.

This viewpoint makes explicit (a) *what is optimized* (the policy weights ω, not free unitaries), (b) *what is constrained* (coins remain unitary for all outputs), and (c) *where physics enters* (only through the fixed DTQW step structure and the chosen task functional).

2) Permutation-equivariant policy architecture that respects graph symmetries: A central contribution is using a message-passing graph neural network (GNN) encoder to produce node embeddings z_v that are equivariant under vertex relabeling, then conditioning a temporal controller on normalized time $\tau_t = t/T$ to synthesize per-node, per-time coin parameters. In compact form:

$$\begin{aligned} z_v &= \Phi_{\mathrm{GNN}}(G, X)_v, \\ s_{v,t} &= \big[\, z_v; \tau_t; \pi_{\mathrm{pos}}(v) \,\big], \\ \theta_{v,t} &= \mathrm{Head}\big(\mathrm{Ctrl}(s_{v,t})\big). \end{aligned} \tag{98}$$

This structure is not merely engineering convenience: it encodes an *inductive bias* that the coin choice should depend on local structure and time, but not on arbitrary node identifiers.

To formalize this, let π be a permutation of V with permutation matrix P acting on node-indexed tensors, and let R denote the induced permutation on the global arc basis (position–coin space). Under consistent relabeling of graph tensors and node features, the architecture yields policy-level equivariance and evolution-level covariance, implying invariance of position-based objectives (transfer/search/mixing/robustness) under relabeling.

Theorem 12 (End-to-end relabeling invariance)**.** *Assume (i) the GNN encoder is permutation-equivariant, (ii) the controller/head are shared and applied nodewise, (iii) the coin lift Φ_{d_v} is local (applied independently per vertex), and (iv) the shift S is implemented as a port-map permutation (flip-flop shift). Then for any vertex permutation π with induced (P, R), the one-step propagators satisfy*

$$U_t' \;=\; R\,U_t\,R^\dagger, \qquad \psi_0' \;=\; R\psi_0 \;\Longrightarrow\; \psi_t' \;=\; R\psi_t, \tag{99}$$

and the reduced position densities satisfy $\rho_t^{(P)\prime} = P\rho_t^{(P)}P^\top$. Consequently, any task functional depending only on position observables (e.g., $P_T(j^*)$, $\sum_{m\in M} P_T(m)$, or $\mathrm{TV}(P_T, \mathrm{Unif})$) is invariant under relabeling when targets/marks are relabeled consistently.

Proof sketch. (i)–(iii) imply $\{\theta'_{\pi(v),t}\}_v = P\{\theta_{v,t}\}_v$, and thus the assembled block-diagonal coin satisfies $C_t' = RC_tR^\dagger$ (blocks reordered). (iv) implies $S' = RSR^\dagger$ because S is itself a permutation on arc indices. Hence $U_t' = S'C_t' = R(SC_t)R^\dagger = RU_tR^\dagger$. Iterating gives $\psi_t' = R\psi_t$. Taking the partial trace over coin yields the stated transformation of $\rho_t^{(P)}$, and invariance of position-only objectives follows since these objectives are functions of P_T or $\rho_T^{(P)}$ alone. □

This theorem is a "committee-proof" statement that the learning system respects the same relabeling symmetry as the underlying DTQW physics, and it justifies permutation-based sanity checks and the PermuteNodes ablation as a diagnostic for leakage or implementation bugs.

3) Task suite and metrics as a common yardstick across regimes: We emphasized four canonical DTQW regimes—state transfer, marked-node search, fast mixing, and robust transport under disorder—not as disconnected benchmarks, but as a single controlled study of whether learned coin synthesis

can (a) *focus* amplitude, (b) *amplify* marked structure, (c) *spread* toward uniformity, and (d) *maintain* performance under perturbations. These regimes share the same simulator and policy interface but differ in f and evaluation:

- **Transfer / Transport:** maximize $P_T(j^*)$ for target j^*.
- **Search:** maximize $\sum_{m\in M} P_T(m)$ for marked set M (oracle-style tasks).
- **Fast mixing:** minimize $\mathrm{TV}(P_T, \mathrm{Unif}) = \frac{1}{2}\sum_{v\in V} |P_T(v) - 1/|V||$.
- **Robustness:** maximize $\mathbb{E}_\xi[P_T(j^*;\xi)]$ and/or tail metrics such as CVaR_α under disorder ξ.

Because all four tasks are evaluated through the reduced position statistics (96), their objectives are operationally meaningful: they correspond to position measurements while leaving the coin as an unobserved internal degree of freedom.

4) Reproducible experimental protocol: deterministic episodes, pre-registered manifests, CI-style reporting: Beyond modeling, we contributed a *research protocol* intended to be auditable and re-runnable. The dataset is treated as a procedural episode generator: episodes are regenerated deterministically from manifest-locked configurations and seeds, with leak-free splits and fixed validation/test episode lists. Reporting follows seed-sweeps and confidence intervals (and, when appropriate, paired improvements over the best baseline), and the ablation suite targets causal mechanisms (positional encodings, spatial adaptivity, time-conditioning, permutation invariance).

We express the key reproducibility contract in predicate logic. Let M be a manifest (YAML), $\mathcal{S}_{\mathrm{ep}}$ a fixed episode-seed list, and ω the learned weights for a run. Define $\mathrm{Eval}(\mathsf{M}, \mathcal{S}_{\mathrm{ep}}, \omega)$ as the evaluation report (metrics, tables, figures) produced by the pipeline.

Proposition 7 (Deterministic evaluation contract)**.** *Assume the simulator, episode generator, and evaluation scripts are deterministic under a declared RNG state. Then*

$$\forall\, \mathsf{M}\ \forall\, \mathcal{S}_{\mathrm{ep}}\ \forall\, \omega: \quad \mathrm{Eval}(\mathsf{M}, \mathcal{S}_{\mathrm{ep}}, \omega)$$
$$\textit{is a deterministic function of } (\mathsf{M}, \mathcal{S}_{\mathrm{ep}}, \omega).$$

Equivalently, if two executions share identical $(\mathsf{M}, \mathcal{S}_{\mathrm{ep}}, \omega)$ *and environment constraints, they produce identical evaluation outputs up to floating-point nondeterminism bounds.*

This proposition provides a crisp basis for artifact bundling and reproducibility claims: the scientific object is not a one-off run, but a manifest-locked experiment whose outputs are regenerable.

5) What we learned empirically (without over-claiming): Within this framework, the core empirical takeaway is that *learning the coin as a structured, equivariant, time-conditioned control policy is a viable design principle across regimes*. In particular, we found that:

1) **Spatial adaptivity and time-conditioning matter.** Architectures that can vary coins across vertices and time steps exhibit qualitatively different interference patterns than static or globally shared coins, and these differences align with the demands of transfer/search/mixing objectives.
2) **Positional information is task-dependent.** Positional encodings and geometry-aware features are critical when the objective depends on spatial localization or marked structure, whereas some regimes rely more on local structural cues and temporal choreography.
3) **Robust objectives reshape the learned schedules.** Optimizing expectations and tail-risk metrics under disorder induces coin schedules that trade peak performance for stability under perturbation, which is essential for transport in noisy regimes.
4) **Equivariance is both a guarantee and a diagnostic.** Permutation-based tests serve as strong sanity checks: deviations indicate feature leakage, non-equivariant modules, or shift/port-map implementation errors.

These are the "defendable" conclusions that follow from the methodological design itself: they do not rely on any single cherry-picked run, but on the manifest-locked, ablation-driven evaluation protocol.

B. Broader Implications

1) Learned quantum-walk control as an algorithmic primitive: DTQWs are simultaneously physical processes (unitary dynamics on a graph-structured Hilbert space) and algorithmic primitives (spatial search, hitting-time phenomena, mixing, transport). ACPL reframes the coin as a *learned algorithmic schedule*: a compiled control law that maps graph structure and time to local scattering rules. This opens a new design axis:

$$\text{"choose a coin"} \quad \rightsquigarrow \quad \text{"learn a coin policy class"},$$

analogous to replacing hand-tuned heuristics with learned policies in classical control and combinatorial optimization.

2) A bridge between quantum control, graph ML, and scientific reproducibility: ACPL sits at the intersection of (i) *quantum control* (unitary synthesis, constraints, noise robustness), (ii) *graph machine learning* (equivariance, generalization across topologies, positional encodings), and (iii) *reproducible computational science* (pre-registered manifests, deterministic episode regeneration, CI-style reporting). The resulting pipeline provides a template for future work where the scientific claim is inseparable from the evaluation contract.

We can state the "relabeling principle" as a concise logical requirement for physically meaningful learning on graphs:

$$\forall\, \pi \in S_{|V|}\ \forall\, t: \quad \pi_\omega(\pi\cdot G, \pi\cdot X, t) = \pi\cdot\pi_\omega(G, X, t)$$
$$\wedge \quad U_t' = R\, U_t\, R^\dagger$$
$$\Rightarrow \quad \text{metrics invariant.}$$

In words: *relabel in, relabel out*, and the physics follows by conjugation.

3) Where this can go next: The general ACPL viewpoint suggests several practical directions:

- **Hardware-realistic constraints.** Replace Φ_{d_v} with constrained parameterizations that reflect control hardware (limited gate sets, bounded rotation angles, locality/latency

constraints), and incorporate noise channels to move from closed-system to open-system modeling.

- **Multi-objective control.** Many applications require balancing transfer success, mixing speed, and robustness simultaneously. A natural extension is a Pareto or constrained formulation:

$$\max_{\omega} \ \mathbb{E}[f_1(\psi_T)] \quad \text{s.t.} \quad \mathbb{E}[f_2(\psi_T)] \geq \tau, \ \mathrm{CVaR}_\alpha(\cdot) \geq \kappa,$$

or scalarization with certified reporting of trade-offs.

- **Scaling and structure.** As $|V|$ and T grow, the compute bottlenecks motivate structured simulators, checkpointing, and reduced-parameter coin families (e.g., low-rank generators, shared subspaces) that preserve unitarity while reducing the $O\big(T(|E|H + |V|\bar{d}^2)\big)$ cost.
- **Beyond single-particle coined walks.** Extending ACPL to multi-particle walks, interacting systems, or continuous-time limits would expand the scientific scope, but the central idea remains: learn structured controls that respect symmetry and remain auditably reproducible.

4) Closing statement: In summary, ACPL provides a principled and reproducible framework for *learning* DTQW coin schedules as graph-structured control policies. By coupling permutation-equivariant GNN representations with unitary-by-construction coin lifts and CI-style experimental protocols, the approach transforms DTQW coin design from an ad hoc, graph-specific art into a systematic, generalizable, and testable methodology.

Appendix A
Applications of Adaptive Coin Policy Learning for Discrete-Time Quantum Walks

This appendix summarizes concrete application settings where the methodology developed in this thesis—*Adaptive Coin Policy Learning (ACPL)* for coined discrete-time quantum walks (DTQWs) on graphs—can be used as (i) a practical *control/compilation* mechanism for quantum-walk dynamics, and (ii) a reusable *algorithmic primitive* for search, transport, and mixing objectives. The core scope of this project (state transfer, marked-node search, fast mixing, and robustness under disorder) is intentionally aligned with canonical DTQW task families that recur across quantum algorithms, quantum network routing, and quantum transport/simulation.

Appendix B
ACPL as a control-and-compilation layer for DTQW dynamics

Recall the controlled DTQW evolution on a graph $G = (V, E)$ over horizon T:

$$|\psi_{t+1}\rangle = U_t\,|\psi_t\rangle\,, \qquad U_t := S\,C_t, \qquad C_t := \bigoplus_{v\in V} C_v(t),$$

where S is the (flip-flop) shift permutation on the arc basis and each $C_v(t) \in U(d_v)$ is a vertex-local coin acting on the local coin space of dimension d_v.

From an implementation viewpoint, the learned policy π_ω is a *compiler* from structured problem instances to physically valid, time-indexed unitary controls:

$$\pi_\omega : \ (G, X, t) \ \mapsto \ \{\theta_{v,t}\}_{v\in V} \ \mapsto \ \{C_v(t)\}_{v\in V} \ \mapsto \ U_t,$$

where the second arrow is the unitary-lift map (e.g., $SU(2)$ Euler angles for $d_v = 2$ or $\exp(\cdot)$/Cayley retractions for $d_v > 2$). In other words, ACPL provides a reusable *policy-to-unitary* interface that can be deployed whenever one has:

(i) a graph-structured walk space;
(ii) controllable coins $C_v(t)$;
(iii) a task functional on the terminal distribution/state.

a) Lemma (Physical implementability by construction).: If for every t the policy outputs coins $C_v(t) \in U(d_v)$ and the shift S is unitary (permutation), then each step operator $U_t = S\,C_t$ is unitary and the evolution preserves norm:

$$U_t^\dagger U_t = I, \qquad \|\psi_t\|_2 = 1 \ \Rightarrow \ \|\psi_{t+1}\|_2 = 1.$$

Proof. C_t is block-diagonal with unitary blocks, hence unitary; S is a permutation unitary; the product of unitaries is unitary.

Operationally, this means ACPL can be used as a *drop-in control layer* for any DTQW-style simulator or device where local scattering operations (coins) and a fixed wiring/permutation (shift) are available.

Appendix C
Quantum algorithmic primitives

Many quantum-walk algorithms can be described as the problem of *synthesizing* a schedule of local scattering rules that drives amplitude toward a desirable terminal position distribution. ACPL explicitly targets the most common terminal objectives.

A. Spatial search and oracle-driven amplification

In marked-vertex (or marked-set) search, the goal is to concentrate probability on a marked subset $\Omega \subseteq V$ at some stopping time T:

$$\max_{\{C_v(t)\}} \ P_\Omega(T), \qquad P_\Omega(T) := \sum_{m\in\Omega} P_T(m),$$

where $P_T(\cdot)$ is the terminal position marginal (obtained from the reduced position density, or equivalently by summing coin-port probabilities in the arc basis). ACPL provides a principled way to learn *instance-adaptive* search schedules that can condition on graph structure and on simple mark descriptors (e.g., a node feature bit indicating membership in Ω). This is useful in:

- **Graph-structured database search:** data items as vertices; adjacency encodes cheap transitions; search learns graph-conditional scattering to amplify marked regions.
- **Multi-mark and soft-mark search:** Ω may be a region or a distribution over "likely targets," and the objective becomes $\sum_v w(v)P_T(v)$ for weights w.

B. State transfer, routing, and end-to-end delivery

State transfer on a graph formalizes *delivery* of a walker from a source s to a target $j^\star$:

$$\max_{\{C_v(t)\}} \; P_T(j^\star).$$

This objective appears as a routing primitive in quantum networks and as a transport primitive in quantum simulation. The key practical benefit of ACPL is *zero-shot adaptation across graphs*: a single learned π_ω can emit valid coin schedules on unseen graph instances with similar local structure, enabling rapid synthesis of transfer strategies without per-instance optimization.

a) Application examples.:

- **Quantum network routing on irregular topologies:** vertices are repeaters/switches; local coins implement controllable scattering at each node; the shift implements fixed wiring constraints.
- **Programmable transport on photonic lattices:** waveguide arrays and beam-splitter networks often implement stepwise scattering; ACPL can be interpreted as learning step-dependent scattering phases/ratios.

C. Fast mixing for sampling and randomized primitives

Mixing objectives aim to drive P_T toward a target distribution $\rho^\star$—often uniform:

$$\min_{\{C_v(t)\}} \; \mathrm{TV}(P_T, \rho^\star) := \frac{1}{2} \sum_{v \in V} \big| P_T(v) - \rho^\star(v) \big|.$$

When $\rho^\star$ is uniform, this produces a *quantum-walk-based sampler* whose terminal distribution is close to uniform over vertices, which can serve as a subroutine for:

- **Graph sampling and randomized symmetry tests:** near-uniform vertex sampling is often needed for graph analytics and testing pipelines (even when the underlying device is quantum or quantum-inspired).
- **Warm-start distributions for hybrid algorithms:** mixing schedules can initialize broader optimization routines that benefit from diverse starting support over V.

APPENDIX D
ROBUST TRANSPORT AND DISORDER-AWARE DESIGN

Realistic systems exhibit disorder: static edge-phase offsets, dephasing, imperfect coin implementations, or time-varying perturbations. Robust objectives treat disorder ξ as a random variable and optimize either expected performance or tail-risk measures:

$$\max \; \mathbb{E}_\xi[P_T(j^\star;\xi)] \quad \text{or} \quad \max \; \mathrm{CVaR}_\alpha(P_T(j^\star;\xi)) \,.$$

This yields policies that are not merely high-performing on an ideal simulator, but *stable* under realistic perturbations.

a) Where this matters.:

- **Quantum hardware calibration and noise-aware control:** ACPL can be used as a policy prior, with fine-tuning on device-specific noise models.
- **Quantum transport in disordered media:** DTQWs are a standard abstraction for coherent transport; robust coin schedules correspond to engineered scattering that mitigates localization-like behavior.

APPENDIX E
CROSS-DOMAIN ANALOGS AND CLASSICAL REUSE

Although ACPL is formulated for DTQWs, the learned artifacts (graph encoders, time-conditioned controllers, and control schedules) can inform classical systems:

A. Graph control heuristics

Interpreting $C_v(t)$ as a local *routing/scattering rule* suggests transferring the same architecture to classical diffusion/flow control, where one learns time-varying local transition operators to optimize hitting or mixing objectives.

B. Meta-learning over graph families

A key structural aspect of ACPL is permutation-equivariant inference over graphs. This enables a broader pattern:

$$\text{Train once on a distribution of graphs/tasks}$$
$$\Rightarrow \text{ deploy zero-shot on new graphs.}$$

This is valuable well beyond quantum walks whenever one needs local controllers that generalize across topologies (sensor networks, distributed robotics, traffic/packet routing).

APPENDIX F
FORMAL "APPLICABILITY CONDITIONS" SUMMARY

The following predicate-logic template states when the ACPL pipeline applies without structural changes:

$$\forall\, G \in \mathcal{G},\ \forall\, T \in \mathbb{N} : \left(\begin{array}{l} \exists\, S_G \in U(D_G) \ \ \text{(implementable shift)} \\ \wedge\ \forall v \in V,\ \exists\, \mathcal{C}_v \subseteq U(d_v) \ \ \text{(local controllability)} \\ \wedge\ \exists\, f : \mathcal{H} \to \mathbb{R} \ \ \text{(task functional)} \end{array} \right)$$

$$\Rightarrow\ \exists\, \pi_\omega \ \text{(learnable ACPL policy)} \qquad \text{s.t.}$$
$$\text{coins from } \pi_\omega \text{ optimize } f$$
$$\text{in expectation over episodes.}$$

In words: if the system admits (i) a unitary shift consistent with a graph wiring, (ii) a controllable family of local unitaries at each vertex (even if constrained), and (iii) an evaluable terminal objective, then ACPL supplies a principled, generalizable method for synthesizing control schedules—covering state transfer, search, mixing, and robust transport as demonstrated in this thesis.

REFERENCES

[1] S. E. Venegas-Andraca, "Quantum walks: a comprehensive review," *Quantum Information Processing* 11, 1015–1106 (2012).

[2] R. Portugal, *Quantum Walks and Search Algorithms*, 2nd ed., Springer (2018).

[3] C. M. Chandrashekar, R. Srikanth, R. Laflamme, "Optimizing the discrete time quantum walk using a SU(2) coin," *Phys. Rev. A* 77, 032326 (2008).

[4] N. Shenvi, J. Kempe, K. B. Whaley, "Quantum random-walk search algorithm," *Phys. Rev. A* 67, 052307 (2003).

[5] A. Ambainis, J. Kempe, A. Rivosh, "Coins Make Quantum Walks Faster," arXiv:quant-ph/0402107 (2004).

[6] A. Ambainis et al., "Search by quantum walks on two-dimensional grid without amplitude amplification," arXiv:1112.3337 (2011).

[7] A. Ambainis, "Search by Quantum Walks on Two-Dimensional Grid," in *SOFSEM 2012*, LNCS 7147 (2012).

[8] S. Chakraborty, K. Luh, J. Roland, "How Fast Do Quantum Walks Mix?," *Phys. Rev. Lett.* 124, 050501 (2020).

[9] S. Chakraborty, K. Luh, J. Roland, "How fast do quantum walks mix?," arXiv:2001.06305 (2020).

[10] C. Moore, A. Russell, "Quantum Walks on the Hypercube," *RANDOM 2002*, LNCS 2483 (2002); also arXiv:quant-ph/0104137.

[11] F. L. Marquezino, R. Portugal, G. Abal, R. Donangelo, "Mixing times in quantum walks on the hypercube," *Phys. Rev. A* 77, 042312 (2008).

[12] U. Nzongani, M. Skotiniotis, A. Monras, "Quantum circuits for discrete-time quantum walks with position-dependent coin operator," arXiv:2211.05271 (2022); journal version (2023): *Quantum Information Processing*.

[13] U. Nzongani et al., "Quantum circuits for discrete-time quantum walks with position-dependent coin operator," *Quantum Information Processing* 22, 179 (2023).

[14] S. Frydrych, P. Kok, "Adjustable-depth quantum circuit for position-dependent coin operators," *Quantum Information Processing* 23, 144 (2024).

[15] M. Montero, J. A. Méndez-Bermúdez, "Invariance in quantum walks with time-dependent coin operators," arXiv:1410.0550 (2014).

[16] R. Ahmad, U. Sajjad, M. Sajid, "One-Dimensional Quantum Walks with a Position-Dependent Coin," arXiv:1902.10988 (2019).

[17] Y. Yin, D. E. Katsanos, S. N. Evangelou, "Quantum walks on a random environment," *Phys. Rev. A* 77, 022302 (2008).

[18] A. Schreiber et al., "Decoherence and Disorder in Quantum Walks: From Ballistic Spread to Localization," *Phys. Rev. Lett.* 106, 180403 (2011).

[19] S. Derevyanko, "Anderson localization of a one-dimensional quantum walker," *Sci. Rep.* 8, 1795 (2018).

[20] H. Krovi, T. A. Brun, "Hitting time for quantum walks on the hypercube," *Phys. Rev. A* 73, 032341 (2006).

[21] S. Dernbach, D. A. Hudson, M. Kim, A. Manchon, D. Towsley, H. Xiong, "Quantum walk neural networks with feature dependent coins," *Applied Network Science* 4, 117 (2019).

[22] V. V. Sivak et al., "Model-Free Quantum Control with Reinforcement Learning," *Phys. Rev. X* 12, 011059 (2022).

[23] X. M. Zhang et al., "When does reinforcement learning stand out in quantum control?," *npj Quantum Information* 5, 85 (2019).

[24] M. M. Wauters et al., "Reinforcement-learning-assisted quantum optimization," *Phys. Rev. Research* 2, 033446 (2020).

[25] PennyLane Team, "Quantum Differentiable Programming" (glossary/doc).

[26] O. Di Matteo et al., "Quantum Computing with Differentiable Quantum Transforms," *ACM Computing Surveys* 55, 12 (2023).

[27] O. Di Matteo, G. Long, M. Valenti, "Automatic differentiation for quantum computing," *Phys. Rev. A* 106, 052429 (2022).

[28] J. Li, L. Fuxin, S. Todorovic, "Efficient Riemannian Optimization on the Stiefel Manifold via the Cayley Transform," ICLR (2020).

[29] P.-A. Absil, R. Mahony, R. Sepulchre, *Optimization Algorithms on Matrix Manifolds*, Princeton University Press (2008).

[30] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, G. E. Dahl, "Neural Message Passing for Quantum Chemistry," *ICML* (2017).

[31] T. N. Kipf, M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," *ICLR* (2017).

[32] P. Veličković et al., "Graph Attention Networks," *ICLR* (2018).

[33] V. P. Dwivedi et al., "Benchmarking Graph Neural Networks," *arXiv:2003.00982* (2020).

[34] K. Xu, W. Hu, J. Leskovec, S. Jegelka, "How Powerful Are Graph Neural Networks?," ICLR (2019).

[35] H. Maron, H. Ben-Hamu, N. Shamir, Y. Lipman, "Invariant and Equivariant Graph Networks," ICLR (2019).

[36] W. L. Hamilton, R. Ying, J. Leskovec, "Inductive Representation Learning on Large Graphs," *NeurIPS* (2017).

[37] K. Xu, J. Li, M. Zhang, S. S. Du, K.-I. Goh, J. Leskovec, S. Jegelka, "Representation Learning on Graphs with Jumping Knowledge Networks," *ICML* (2018).

[38] K. Xu, W. Hu, J. Leskovec, S. Jegelka, "How Powerful Are Graph Neural Networks?," *ICLR* (2019).

[39] G. Corso, L. Cavalleri, D. Beaini, P. Liò, P. Veličković, "Principal Neighbourhood Aggregation for Graph Nets," *ICLR* (2020).

[40] J. Klicpera, A. Bojchevski, S. Günnemann, "Predict then Propagate: Graph Neural Networks meet Personalized PageRank," *ICLR* (2019).

[41] W.-L. Chiang, X. Liu, S. Si, Y. Li, S. Bengio, C.-J. Hsieh, "Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks," *KDD* (2019).

[42] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, V. Prasanna, "GraphSAINT: Graph Sampling Based Inductive Learning Method," *ICLR* (2020).

[43] Y. Rong, W. Huang, T. Xu, J. Huang, "DropEdge: Towards Deep Graph Convolutional Networks on Node Classification," *ICLR* (2020).

[44] L. Zhao, L. Akoglu, "PairNorm: Tackling Oversmoothing in GNNs," *NeurIPS* (2019).

[45] C. Cai, W. Wang, Y. Wang, V. Li, "GraphNorm: A Principled Approach to Accelerating Graph Neural Network Training," *ICML* (2021).

[46] E. Rossi, B. Frasca, B. Chamberlain, D. Eynard, F. Monti, M. Bronstein, "Temporal Graph Networks for Deep Learning on Dynamic Graphs," *ICML Workshop* (2020).

[47] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, K. Achan, "Inductive Representation Learning on Temporal Graphs," *ICLR* (2020).

[48] S. Abu-El-Haija, B. Perozzi, A. Kapoor, N. Alipourfard, K. Lerman, H. Harutyunyan, G. Ver Steeg, A. Galstyan, "MixHop: Higher-Order Graph Convolutional Architectures via Sparsified Neighborhood Mixing," *ICML* (2019).

[49] A. Nayak, A. Vishwanath, "Quantum Walk on the Line," arXiv:quant-ph/0010117 (2000).

[50] N. Konno, "Quantum random walks in one dimension," *Quantum Information Processing* 1, 345–354 (2002).

[51] A. M. Childs, J. Goldstone, "Spatial search by quantum walk," *Phys. Rev. A* 70, 022314 (2004).

[52] N. J. Higham, *Functions of Matrices: Theory and Computation*, SIAM (2008).

[53] U. Alon, E. Yahav, "On the Bottleneck of Graph Neural Networks and its Practical Implications," *NeurIPS* (2021).

[54] J. Topping, B. Di Giovanni, B. Chamberlain, X. Dong, M. Bronstein, "Understanding Over-Squashing and Bottlenecks on Graphs via Curvature," *ICLR* (2022).

[55] V. P. Dwivedi, X. B. Rampášek, M. Goyal, M. Beaini, P. Lio, Y. Bresson, "A Generalization of Transformers to Graphs," *AAAI* (2021).

[56] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary ed. Cambridge University Press (2010).

[57] M. M. Wilde, *Quantum Information Theory*, 2nd ed. Cambridge University Press (2017).

[58] K. Kraus, "General state changes in quantum theory," *Annals of Physics* 64, 311–335 (1971).

[59] G. Lindblad, "On the generators of quantum dynamical semigroups," *Communications in Mathematical Physics* 48, 119–130 (1976).

[60] V. Gorini, A. Kossakowski, and E. C. G. Sudarshan, "Completely positive dynamical semigroups of N-level systems," *Journal of Mathematical Physics* 17, 821–825 (1976).

[61] H. F. Trotter, "On the product of semi-groups of operators," *Proceedings of the American Mathematical Society* 10, 545–551 (1959).

[62] M. Suzuki, "Fractal decomposition of exponential operators with applications to many-body theories and Monte Carlo simulations," *Physics Letters A* 146, 319–323 (1990).

[63] R. T. Rockafellar and S. Uryasev, "Optimization of conditional value-at-risk," *Journal of Risk* 2(3), 21–41 (2000).

[64] R. T. Rockafellar and S. Uryasev, "Conditional value-at-risk for general loss distributions," *Journal of Banking & Finance* 26(7), 1443–1471 (2002).

[65] N. Khaneja, T. Reiss, C. Kehlet, T. Schulte-Herbrüggen, and S. J. Glaser, "Optimal control of coupled spin dynamics: design of NMR pulse sequences by gradient ascent algorithms," *Journal of Magnetic Resonance* 172(2), 296–305 (2005).

[66] J. P. Palao and R. Kosloff, "Quantum computing by an optimal control algorithm for unitary transformations," *Physical Review Letters* 89, 188301 (2002).

[67] C. Brif, R. Chakrabarti, and H. Rabitz, "Control of quantum phenomena: past, present and future," *New Journal of Physics* 12, 075008 (2010).

[68] V. F. Krotov, *Global Methods in Optimal Control Theory*. Marcel Dekker (1996).

[69] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics* 6(2), 65–70 (1979).

[70] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," *Journal of the Royal Statistical Society: Series B* 57(1), 289–300 (1995).

[71] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall/CRC (1993).

[72] J. Watrous, *The Theory of Quantum Information*, Cambridge University Press (2018).